

SQLST

Sürüm 0.29.0 — Kullanım El Kitabı

Hafif, hızlı ve korumalı **çok motorlu** veritabanı sorgu & yönetim aracı

LST · 2026

SQL Server · PostgreSQL · MySQL / MariaDB · Oracle · MongoDB

1. Amaç ve Felsefe

SQLST, SQL Server Management Studio'nun (SSMS) kurulamadığı ya da fazla ağır kaldığı ortamlar için geliştirilen, tek dosyalık, kurulumu yönetici izni istemeyen bir veritabanı sorgu ve yönetim aracıdır. Günlük işin %95'ini — sorgu çalıştırma, nesne inceleme, SP düzenleme, veri dışa aktarma — SSMS'ten daha hızlı ve daha güvenli yapmayı hedefler.

0.3.0 ile gelen ana yenilik çoklu motor desteğidir: aynı arayüzden SQL Server, PostgreSQL, MySQL/MariaDB, Oracle ve MongoDB sunucularına bağlanabilirsiniz. Bağlantı ekranında motoru seçersiniz; gerisi — nesne gezgini, sorgu sekmeleri, sonuç grid'i, dışa aktarma, geçmiş — alışık olduğunuz gibi çalışır.

> **0.29.0'da ne değişti — ■ Durdur #temp'i korur · 🧱 kolon seçme:**
>
> - ■ **Durdur bağlantıyı korur:** çalışan sorguyu durdurduğunuzda sekmenin bağlantısı artık kapanmaz — **#temp tablolarınız ve SET ayarlarınız yerinde kalır** (SSMS gibi). Sunucu iptale birkaç saniyede yanıt vermezse bağlantı eskisi gibi yenilenir ve mesaj bunu söyler.
> - 🧱 **Kolon seçme (SSMS gibi):** sonuç grid'inde kolon **başlığına tıklamak kolonun tamamını seçer** — Ctrl+tık kolon ekler, Shift+tık aralık seçer, Ctrl+C kopyalar, durum satırında toplam/ortalama görünür. **Sıralama artık başlıkta sağ tık menüsünde** (↑ Artan / ↓ Azalan / ✕ Sıralamayı kaldır); sıralı kolonda ▲/▼.

> **0.28.0'da ne değişti — ⏱ SQL Agent · 🔎 Full-Text yönetimi · 🪟 yeni pencereler · 🎨 yeni görünüm:**
>
> - ⏱ **SQL Agent (YENİ, SQL Server):** sol menüdeki **SQL Agent** — job'ların son/sonraki koşusu, sonuç rozetleri ("Hata · adım 2"), çalışan job'un canlı süresi, geçmiş ve tam hata mesajı, adımlar ve Türkçe zamanlama cümlesi. **Başlat** (baştan ya da seçili adımdan) · **Durdur** · **Aç/Kapat** · **Script olarak al**. **+ Yeni job** · ✎ **Düzenle** · ⧉ **Kopyasını oluştur** sihirbazı script üretir — düzenleme job'u silmeden günceller (geçmiş korunur). Express/LocalDB'de Agent olmadığını açıkça söyler. Ayrıntı: Bölüm 36.
> - 🔎 **Full-Text yönetimi:** FTS ekranında Index'ler · Kataloglar · Stoplist'ler sekmeleri — tam/artımlı doldurma, durdurma, değişiklik izleme (AUTO/MANUAL/OFF), katalog birleştirme; doldurma sürerken ekran kendiliğinden tazelenir.
> - 🪟 **Yeni onay/uyarı pencereleri:** Windows'un düz kutuları yerine temaya uyan SQLST pencereleri — düğmeler ne olacağını söyler ("▶ Başlat", "Yine de çalıştır", "🗑 Sil"); geri alınamaz işlerde düğme kırmızıdır ve Enter "Vazgeç"e basar; hatalarda kısa açıklama + kopyalanabilir ayrıntı.
> - 🎨 **Yeni görünüm:** arayüz ve tablolar Segoe UI; sorgu sekmeleri Visual Studio tarzı; tablolarda hücre çizgileri; durumlar renkli rozet (FTS, Profiler, denetim, REST karşılaştırma).

0.27.0'da ne değişti — 📑 ham değerler · 🔎 Full-Text Search · 🌐 yeni REST · 🎨 Antrasit teması:

- 📑 **Değerler veritabanında nasılsa öyle (önemli):** tarih, sayı ve diğer alanlar artık bilgisayarın bölge ayarına göre biçimlenmiyor — SSMS ile birebir: `2026-10-05 14:23:11.123`, `1250.75`, bit `1/0`, GUID büyük harf. Grid, Ctrl+C, kopyala, CSV, rapor ve hücre görüntüleyici aynı metni verir; edit modunda ondalık için nokta da virgül de doğru okunur.
- 🔎 **Full-Text Search (YENİ, SQL Server):** sol menüdeki **Full-Text** — FTS index envanteri, kelime/önek/yakınlık/çekim/serbest metin arama yardımcısı (skorlu sıralama dahil) ve index kurulum sihirbazı (script üretir, Güvenli Yazma ile siz çalıştırırsınız). **Veri Arama** da FTS'li tablolarda kendiliğinden CONTAINS kullanır (⚡ FTS) — uzun metnin içinde geçen değerleri de bulur. Ayrıntı: Bölüm 35.
- 🌐 **REST istemcisi yenilendi (Postman düzeni):** URL'ye `?ad=değer` yazınca parametre tablosu kendiliğinden dolar (iki yönlü), renkli durum rozetleri, Çerezler sekmesi, sekmeli koleksiyonlar/geçmiş.
- 🕸️ **Kayıt Haritası:** haritanın sonucundaki bir satırdan tekrar harita açınca artık o satırın KENDİ kaydı geliyor (talep → belge → belgenin altındakiler... sınırsız iniş); çok referanslı ana tablolar artık yanlışlıkla "tanım tablosu" sayılıp atlanmıyor.
- ⧉ **Ctrl+C:** hücrede Ctrl+C değerin tamamını, kırpmadan ve sona satır sonu eklemeden kopyalar.
- 📁 **.sql dosyaları Gezgin'den:** sağ tık → Birlikte aç → SQLST (varsayılan uygulamanızı değiştirmez).
- 🎨 **Antrasit + Turkuaz teması (YENİ):** klasik düzen — antrasit bağlantı bandı, kutulu sekmeler, düz düğmeler, ince çizgili tam ızgara grid; açık ve koyu.

0.26.1'de ne değişti — 🔍 Profiler saha düzeltmeleri + 🎨 açık VS rayı:

- 🔍 **Profiler akıcılık:** canlı akışta ekran her 2 saniyede baştan kurulmuyor — takılma ("donma") giderildi; **seçtiğiniz satır, kaydırma konumu ve tam-metin paneli akış sürerken artık kaybolmaz**.
- 🔍 **Temiz akış:** SQLST kendi izleme trafiğini (2 sn'lik okuma sorgusu, bağlantı havuzu sıfırlamaları) akışa yazmaz — gördüğünüz her satır gerçek uygulama trafiğidir.
- 🔍 **Bağlantı değişimi:** Profiler açıkken başka sunucuya geçiş, eski sunucu yanıt vermese bile takılmaz (izleme oturumu temizliği kısa tavanla arkada tamamlanır).
- 🎨 **Visual Studio (SSMS) açık teması:** sol menü (ray) artık gri değil **SQL Server mavisi** — koyu kanat değişmedi.

0.26.0'da ne değişti — 🔍 SQL Profiler (YENİ araç — 5 parça):

- 🔍 **Canlı sorgu izleme:** sol raydaki yeni **Profiler** düğmesi (SQL Server'da) sunucudaki **TÜM uygulamaların** trafiğini canlı akıtır — SSMS Profiler'ın karşılığı ama modern

Extended Events üstünde (sunucuya çok daha az yük). Şablonlar: Standart · Yavaş sorgular (ms eşiği sunucuda) · Hatalar · Deadlock; veritabanı süzgeci, arama, Duraklat/Durdur, Kopyala/CSV. Durdurunca oturum sunucudan silinir. Ayrıntı: **Bölüm 34**.

- 🧠 **Grupla (SSMS'te yok):** aynı sorgunun parametre varyantları TEK satırda canlı sayaçla (kez · ortalama · en yüksek · toplam CPU) — "hangi sorgu sunucuyu yiyor" tek bakışta.
- 🕸️ **Kilitlenme şeması:** Deadlock şablonuyla yakalanan her kilitlenme görsel şemaya açılır (⛔ kurban kırmızı, sahip/bekleyen okları); **⬇ .xdl kaydet** SSMS'in açtığı biçimde dosyalar.
- 🤖 **AI Yorumla:** seçili olayı/grubu/kilitlenmeyi asistana yorumlatın — muhtemel neden + somut öneri (index/sorgu); kilitlenmede modele ham XML değil çözülmüş özet gider.
- ⧉ **Sekmede aç:** akışta yakalanan sorguyu tek tıkla yeni sorgu sekmesine taşır (olayın veritabanı önseçili) — planına bakın, düzeltin, çalıştırın.

0.25.0'da ne değişti — 🎨 yeni görünüm (SSMS/VS sistem tasarımı) + yeni logo + editör ve Log Analizi yenilemeleri:

- 🎨 **Tek tasarım, 7 renk ailesi:** uygulamanın görünümü SSMS 21'in oturduğu Visual Studio kabuğunun diline çekildi ve bu tasarım TÜM renk ailelerine uygulandı — **monokrom sol menü** (renkli ikon kutucukları kalktı; düz tek tonlu ray + çizgi ikonlar), her ailenin kendi vurgu renginde **durum çubuğu** ve bağlantı ekranında aynı renkte düz başlık bandı. Yeni **Visual Studio (SSMS)** ailesi 7. varyant olarak menüde (koyu + açık; imza mavi çubuk). Aileler arasında yalnız RENK değişir, tasarım aynıdır.
- 🪩 **Yeni logo:** mor karo maviye döndü (VS/SSMS kimliği); amber şimşek korundu. Başlık çubuğu / görev çubuğu / kurulum ikonu da yenilendi.
- 🔤 **Editör SSMS gibi:** yazı artık piksel-kenetli çizilir (bulanıklık bitti), uzun satır SSMS gibi sarılmaz (yatay kaydırma), sözdizimi renkleri SSMS varsayılanı — anahtar sözcük saf mavi (kalınsız), 'metin' kırmızı, **yerleşik fonksiyonlar macenta**, operatörler gri; sayılar bilinçli olarak turuncu kaldı.
- 📑 **Log Analizi:** belirlenemeyen "seviye" alanı combodan ve grid kolonundan tamamen kaldırıldı; yerine **Ek kolonlar** geldi — gruplanan her hatanın örnek satırından seçtiğiniz kolonlar (ör. StackTrace'e göre gruplayıp yanında ExceptionMessage) grid'de yan yana gösterilir, Kopyala/CSV'ye de girer.
- 📦 **Paket Aktarımı hata izi:** her aktarımın özeti ve hata ayrıntıları artık uygulama günlüğüne yazılır (rapor, günlük dosyasının yolunu söyler); hata atlandıysa `⬇ Hata raporunu kaydet` atlanan satırların TAM listesini dosyaya verir (ayrıntı tavanı 20 → 10.000).

0.24.9'da ne değişti — kaydırıcı tutamağı ARTIK GERÇEKTEN görünüyor: 🖱 "Ufacık nokta" sorununun gerçek kökü bulundu. Birkaç sürümdür tutamağı "büyütüyorduk" (24 → 36 → 72 px) ama çok satırlı sonuçlarda hep minicik kalıyordu. Sebebi ölçümle bulundu: tutamağa asgari boy vermenin kullandığımız yolu WPF'te **hiç çalışmıyormuş** — tutamak büyütülüyor ama kaydırıcı rayı onu doğal boyuna geri kırpıyordu; 10.000 satırlık sonuçta ekrana yalnız ~4 piksel

çiziliyordu. Önceki sürümlerdeki "büyütmelerin" hiçbiri bu yüzden ekranda görünmedi. Artık asgari boy rayın kendi hesabına işleniyor ve tutamak **gerçekten çiziliyor**: 10.000 satırlık sonuçta ~4 piksel yerine rayın %60'ı. Uzun rayda 160 piksele kadar çıkar, kısa rayda rayla birlikte küçülür ve her zaman sığar.

0.24.8'de ne değişti — kaydırıcı düzeltmesi: 🖱 Grid'i küçültünce kaydırıcı kayboluyordu. 0.24.7'de kaydırıcı tutamağını büyütmüştük (36 → 72 px), ama tutamak **kısa rayda raya sığmıyordu**: sonuç bölmesini kısalttığınızda — özellikle çok kolonlu sonuçlarda, yatay çubuk da yer kapladığı için — tutamak taşıyor ve kaydırıcı kayboluyordu. Artık tutamağın boyu **raya göre** hesaplanıyor: uzun rayda yine 72 px (çok satırlı sonuçta rahat tutulur), ray kısaldıkça birlikte küçülüyor ve **her zaman sığıyor**. 0.24.7'den gelen bir kusurun düzeltmesidir.

0.24.7'de ne değişti — saha turu:

- 🕯 **Kurumsal ağda AI çalışmıyordu.** Proxy'li bir ağda asistan "*Sağlayıcı 407 döndürdü*" deyip bir proxy oturum sayfası basıyordu. Sebebi şuydu: 🏠 Yerel AI kendi makinenizdeki Ollama'ya bağlanır, ama Windows'un proxy ayarında `127.0.0.1` muafiyet listesinde değilse **kendi makinenize giden istek bile** proxy'ye gidiyordu. Artık yerel adresler proxy'yi hiç kullanmıyor. Bulut ucu kullananlarda ise proxy'ye Windows kimliğiniz sunuluyor. Yine 407 alırsanız ham HTML yerine **ne yapmanız gerektiğini söyleyen bir mesaj** görürsünüz.
- 🕸 **Kayıt Haritası artık yalnız DOLU ilişkileri getiriyor** — boş çıkanlar açılmaz, dolu çıkan bir tablonun kendi bağlantıları da (▲▲/▼▼) eklenir, tanım/lookup tabloları atlanır. Ayrıntı aşağıdaki kutuda.
- 🖱 **Kaydırıcı tutamağı büyütüldü** (36 → 72 px). Çok satırlı sonuçlarda tutamağın boyu asgari değere dayanır; eski taban fareyle tutmak için küçük kalıyordu.
- 🪄 **SP Sihirbazı koyu temada** — tema değiştirdiğinizde bazı başlıklar eski temanın renginde donup kayboluyordu; artık tema değişimini takip ediyorlar.

0.24.6'da ne değişti — AI asistanı dakikalardan saniyeye indi: ⚡ Artık ~1 saniye. Yerel model, istemi *okurken* dakikalar harcıyordu — cevabı yazarken değil. Ölçüldü: saniyede ~15 kelime işliyor, dolayısıyla her soruyla birlikte gönderilen tablo listesi tek başına **90 saniye** ediyordu.

	Önce	Şimdi
"merhaba"	90–134 sn	**~1 sn**
Veri sorusu	86–128 sn	**~1 sn**

Üç değişiklik birlikte çalışıyor: **(1)** Asistan sekmesini açtığınızda soruların ortak başlangıcı modele önceden okutulur — siz yazarken, arka planda; **(2)** veriyle ilgisi olmayan mesajlara ("merhaba") tablo listesi hiç gönderilmez; **(3)** asistan artık **yalnız sizin yazdığınıza** sadık — başka bir sorgu sekmesindeki metin kendiliğinden karışmaz.

Bilmeniz gereken tek şey: sekmeyi ilk açtığınızda hazırlık ~90 saniye sürer (arka planda). Sekmeyi açar açmaz soru sorarsanız kalanını beklersiniz; bekleme balonundaki sayaç nerede olduğunuzu gösterir.

0.24.5'te ne değişti — AI artık bizim ajanımız, ve beklerken ne olduğu belli: ⛔ **Google Gemini kaldırıldı.** SQLST artık Google'a **hiçbir istek atmıyor**. Yapay zeka kendi ajanımızla çalışıyor: 🏠 **Yerel** (Ollama + `qwen2.5-coder:7b`) — anahtar yok, internet yok, **veri makineden çıkmıyor**. Kurumun kendi OpenAI-uyumlu ucu da kullanılabilir. Yapılandırmasında Gemini yazan makineler **kendiliğinden** Yerel'e geçer; bir şey yapmanız gerekmez (eski anahtarınız silinmez, yalnız kullanılmaz).

🔥 **Günün ilk sorusu da beklemiyor:** Asistan sekmesini açtığınız anda model sessizce belleğe alınmaya başlar; siz sorunuzu yazarken yükleme arka planda biter. Ölçüm: ön yüklemesiz "merhaba" **13,6 sn**, ön yüklemeyle **6,2 sn** — yükleme payı 6,8 sn'den **0,1 sn**'ye iner.

⏱️ **"Cevap vermiyor mu, çalışıyor mu?" sorusu bitti:** bekleme balonunda artık **geçen süre sayacı** döner ("düşünüyor… 14 sn"). Eskiden ekranda sabit bir yazı durduğu için çalışan istekle takılmış istek ayırt edilemiyordu.

0.24.4'te ne değişti — AI asistanı: hızlı, ısrarcı ve düzgün hizalı: 🔥 **Yerel AI belirgin hızlandı:** model artık sorular arasında **30 dakika bellekte kalıyor** (eskiden 5 dakikada düşüyor, sonraki soru yeniden yüklenmeyi bekliyordu — ölçüm: soğuk **108 sn** ↔ sıcak **24 sn**). Yani arka arkaya sorulan sorularda yalnız ilki yavaştır. Ayrıca cevap uzunluğuna tavan kondu: model kendini tekrar edip dakikalarca yazmaya başlarsa üretim kesilir.

🔁 **"Cevap alınamadı" büyük ölçüde bitti:** bulut sağlayıcılar yoğun saatlerde "*This model is currently experiencing high demand*" (503) döndürebiliyor — bu **geçici** bir durumdur ama SQLST ilk denemede pes edip ham hata basıyordu. Artık böyle geçici hatalarda **kendiliğinden 3 kez deneniyor** (2 sn, sonra 5 sn bekleyerek); yine de olmazsa ne yapılacağını söyleyen Türkçe bir mesaj gelir. Kota hatası (429) bilerek tekrarlanmaz — tekrarlamak kotayı daha da yakar.

💬 **Sohbet düzeni:** balonlar artık pencerenin gerçek kenarlarına yaslanıyor (soru sağda, cevap solda). Eskiden geniş ekranda ortadaki dar bir sütuna sıkışıyordu.

0.24.3'te ne değişti — donmanın İKİNCİ kapısı kapandı (sanallaştırma emniyet kemeri): 0.24.2'nin canlı testinde büyük tabloda donma **tekrarladı** — ama bu kez 0.24.2'nin iz mekanizmaları çalıştı ve teşhis ilk kez tamamen kayıtla yapıldı: `donma.txt` blok *sürerken* "151 sn · yığın 4.574 MB" yazdı, `faz.txt` `grid-hazir`'da takılı kaldı. Kök: sonuç grid'inin yüksekliği bir bağlamadan gelir; bağlama değer üretemeyince grid **sonsuz yükseklik** alıyor → satır sanallaştırması ölüyor → 33.964 × 43 = 1,46 milyon hücrenin görseli kuruluyor (yerel ölçüm: sanallaştırma ölüyken 3.000 satır 46 sn + 1,45 GB; canlıyken 0,9 sn). Düzeltme **çift emniyet**

kemeri: grid'e MaxHeight + bağlamaya FallbackValue — yükseklik hangi yolla bozulursa bozulsun sanallaştırma yaşar. Kanıt: aynı tam ölçek artık **1 saniyede** açılıyor.

0.24.2'de ne değişti — ÇÖKMENİN KANITLI DÜZELTMESİ + Edit modu editörü: Bu sürüm, dört turdur süren **"büyük tabloya SELECT atınca çöküyor"** dosyasını kanıtla kapatıyor.

🧱 **Çökme çözüldü (kanıt zinciri sahadan geldi):** çökme dökümleri **StackOverflowException**'ı WPF *metin ölçümü* zincirinde gösterdi; uçuş kaydı, 33.964 satırlık KonfigurasyonParametre'de "Grid hazır"dan hemen sonra **UI'ın 832 saniye kilitlenip** yığının **6,4 GB'a** şiştiğini kaydetti. Kök: kolon genişliğinde hiç tavan yoktu — binlerce karakterlik config değerleri Auto genişlikte ölçüm fırtınası yaratıyordu. Artık kolonlar **400 px tavanlı** (elle genişletilebilir, tam değer çift tıkta): aynı desen testte **5 saniyede** açılıyor. Donma gözlemcisi de düzeldi: eskisi tam donma anında susuyordu; yenisi ayrı iş parçacığından bakar ve blok *sürerken* `donma.txt` 'ye yazar.

⚡ **Log Analizi (ExceptionLog'daki "120 sn" hatasının cevabı):** son-24-saat süzgeci `$expr + $$NOW` yerine **düz tarih literali** kullanıyor — eski biçimin index kullanabilmesi sunucu sürümüne bağlıydı (dev koleksiyonda tam tarama → 120 sn tavanı aşılıyordu); yeni biçim her sürümde index'e oturur (yerel ölçüm: 7× hız, 5.000 kayıt 623 ms).

➖ **Edit modu:** satır sınırı **200 → 1.000**; asıl sinsi olan da düzeldi — geniş satırlı tabloda bellek bütçesi 178 gibi sayılarda *sessizce* kesiyordu, artık bütçe 256 MB ve kesilme **nedeniyle birlikte** bilgi satırında yazıyor. Sorgu kutusu da düz metin kutusu olmaktan çıktı: **gerçek editör** (renklendirme + IntelliSense + Ctrl+F), ana editörle aynı öneriler.

🔧 **Ayrıca:** find yardımcısında koleksiyon alanları yeniden geliyor (0.24.1'de "alan okunamadı" diyordu — keşif sorgusu yanlışlıkla 0 satır limitiyle koşuyordu).

0.24.1'de ne değişti — "saha turu-4": bozulanların onarımı + çökmeyi görünür kılan iz: Bu sürüm, 0.24.0'ı canlı kullanırken bildirilen **8 bulguyu** kapatıyor. Turun kendine özgü yanı: bulguların **üçü bizim önceki turda soktuğumuz regresyonlardı** — açıkça öyle yazıyoruz.

🔴 **Onarımlar (0.24.0'te bozulmuştu): Mongo select** sonuç döndürüyor ama *grid boş* kalıyordu — mesajda "200 belge" yazıyor, tek satır görünmüyordu. Sebep: sonuç boru hattı yeni akışlı alıcıya taşınırken yalnız SQL yolu besleniyordu. Artık motor-bağımsız: alıcıyı beslemeyen her yürütücü yine çalışır. Aynı kök nedenden, ağaçta koleksiyona sağ tık → **"İlk 200 satır"** da düzeldi. **Adsız kolon** (`SELECT COUNT(*)`) hücresi boş görünüyordu: yer tutucumuz `(adsız)` idi ve WPF, parantezle *başlayan* yolu eklenmiş özellik sanıp bağlamayı sessizce düşürüyordu — ölçtük, `ort süre (ms)` gibi gerçek adlar sağlam, kıran tek şey baştaki parantezdi.

🧭 **Çökme: artık iz bırakıyor.** Dört turdur çökmenin *nerede* olduğunu bilmiyorduk, çünkü kayıtta çökme ile normal kapanış **birebir aynı** görünüyordu. Artık `faz.txt` var: tek satırlık,

üzerine yazılan bir iz (`sorgu-basladi → okuma-bitti → grid-hazir → yerlesim-bitti → temiz-cikis`). Çökmeden sonra **son satır ölüm fazını adıyla söyler**; `temiz-cikis` yazıyorsa o oturum çökme değildir. Donma nöbetçisi de artık izlediği iş parçacığının *üstünde* değil — eskisi tam da donma anında susuyordu. Ayrıca sürecin bellek tavanı **her** okuma yolunda geçerli (şema okumalarında kapalıydı) ve Mongo'da bayt bütçesi `limit` yazıldığında kapanmıyor, **yükseliyor** (96 → 256 MB).

⏱️ **Log Analizi.** Analiz sorgusuna sunucu tarafı süre sınırı *hiç* gitmiyordu; kaçan bir tarama sunucuda sonsuza kadar sürebiliyordu (■ Durdur yalnız istemciyi keser). Artık **120 sn** tavanı var — aşılırsa ne yapılacağını söyleyen bir mesaj gelir. Analiz öncesi yapılan "index var mı?" sorusu da 3 sn ile sınırlandı: gelmezse hızlı yola düşülür (eskiden yanıt gelmezse analiz *hiç* başlamıyordu). Durum satırı artık **saniye sayıyor**. Ve bir doğruluk düzeltmesi: örneklem tavana dayandığında sayılar artık "gerçek toplam" diye yazılmıyor, 🔢 **Gerçek toplamları getir** düğmesi çıkıyor.

🔧 **Diğer:** modelsiz pencerelerde **Kapat düğmesi ve Esc çalışmıyordu** (WPF'te `IsCancel` yalnız iletişim kutularında iş görür) — beş pencerede birden düzeldi · find yardımcısında alan seçilince artık **alan değil operatör** öneriliyor, açılan ayracın **kapanışı da yazılıyor** ve **Tab** öneri uygulamak yerine sonraki kutuya geçiyor · Mongo'da artık **Shell yapıştır ekranı** Compass'ın kendi çıktısını (`db.getCollection('ad')…`) da çeviriyor · **kaydırma çubuğu** kalınlaştı (8 → 10 px, üstüne gelince 12) ve uzun listede tutamaç küçülüp kaybolmuyor · önizleme artık Mongo koleksiyonlarında görünmüyor (orada satır getirilemiyordu, yarım bir özellikti).

0.24.0'te ne değişti — "saha turu-3": çökme kök nedeni, VPN arkasında Mongo hızı ve "hiçbir şey olmuyor" diyen ekranlar: Bu sürüm, 0.23.0'ı canlı kullanırken bildirilen **7 bulguyu** kapatıyor. Üç bulgunun ortak dersi aynı: kullanıcının "çalışmıyor" dediği yerde özellik aslında *çalışıyor*, ama ekranda hiçbir şey görünmüyordu.

🔴 **Sonuç boru hattı yeniden kuruldu (çökme üçlüsü: 3, 7, 9):** Aynı satırlar bellekte **iki kez** duruyordu — önce bir liste, sonra ondan kopyalanan grid tablosu. Artık okuma sırasında **doğrudan** grid tablosuna, üstelik gerçek tipleriyle (kutulama yok) yazılıyor. **Ölçüm** (200.000 satır × 10 kolon): satır başına **708 → 455 bayt** (%36 az) ve düşüş tam da çökmelerin olduğu *yükleme* anında. 0.23.0'te tanıtılan **"ilk 1.000 satır"** bandı **kaldırıldı** — sonucu ikiye bölüp ilk parçayı ayrıca kopyalamak hem bellek hem iş demekti; SSMS de böyle çalışmaz.

⚡ **MongoDB log analizi: asıl darboğaz AĞ'dı (ölçümle bulundu):** 0.23.0'teki düzeltme yetmemişti çünkü *yanlış* darboğazı düzeltmişti. Ölçüm sunucunun zaten hızlı olduğunu gösterdi (200 bin belgede 222 ms); fark aynı sorgunun **19,8 MB** veri taşımasıydı — VPN arkasında bu dakikalar demek. Üç düzeltme: mesajlar **sunucuda 300 karaktere** kırpılıyor (imza mesajın başından çıktığı için gruplama etkilenmiyor) · Mongo örneklemi 20.000 → **5.000** · alan keşfi artık 50 tam belge indirmiyor, yalnız **alan adı + tipi** taşıyor (846 ms / 101 KB → **50 ms / 13 KB**). Toplamda ~10× daha az veri. Ayrıca özete **faz süreleri** eklendi (`⏱ sorgu 0,8 sn · gruplama`

`0,2 sn · 5.000 kayıt · ~1,8 MB`) — bir daha "yavaş" denirse hangi fazın yediği tahmin edilmeyecek.

🚪 **"Basıyorum, hiçbir şey olmuyor" diyen üç ekran:** (1) **"Bu id nerede geçiyor?"** — pencere açılmadan ÖNCE tüm koleksiyon envanteri bekleniyordu; büyük veritabanında saniyelerce boş ekran. Artık pencere **anında** açılıyor, envanteri kendi içinde okuyor ("Koleksiyon envanteri okunuyor...") ve ■ **Durdur** bu evrede de kesiyor. (2) 📋 **Shell yapıştır** — ekranı hiç yoktu: panoyu sessizce okur, çeviremezse yalnız alt durum çubuğuna yazardı. Artık **ekran açılıyor**: pano ön-dolu geliyor, metni düzenleyebiliyorsunuz, çeviri anında yenileniyor ve hata girdinin hemen altında görünüyor. Üstelik **Compass'ın kendi "Copy" çıktısı** (`db.getCollection('ad')…`) artık çevriliyor — eskiden reddediliyordu. (3) **Find yardımcısı** — Sort/Filter/Project kutularında **koleksiyonun alanları** önerilmiyordu (yalnız alandan *sonrası*: 1/-1, $operatörler). Sebebi: koleksiyon listesi hız için alansız çekiliyor. Artık koleksiyon seçilince o koleksiyonun alanları getiriliyor ve yanında **"12 alan"** yazıyor.

🆕 **Kolaylıklar: 👁 Tablo önizleme artık sağ tık menüsünde** de var (Ctrl+üzerine gelme ipucuyla birebir aynı içerik; kalıcı pencere, kopyalanabilir, "▶ İlk 100 satırı sekmede aç") · **Denetim izi gridi** sağa **kaydırılabiliyor** ve satıra **çift tıkla** kaydın tam SQL'i ayrı pencerede açılıyor (Zaman kolonu da artık saniyeyi kesmiyor).

0.23.0'te ne değişti — "saha turu-2": büyük veride kararlılık ve HIZ: Bu sürüm, 0.22.0'ı canlı kullanırken bildirilen **13 bulguyu** kapatıyor. Hızla ilgili kararların hepsi **ölçülerek** verildi (yerel MongoDB / LocalDB üzerinde); makul görünen üç fikir ölçüm sonucu *elendi*.

🔴 **Büyük tablo/koleksiyon (üç kez bildirildi):** (1) **Aşamalı gösterim** — sorgu sonucunun ilk **1.000 satırı** gelir gelmez ekrana düşer, kalanı arka planda akmaya devam eder; beklemek istemezseniz ■ **Durdur** ile o hâliyle çalışırsınız (eldeki satırlar silinmez). MSSQL ve MongoDB'de aynı davranış. *(Bu band **0.24.0'te kaldırıldı** — sonucu ikiye bölmek belleğe ve işe mal oluyordu; yerine tek geçişte yükleme geldi, bkz. yukarıdaki 0.24.0 kutusu.)* (2) **Uygulama bellek tavanı** — eski koruma yalnız MAKİNE yüküne bakıyordu; geniş RAM'li makinede hiç ateşlemiyordu. Artık sürecin kendi yığını ölçülüyor: tavana dayanınca okuma kesilir, eldeki veri kalır, bandda nedeni yazar. (3) **MongoDB çökmesinin nedeni bulundu**: bellek bütçesi *ham* BSON'u sayıyordu; bellekteki belge grafiği ondan ~4 kat büyük — bütçe düzeltildi ve belgeler satırlara çevrilir çevrilmez bırakılıyor (tepe bellek yarıya indi). (4) **Donma nöbetçisi**: arayüz 1,5 sn'den uzun bloke kalırsa log dosyasına saniyesi, yığın boyutu ve o anki sekme yazılıyor — bir daha donma olursa nedeni kayıtta.

⚡ **MongoDB hızı (hepsi ölçümle): Log Analizi** artık zaman alanının index'li olup olmadığına göre strateji seçiyor — index yoksa "son 24 saat" süzgeci tüm koleksiyonu tarıyordu (ölçüm: 600 bin belgede 1,7 sn ↔ 11 ms), onun yerine **son 20.000 kayıt** okunuyor ve özet kalıcı çözümü (createIndex komutu) yazıyor. **"Bu id nerede geçiyor?"** taraması 6'lı gruplar hâlinde **eşzamanlı** koşuyor (8 aday: 4.260 ms → 654 ms) ve index'li alanlar öne alınıyor (2 ms ↔ 493 ms) — cevap

anında görünüyor; ■ Durdur artık gerçekten anında kesiyor. **$lookup sihirbazı** pipeline'ın sonuna **$limit 200** koyuyor (kutudan değiştirilir, 0 = sınırsız) ve hedef alan index'siz ise uyarıp createIndex komutunu veriyor — ölçümde limitsiz sorgu 60 sn'de iptal oluyordu, limit + index ile **143 ms**.

🆕 **Yeni:** SQL yönetim panelinde **"Yavaş sorgular"** sekmesi (ortalama süreye ve toplam süreye göre iki liste) · sekme başlığında **tıklanabilir** 📌 **raptiye** (sabitlemek için artık sağ tık gerekmiyor) · LINQ⇄SQL çevirici **gerçek EF sorgularını** çeviriyor (alt sorgulu join, ilişkili Count/Any, enum ve dış nesne alanları, sabit olmayan LIKE deseni, bit kolon koşulu).

🔧 **Düzeltmeler:** hücre üzerinde **lookup ipucu** artık şema henüz yüklenmemişken de çalışıyor (ve çözülemiyorsa nedenini yazıyor) · motora özgü sağ tık menüsü öğeleri **yalnız kendi motorunda** görünüyor (Mongo'ya özel "bu id nerede geçiyor" MSSQL'de, MSSQL'e özel INSERT/kayıt haritası Mongo'da görünüyordu) · Kor **açık** temada sol menü başlığı okunur oldu (beyaz kalmıştı) ve 12 paletin metin/zemin kontrastı artık testle korunuyor · **"REST'e gönder" kaldırıldı** (kullanılmıyordu).

0.22.0'da ne değişti — "saha turu": kararlılık düzeltmeleri + günlük işi hızlandıran 10 yeni kolaylık: Bu sürüm, canlı kullanımda bildirilen **29 bulgunun tamamını** ve iki fikir turundan çıkan 10 yeni özelliği içeriyor.

🔴 **Kararlılık (en önemlisi):** (1) **Çok büyük hücre değerleri artık uygulamayı çökertmiyor** — MB'lık JSON/metin içeren satırlarda gridde yalnız ilk 2.000 karakter gösteriliyor (tam değer çift tıkta, kopyalamada ve dışa aktarmada aynen duruyor). (2) ■ **Durdur artık ANINDA kesiyor** — sunucudan cevap beklenmiyor, pencere donmuyor; ayrıca durdurduktan sonra **aynı sekmede hemen yeni sorgu çalıştırılabiliyor** (eskiden asılı sorgu o sekmeyi kilitliyordu). Durdur sekmenin bağlantısını **korur** — `#temp` tablolar ve SET ayarları yerinde kalır (SSMS gibi); bağlantı yalnız sorgu iptale hiç yanıt vermezse (birkaç saniye içinde) kapatılır ve mesaj bunu söyler. (3) **F5 her sekmede çalışıyor** — bir sekmede sorgu sürerken başka sekmede kısayollar ölmüyor. (4) **Sekme değiştirince sonuç gridi başa dönmüyor**: kaydırma konumu, seçili satır ve imleç yerinde kalıyor — bir sekmeden veri takip edip diğerinde çalışmak mümkün. (5) `ALTER DATABASE` gibi ifadeler Güvenli Yazma açıkken de çalışıyor.

⚡ **Grid ve editör kolaylıkları (yeni):** (6) **FK hücresinde otomatik açıklama** — `3` yazan hücrede fareyle beklerseniz **3 → Onaylandı** görünür (tanım tablosu bir kez okunur, sonrası sorgusuz). (7) **Hücreden hızlı filtre** — sağ tık → "Bu değere filtrele / hariç tut / ≥ / ≤"; koşul sorguya eklenip yeniden çalışır, **Ctrl+Z** ile geri alınır. (8) **Kolon istatistiği** — kolon başlığına sağ tık: MIN/MAX/ortalama/toplam/ farklı değer/NULL sayısı tek sorguda (sorgunuzun süzgeci korunarak). (9) **Editörde tablo önizleme** — tablo adının üstünde **Ctrl'ye basılı tutup beklerseniz** kolonlar ve ilk 5 satır açılır. (10) **WHERE'de tanım değeri önerisi** — `WHERE DurumId =` yazınca değerler anlamıyla listelenir (`3 Onaylandı`), seçince koda yalnız sayı yazılır.

📑 **Sekmeler:** ✕ artık her sekmede sağ kenarda · sağ tık menüsü (**Sabitle** 📌 · **Kapat** · **Diğerlerini kapat** · **Tümünü kapat**) · sabitlenen sekme kapatılamaz · **LINQ⇄SQL ve SP Sihirbazı artık pencere değil sekme**; araç pencereleri de ekranı kaplamak yerine makul boyutta açılıyor.

🍃 **MongoDB:** (1) **"Bu id nerede geçiyor?"** — bir değeri tüm koleksiyonlarda arayıp hangisinde kaç belge olduğunu listeler (Mongo'da yabancı anahtar olmadığı için alan tipine göre tarar). (2) **$lookup sihirbazı** — iki koleksiyonu birleştiren pipeline'ı seçimlerle üretir. (3) **Shell komutu yapıştır** — Compass'tan kopyalanan `db.koleksiyon.find({…})` çalıştırılabilir hâle çevrilir. (4) **Yönetim Paneli'ne "Index önerileri / Yavaş sorgular"** sekmesi — gerçekten yavaş çalışmış ve index kullanmamış sorgulardan `createIndex` önerisi üretir (profiler kapalıysa nasıl açılacağını söyler).

🌐 **REST:** sağ üstteki butonlar artık üst üste binmiyor · "Gövde (JSON)" sekmesi **Body** oldu ve **anahtar/değer girip JSON'u canlı görebiliyorsunuz** · Params ile Değişkenler arasındaki fark ekranda anlatılıyor.

🔁 **LINQ ⇄ SQL:** LINQ→SQL çevirisindeki 4 hata giderildi (en önemlisi: JOIN kaynağındaki `.Where` koşulu **sessizce düşüyordu**) · SQL→LINQ artık JOIN ON'daki ek koşulları ve `@parametre` 'leri çeviriyor · çevir düğmeleri kendi panellerinin üstüne alındı.

🪵 **Log Analizi:** Durdur asılı kalmıyor · analiz **belirgin hızlandı** (pahalı tam sayım artık yalnız "🔢 Gerçek toplamları getir" düğmesiyle) · MongoDB'de İlk/Son görülme artık doluyor · koyu temada etiketler okunuyor · tarih seçimi kart hâline geldi (Bugün / Son 7 gün / Son 30 gün).

🎨 **Görünüm:** Açık **Kor** teması bütünleşti (siyah şerit/ray açık küle geçti) · koyu Kor'da editör zemini yumuşatıldı · nesne ağacında **FK kolonları** 🔗 **ile işaretleniyor** · sonuç şeridi butonları belirginleşti · sağ tık → **UPDATE örneği** (INSERT örneğinin eşi; PK yoksa güvenli `WHERE 1 = 0` ile) · 🔍 Prova artık UPDATE'in **olacak değerlerini** gösteriyor · ORDER BY yazılmamış tek-tablolu sorgularda sonuç **anahtar sırasına** diziliyor (SQL değişmez, bilgi notu düşülür).

0.21.7'de ne değişti — 🌐 REST İstemcisi büyük genişleme + Log Analizi hızı + düzeltmeler: REST İstemcisi'ne 14 yeni yetenek (asıl fark: veritabanıyla entegre çalışması): (1) **Yanıt → tabloya kaydet** — JSON yanıttan kolonları çıkarıp **görsel tasarımcıda** tablo oluşturup satırları aktarır; (2) ~~Sorgu sonucu → toplu istek~~ — *bu özellik sonradan KALDIRILDI (pratikte kullanılmıyordu): grid'deki "🌐 REST'e gönder" düğmesi ve REST penceresindeki "🔁 Toplu Gönder" artık yok*; (3) **DB değeri → değişken** — bir değişken canlı SQL'den doldurulur; (4) **Yanıt ↔ DB karşılaştır** — API yanıtı bir sorgu sonucuyla anahtar kolonda diff'lenir; (5–7) **AI**: tarifle istek üret · yanıttan CREATE TABLE öner · iki yanıtı diff yorumla; (8) **Ortam değişkenleri** (dev/test/prod); (9) **API Key** kimliği (Basic/Bearer'a ek); (10) **cURL yapıştır → istek**; (11) **Kayıtlı istek koleksiyonları** (Grup/Ad); (12) **İstek zinciri** — yanıttan değeri değişkene çıkar; (13) **Kod üret** (cURL/C#/Python); (14) **JSON path** ile alan çıkar.

Ayrıca: **Log Analizi belirgin hızlandı** (gruplama arka planda; sonuç tabloyu tamamen kapsıyorsa pahalı tam-tarama sayımı atlanır); **yerel/LAN adreslerine** (192.168.x.x gibi) SOAP/REST istekleri artık kurumsal proxy'de asılmıyor; **sonuç hücresine çift tıkta 🔎 Lookup** — kolon bir yabancı anahtarsa referans tablonun açıklamasını (tanım) gösterir; AI sohbet kutusuna **çok satırlı sorgu yapıştırma** ve LINQ→SQL'de **tablo önceliği** düzeltmeleri; ayrıca yeni 🎨 **Kor teması** (kırmızı-sarı-siyah) — açık ve koyu (tema menüsünden seçilir).

0.21.6'da ne değişti — AI hızı + pencere/sekme düzeni: (1) **Yapay zeka artık çok daha hızlı yanıtlıyor:** genel sorularda (örneğin bir selamlaşma) tüm veritabanı şeması yerine yalnızca **tablo adları** gönderiliyor; belirli bir tabloyu sorunca o tablonun kolonları eklenir. Böylece istek küçülür ve özellikle güçlü ekran kartı olmayan makinelerde yanıt beklemesi ciddi kısalır. (2) **Veri arama** ve **Log analizi** artık ayrı pencere değil, uygulama içinde **sekme** olarak açılıyor (Kod arama gibi); **Karşılaştır** ise SOAP istemcisi gibi **ayrı pencere** olarak açılıyor. (3) Bir yardımcı pencereyi (SOAP, REST, LINQ, AI cevabı vb.) kapattığınızda **ana pencere artık arkaya düşmüyor**, önde kalıyor.

0.21.5'te ne değişti — canlı kullanım düzeltmeleri (şema, AI, plan): Uygulama gerçek veritabanlarında kullanılınca çıkan dört sorun giderildi: (1) **Büyük veritabanında şema okunamıyordu** ("beklenmeyen sonuç: 4 küme gelmedi") — iç şema okuması artık satır/bellek bütçesine takılmadan tam okunuyor, LINQ⇄SQL ve otomatik tamamlama büyük veritabanlarında da açılıyor. (2) **Yapay zeka büyük şemada yanıt vermiyordu** ("bağlam boyutu aşıldı") — artık AI'a tüm şema yerine yalnız **sorguda geçen ilgili tabloların** şeması gönderiliyor (istek küçülür, hızlanır); ayrıca yerel modelin bağlam penceresi büyütüldü. (3) **Plan / Performans değerlendir** düğmeleri sorgu çalışırken artık **devre dışı** — çalışırken basınca sessizce hiçbir şey yapmıyor görünmesi (ve planın sorguyu yeniden çalıştırması) giderildi. Bağlantı sorunları da netleştirildi (yerel SQL Server örneği için sunucu adı tam olarak `(localdb)\MSSQLLocalDB` yazılmalı).

0.21.4'te ne değişti — AI'lı kurulum artık ÇALIŞAN 2 dosyalık paket (kurulum + model yan yana): Önceki AI'lı kurulum tek bir ~6 GB `.exe` idi ve **çift-tıklayınca "bu uygulama bilgisayarınızda çalışamıyor"** veriyordu. Sebep: **Windows'ta çalışabilir tek bir `.exe` 4 GB'ı geçemez** (PE/yükleyici sınırı) — yapay zeka modeli tek başına 4,36 GB olduğundan tek dosya **hiçbir araçla mümkün değil**. Çözüm: AI'lı kurulum artık **güvenilir Inno kurulumu (~1,65 GB) + model dosyası (4,36 GB) yan yana = 2 dosyalık bir klasör**. Klasörü kopyalayıp `.exe` 'ye çift-tıklarsınız; kurulum modeli yanındaki dosyadan alır (internetsiz). Kurulum artık sıfır makinede baştan sona sınandı ve **~2,5 dakikada** tamamlanıyor. Uygulama kurulumunun kendisi eskisi gibi tek Inno `.exe` 'dir. Ayrıca AI'lı kurulum makineyi doğrudan **Yerel (internetsiz)** yapay zekaya ayarlar; buluta ait ayar/anahtar taşınmaz.

0.21.3'te ne değişti — AI'lı kurulum GERÇEK temiz-makine testinden geçti (baştan sona tamamlanıyor): Bir bilgisayardaki her şey (uygulama + Ollama + model) tamamen kaldırılıp AI'lı kurulum **sıfırdan** denendi ve iki önemli sorun bulunup düzeltildi: (1) **Kurulum bir noktada**

takılıp kalıyordu — yerel yapay zeka bileşeni (Ollama) kurulduktan sonra kurulum ilerlemiyor, model bir türlü oluşmuyordu; artık kurulum sınırlı bir bekleme ile güvenle devam ediyor ve yerel modeli baştan sona kuruyor (temiz makinede uçtan uca **~2,5 dakikada** tamamlandığı, modelin gerçek SQL ürettiği doğrulandı). (2) **AI'lı kurulum artık makineyi doğrudan Yerel (internetsiz) yapay zekaya ayarlıyor** — böylece kutudan çıkar çıkmaz veri makineden çıkmadan çalışır; buluta ait hiçbir ayar/anahtar AI'lı kuruluma girmez.

0.21.2'de ne değişti — kod incelemesi düzeltmeleri (AI kurulumu artık gerçekten çalışıyor): Detaylı inceleme + canlı testte AI'lı kurulumu kıran **iki gerçek hata** yakalanıp düzeltildi: (1) modelin kayıt dosyasındaki geçersiz bir ayar (`keep_alive`) yüzünden model oluşturma adımı çöküyordu; (2) tek-dosya kurulumun (SFX) kurulum-sonrası betiği doğru başlatılamıyordu — doğru SFX modülü (`7zSD.sfx`) ve çağrı biçimiyle düzeltildi (gerçek SFX testleriyle doğrulandı). Ayrıca iki küçük iyileştirme: AI'ın SQL'i düzeltildiğinde bu artık sohbette de açıkça görünüyor; bağlantı yokken üretilen SQL yanlışlıkla "bozuk" işaretlenmiyor.

0.21.1'de ne değişti — küçük cila: AI asistanına soru sorunca alt bilgide görünen "Şema özeti hazırlanıyor…" iç işleyiş metni, bir selamlaşmada kafa karıştırıyordu; sade "Hazırlanıyor…" / "Yanıt hazırlanıyor…" oldu. Ayrıca **AI'lı tek kurulum** (Ollama + model gömülü, ~5 GB) ilk kez gerçekten üretildi.

0.21.0'da ne değişti — 🏠 Kendi Yapay Zekamız (internetsiz, veri makineden çıkmadan):

- **Artık AI için internet ya da bulut gerekmiyor.** Yapay zeka **tamamen sizin makinenizde** çalışabiliyor: soru sorarsınız, cevap gelir; ama ne sorunuz ne şemanız ne de bir tek satır veriniz **makineden dışarı çıkmaz**. Kapalı ağlar (Mersis gibi) için biçilmiş kaftan.
- **Tek kurulumla her şey:** AI'lı kurulumu çalıştırdığınızda gerekli her şey (yapay zeka motoru ve modeli) aynı kurulumdan gelir — ayrıca bir şey indirmenize gerek yok, yönetici yetkisi de istemez.
- 🛡 **Yanlış SQL'e kalkan:** yapay zekanın yazdığı her SQL, size verilmeden önce SQLST'nin kendi denetçisinden geçer; bir yanlışlık varsa ya kendisi düzeltir ya da sizi açıkça uyarır — "yaklaşık doğru" bir sorgu sessizce elinize tutuşturulmaz.
- Ayrıntılı, sade anlatım için **Bölüm 25**.

0.20.8'de ne değişti — 🔁 LINQ ⇄ SQL kapsam sıçraması + arama vurgusu:

- 🔁 **Çeviri kapsamı büyük ölçüde genişledi (iki yönde, simetrik):** LEFT JOIN (⇄ `DefaultIfEmpty`), EXISTS ⇄ `Any` , IN alt sorguları ⇄ `Select().Contains()` , UNION/UNION ALL/INTERSECT/EXCEPT ⇄ küme metotları, **CTE'ler ve türetilmiş tablolar** ⇄ `var x = (…);` kalıbı, virgüllü FROM ⇄ çoklu `from` , ISNULL/COALESCE/NULLIF ⇄ `??` , CAST/CONVERT, SUBSTRING, DATEADD/DATEDIFF, karışık LIKE ⇄ `EF.Functions.Like` . Gidiş-dönüş (SQL→LINQ→SQL) testle kanıtlı. Ayrıntı: **Bölüm 33**.

- **Ctrl+F eşleşme vurgusu düzeldi:** koyu temada okunmayan açık yeşil işaret, artık tüm 10 temada okunur yarı saydam vurgu (amber; amber paletlerde camgöbeği).

0.20.7'de ne değişti — 💾 Donma koruması (KRİTİK):

- **Dev tabloda ağır sorguda donma çözüldü:** bellek dolu makinelerde (başka uygulamalar RAM'in %85+'ını kullanırken) büyük sonuç kümesi uygulamayı dakikalarca kilitleyebiliyor, Windows "yanıt vermiyor" deyip kapattırıyordu. Artık **uyarlanır bütçe** var: makine bellek baskısındayken sınırlar otomatik **10.000 satır / 24 MB**'a iner ve bantta açıkça söylenir — sonuç saniyeler içinde gelir, donma yaşanmaz. Makine rahatken normal sınırlar (100.000 satır) aynen geçerlidir.
- 🛩️ **Uçuş kaydedici:** her sorgunun evreleri (başladı → okuma bitti → grid hazır) log dosyasına anında yazılır — beklenmedik bir sorun olursa artık her zaman iz vardır (`%APPDATA%\SQLST\logs`).
- 🧭 **Faz izi** `0.24.1` **:** aynı klasörde `faz.txt` adında **tek satırlık** bir dosya tutulur: uygulamanın o an hangi evrede olduğu, üzerine yazılarak (`sorgu-basladi → okuma-bitti → grid-hazir → yerlesim-bitti → temiz-cikis`). Uygulama beklenmedik biçimde kapanırsa **son satır hangi evrede kapandığını adıyla söyler**; `temiz-cikis` yazıyorsa o oturum normal kapanmıştır. Bu dosya bilerek çok ilkeldir (kayıt altyapısını kullanmaz), çünkü bellek tükendiğinde normal kayıt da yazılamaz. Arayüz 1,5 sn'den uzun bloke kalırsa aynı klasöre `donma.txt` düşer.

0.20.6'da ne değişti — 🔁 LINQ ⇄ SQL (YENİ) + 📂 Dosya Aç + 💾 çökmezlik sertleştirmesi:

- 🔁 **LINQ ⇄ SQL (YENİ araç):** sol rayda iki yönlü çevirmen — C# LINQ'inizi motorun SQL'ine, T-SQL SELECT'inizi C# LINQ'ine çevirir; **kendi motorlarımızla, yapay zekasız ve çevrimdışı**. Şema+FK farkındalıklı (navigation'lar JOIN olur), SQL tarafında editör önerileri (Ctrl+Space), üretilen SQL tek tıkla sekmede çalışır. Ayrıntı: **Bölüm 33**.
- 📂 **Dosya Aç:** bir .sql dosyasını kendi adıyla sekmede açın — sol ray düğmesi, `Ctrl+O` ya da pencereye sürükle-bırak; kodlama otomatik (BOM/UTF-8/Türkçe ANSI, bozulma yok). Bölüm 7'deki kutuya bakın.
- 💾 **Çökmezlik sertleştirmesi (KRİTİK):** dev tabloda (10M+ satır) ağır sorgular bellek yetmezliğiyle uygulamayı öldürebiliyordu. Artık **fiziksel RAM nöbetçisi** var: makinenin bellek yükü kritik eşiğe dayandığında okuma güvenlik için kesilir ve "ilk N satır" bandı gösterilir — uygulama ASLA çökmez. Beklenmedik ölümcül durumlar için de artık `son-cokme.txt` izi bırakılır.
- 🔗 **Kolon bağları (0.20.5'ten):** gezginde kolona sağ tık → tüm FK bağlantıları.

0.20.5'te ne değişti — 🔗 Kolon bağları:

- 🔗 **Kolon bağları (YENİ):** nesne gezgininde bir kolona **sağ tık** → **"Tüm bağlantılarını göster"** — kolonun yabancı anahtar bağları küçük bir pencerede: **▲ giden** (bu kolon

hangi tabloyu işaret ediyor) + ▼ **gelen** (bu kolonu hangi tablolar işaret ediyor), kısıt adları ve bileşik anahtar eşleşmeleriyle. Satıra **çift tık** → ilişkiyi ortaya çıkaran **JOIN sorgusu** (ilk 100 satır) yeni sekmede çalışır. FK grafı şema önbelleğinden — ağa çıkılmaz, pencere anında açılır. Ayrıntı: **Bölüm 6**.

0.20.4'te ne değişti — 🌐 REST İstemcisi kullanıcı dönüşleri cilası:

- **Yanıt eylemleri yeniden düzenlendi: 🗎 JSON Olarak Aç** (yanıtı ayrı, KÜÇÜK açılan, büyütülebilir pencerede gösterir) · ⧉ **URL'i Kopyala** (tam URL'i query dahil panoya) · ↴ **Kopyalanmış URL'i Aç** (panodaki URL'i istek alanına yükler). İlk sürümdeki *yanıt→grid*, *{{değişken}} ortamları* ve *cURL içe/dışa* sadeleştirme ile kaldırıldı.
- **Yerleşim:** sol panel (kayıtlı + geçmiş) genişledi ve oranlı oldu; istek alanı büyük, yanıt alanı küçük açılır — aradaki **ayırıcı çubuk** sürüklenerek oran ayarlanır. Kayıtlı istek **sağ tık** → ↧ **Yükle / 🗑 Sil**.
- **Hücre dikey ortalama:** REST grid'lerinde ve **edit modu** tablo düzenlemesinde girilen metin/imleç artık hücrenin ortasında (üste yapışmıyor).

0.20.3'te ne değişti — 🌐 REST İstemcisi (YENİ) + Log Analizi durdurma + Mongo hız:

- 🌐 **REST İstemcisi (YENİ araç):** SOAP istemcisinin kardeşi bir HTTP/REST test aracı. Sol raydan açılır. Method (GET/POST/PUT/PATCH/DELETE) + URL + query/başlık/gövde (JSON) + Basic/Bearer kimlik → **Gönder** → durum kodu · süre · boyut + **renklendirilmiş JSON yanıt**. Kurumsal proxy + localhost muafiyeti hazır (SOAP altyapısı). Ayrıntı için **Bölüm 32**.
- **AI Yorumla + geçmiş/kayıtlı istekler:** yanıtı asistana yorumlatma; her gönderim geçmişe düşer, istekler adla kaydedilip geri yüklenir. (İlk sürümdeki yanıt→grid · ortamlar · cURL, 0.20.4'te sadeleştirilerek kaldırıldı — üstteki kutuya bakın.)
- **Log Analizi artık zaman aşımına düşmüyor + ■ Durdur:** dev log tablolarında analiz artık **süre sınırı olmadan** çalışır (eskiden 30 sn'de kesilip hep zaman aşımı dönüyordu). Örneklem sonuçları **hemen** görünür, gerçek toplam arkadan gelir; çalışan analizi/detay sorgusunu ■ **Durdur** ile anında keser (o an örneklem sonuçları ekranda kalır).
- **MongoDB koleksiyonları anında yükleniyor:** Log Analizi ve tablo seçicilerinde Mongo koleksiyon listesi artık tüm alanların taranmasını beklemez — sadece adlar (tek tur) gelir; alanlar koleksiyonu seçince yüklenir.

0.20.2'de ne değişti — 📊 Log Analizi 2.0 + nesne gezgini geliştirmeleri:

- **GERÇEK toplam sayılar:** Log Analizi artık "örneklemdeki" değil, **tüm tablo/aralıktaki gerçek tekrar sayısını** gösterir — bir hata imzasının kalıbı tek taramada sayılır (dev tabloda bile). Sayım alınamazsa (çok büyük tablo, süre aşımı) örnekleme düşer ve bunu **açıkça söyler**.
- **Seviye süzgeci + renkli rozet:** seviye kolonu (Level) otomatik bulunur; **Hata / Uyarı / Bilgi** çipleriyle süzersiniz (süzgeç sorguya girer — karışık log tablosunda hatalar öne

çıkar), her grupta renkli seviye rozeti görünür.
- **Detaya in (çift tık):** bir gruba çift tıklayın → arkasındaki **gerçek ham kayıtlar** (tüm kolonlarıyla) yeni pencerede + o kayıtların **zaman dağılımı grafiği** (saatlik/günlük/...): "bu hata artıyor mu, ne zaman yoğunlaştı".
- **İlk görülme + "YENİ" işareti:** her grubun ilk ve son görülme zamanı; ilk kez son 24 saatte görülen hatalar **YENİ** rozetiyle işaretlenir.
- **Konfor:** sonuç listesinde arama · listeyi **Kopyala / CSV'ye aktar** · son seçilen veritabanı/tablo/mesaj kolonu **hatırlanır** (bir dahaki açılışta hazır gelir).
- **Nesne gezgini — kolon arama:** gezgin arama kutusu artık nesne adının **yanı sıra kolon adlarında da** süzer — "`musteriId` hangi tablolarda var?" tek kutuda yanıtlanır (bellekten, ağ yok); eşleşen kolon ağaçta **kalın** gösterilir.
- **Nesne gezgini — 🎨 Görsel sorguya ekle:** bir tabloya sağ tık → **"Görsel sorguya ekle"** ile tabloyu doğrudan Görsel Sorgu tuvaline koyarsınız (sürüklemeye gerek yok; SQL Server).

0.20.1'de ne değişti — Yerel (localhost) WCF servisleri: URL ve Dosya ile WSDL yükleme:

- **Yerel adresler artık proxy'ye takılmıyor:** `localhost` / `127.0.0.1` ve noktasız (intranet) adreslerdeki servisler kurumsal proxy'den **muaf** tutulur — tıpkı tarayıcı ve SoapUI gibi. Böylece kendi makinenizdeki (ör. `http://localhost:46792/Mersis.svc?wsdl`) WCF servisi **URL ile** sorunsuz yüklenir (eskiden proxy "HTTP 403 URLBlocked" döndürüyordu).
- **Dosya ile yükleme de artık şemayı çözer:** WSDL'i dosyaya kaydedip yüklediğinizde, içindeki şema parçaları (`?xsd=xsd0…` — WCF bunları ayrı yayınlar) yine yerel adresten indirildiği için **tüm operasyonların örnek zarfı** üretilir. Önceki sürümde bu parçalar proxy'ye takıldığından her operasyon "öğesi şemada bulunamadı" diyordu; bu bitti (servis çalışır durumdayken).

0.20.0'da ne değişti — URL'den WSDL yükleme: kurumsal filtre/WAF engeli (HTTP 403):

- **Tarayıcı gibi istek:** WSDL indirme ve SOAP çağrıları artık tarayıcıya benzer `User-Agent` ve `Accept` başlıklarıyla gider. Bazı kurumsal web filtreleri/WAF'lar tarayıcı dışı ("araç") istekleri bot sayıp **403** döndürüyordu; tarayıcının açabildiği çoğu URL artık uygulamadan da açılır.
- **Anlaşılır 403 mesajı:** yine de engellenirse artık "kurumsal web filtresi/güvenlik duvarı bu URL'i engelliyor olabilir — WSDL'i tarayıcıdan indirip 📁 **Dosya** ile yükleyin ya da ağ yöneticinizden URL'i izin listesine ekletin" yönlendirmesi görünür (eski jenerik hata kodu yerine).

0.19.18'de ne değişti — SP Sihirbazı Tanımla adımı + v19 kapanış cilası:

- **Tanımla adımı tam ekranı kullanır:** SP Sihirbazı'nın ilk adımındaki GİRİŞ/ÇIKIŞ alanları artık ekranın tüm yüksekliğini doldurur (eskiden üstte kümelenip alt boş kalıyordu) — çok

sayıda kolon adı için bol yer.

0.19.17'de ne değişti — 🪄 SP Sihirbazı kullanım kolaylığı + tam ekran + görsel cila:

- **Aynı ad çok tabloda mı? Tabloyla nitele:** giriş/çıkış alanında `degisiklikTanim.adi` ya da `ortak.degisiklikTanim.adi` yazarsanız o alan **doğru tablodan** alınır — "sadece adi/Id yazınca çok tablo geliyor" sorunu bitti (parametre/kolon adı otomatik son parçadır).
- **Eksik JOIN bağını elle ekleyin (YENİ):** FK tanımlı değilse ve ad kuralı da tutmuyorsa (ör. `degisiklik.degisiklikId = degisiklikTanim.Id`), Eşle adımındaki **"Elle JOIN ekle"** ile iki tablo/kolonu bağlarsınız; sihirbaz o tabloyu (ve gerekiyorsa aradaki köprüleri) JOIN'e katar. Böylece **her senaryoda** doğru sorgu çıkar.
- **JOIN yolunu düzenleyin:** her JOIN adımının türünü **INNER/LEFT** seçer, ◀▶ ile sırasını değiştirirsiniz (bağımlılığı bozan taşıma reddedilir).
- **Tam ekran:** SP Sihirbazı, SOAP İstemcisi ve diğer büyük araç pencereleri (Karşılaştırma, Log Analizi, Geri Al, İçe/Paket Aktarım, Bağımlılık Ağacı, Veri Arama, Nesne Tarihçesi, Tablo Oluştur…) artık **tam ekran** açılır. Açık pencereler arasında **Alt+Tab** ile serbestçe geçersiniz.
- **Görsel cila:** pencerelerdeki bölme çubukları artık koyu temada açık bir bant bırakmıyor (üstüne gelince beliren ince tutamaç); CREATE OR ALTER seçeneği kaldırıldı (SP hep CREATE).

0.19.16'da ne değişti — 🪄 SP Sihirbazı yeniden tasarlandı + WSDL proxy düzeltmesi:

- **SP Sihirbazı artık adımlı ve profesyonel: ① Tanımla → ② Eşle → ③ Koşullar & Kurallar → ④ Önizle & Üret.** Üstteki adımlara tıklayarak gezersiniz; pencere büyür/taşınır.
  - **Canlı SQL önizleme:** son adımda üretilecek SQL **renkli** ve **her değişiklikte anında güncellenir**. 📋 Panoya kopyala / 🪄 Editörde aç.
  - ② **Eşle:** her çıkışı **yeniden adlandırın (AS)** ve **toplama** (COUNT/SUM/MIN/MAX/AVG) seçin — toplamada GROUP BY otomatik. **JOIN yolu** görünür; her bağın türünü **INNER/LEFT** seçersiniz (ad kuralından çıkarılanlar LEFT + uyarı).
  - ③ **Koşullar:** WHERE koşulları **AND/OR** ile; parametre kuralları (IF ile "0 olamaz" gibi kontroller); **ORDER BY** sıralaması.
  - ① **Tanımla: CREATE OR ALTER**, **TOP (n)**, **DISTINCT**. Karşılığı olmayan çıkış hâlâ `NULL AS`; FK tanımsızsa yol ad kuralından (XId→X) bulunur.
- 🔌 **SOAP — URL'den WSDL yükleme (proxy düzeltmesi):** kurumsal proxy arkasında URL'den WSDL yüklerken alınan **HTTP 407 (Proxy Authentication Required)** giderildi. WSDL indirme ve SOAP çağrıları artık sistem proxy'sini **Windows oturum kimliğinizle** (entegre kimlik) geçer — tarayıcının yaptığı gibi. Proxy ayrı bir hesap istiyorsa hata mesajı bunu açıkça söyler.

0.19.15'te ne değişti — kritik kararlılık + SP Sihirbazı + SOAP araması:

- 🛡 **Büyük sonuçta ÇÖKME giderildi (kritik):** dev bir sorgu (ör. yığın izli devasa bir log koleksiyonu ya da geniş bir tablo) çalıştırıp gride tıklamak uygulamayı düşürebiliyordu. İki koruma eklendi: MongoDB okuması artık SQL tarafındaki gibi **bellek bütçesiyle** sınırlı (aşılınca "ilk N satır" bandı çıkar, veri kaybı yok); dev sonuçta **Ctrl+A / "tümünü seç"** artık milyonlarca seçim nesnesi üretmez — eşik üstünde seçim yapılmaz, istatistik (COUNT/SUM/…) tüm sonuç üzerinden arka planda hesaplanır. **Beş motorda da** geçerli.
- 🪄 **SP Sihirbazı çok daha akıllı:**
  - **Kendi yolunu bulur:** girişte yalnız aradığınız alanı yazın (ör. TcKimlikNo) — sihirbaz `XId` biçimli kolonlardan (KisiId → Kisi, FirmaId → Firma) hedef tablolara **kendisi** JOIN yolu kurar; ara tabloları yazmanız gerekmez. **FK tanımlı olmasa bile** ad kuralından çıkarır (bu bağlar LEFT JOIN + "doğrulayın" notuyla gelir).
  - **Çıkışları tüm tablolarda arar:** yazdığınız her çıkış alanı hangi tabloda varsa oradan bulunur; birden çok tabloda varsa aday listesinden seçersiniz.
  - **Karşılığı yoksa boş kolon:** hiçbir tabloda olmayan bir çıkış alanı yazarsanız SP'ye `NULL AS [Alan]` olarak yine eklenir (eşleşmeyen giriş de parametre olarak konur).
  - **Zengin taslak:** üretilen SP artık başında analiz özeti (hangi ad hangi tabloya), hizalı SELECT ve açıklamalı JOIN'lerle gelir — tek satırlık basit sorgu değil.
- 🕯 **SOAP istemcisinde operasyon araması:** WSDL'den yüklenen operasyon listesinin üstüne, nesne gezginindeki gibi bir **arama kutusu** geldi (büyüteç + ✕) — çok operasyonlu serviste aradığınız metoda yazdıkça süzerek ulaşırsınız.

0.19.14'te ne değişti — canlı test düzeltmeleri (akıcılık + kullanım kolaylığı):

- **Sorgu akarken uygulama artık donmuyor:** büyük sonuçlar (ör. dev bir log koleksiyonu) sunucudan akarken arayüz — açık araç pencereleri dahil — serbest kalır; "(Not Responding)" donmaları giderildi. **Beş motorda da** geçerli.
- **Sekmeler anında kapanır:** sekme kapatınca bağlantı kapatma/geri alma işleri arka planda yürür — "donuyor gibi geç kapanıyor" davranışı bitti.
- 🗺 **Veritabanı Haritası doğru veritabanıyla açılır:** artık hep listedeki ilk DB değil, **aktif sekmenin** veritabanı çizilir; sekme başlığında hangi DB'nin resmedildiği yazar (🗺 Harita · SatisDb).
- 🔀 **Eşitlemede ☑ seçim kolonu:** şema/veri fark listelerinde satırları onay kutusuyla işaretleyip ✅ Seçilenleri eşitle dersiniz — Ctrl/Shift bilmek gerekmez.
- **Mongo find yardımcısı: yalnız DEĞERLERİ yazın.** Alan adını yazıp öneriyi seçince `"Alan": {` kendiliğinden gelir; `$ne` gibi operatörler artık her konumda önerilir ve iç yapıyı kendisi kurar; süslü parantez / tırnak / kapanış derdi kalmadı — `KullaniciId: { $ne: -666` gibi yarım girişler bile Üret'te kendiliğinden onarılır.

0.19.13'te ne değişti — büyük paket: arama · bağımlılık · eşitleme · 10 tema · çoklu motor eşitliği:

- 🔎 **Veri Arama (YENİ):** sol menüden bir değeri (TC no, sicil, metin…) **tüm tabloların verisinde** arar — hangi tablolarda geçtiğini listeler, çift tıkla satırlar sekmede açılır. Sayı yazarsanız sayısal+metin, metin yazarsanız metin kolonları taranır. Varsayılan tarama tablo başına 10 sn ile korunur; **"Tablolar ▾ "**den kendiniz seçerseniz süre SINIRSIZDIR ve süreye takılan tablolar **"⏱ Atlananları seçip SINIRSIZ ara"** ile ikinci turda taranır. Eski "Metin ara" aracının adı **"Kod Arama"** oldu (nesne tanımlarında arar) — ikonları da ayrıştı.
- 🔗 **Bağımlılık ağacı (YENİ):** tablo/view/SP'ye sağ tık → **⬅ bunu kullananlar** (ALTER'dan kim etkilenir?) ve **➡ bunun kullandıkları** (SOAP arkasındaki SP zinciri) ağaç olarak; düğüm açtıkça derinleşir, döngüler ↻ ile işaretlenir, çift tık script'i sekmede açar. Dört SQL motorunda da çalışır (PostgreSQL/MySQL'de fonksiyon eşleşmeleri ~ işaretli yaklaşıktır; MySQL view bağımlılıkları için 8.0.13+ gerekir; Oracle ALL_DEPENDENCIES ile kesindir).
- 🔀 **Karşılaştırmada EŞİTLEME (YENİ):** hem **veri** hem **şema** fark listelerinde artık satırları seçip (*Ctrl/Shift ile birden çok*) **"Seçilenleri eşitle"** ya da **"Tümünü eşitle"** diyebilirsiniz — hedefi kaynağa eşitleyen script üretilir: veri için DELETE→UPDATE→INSERT, şema için CREATE/ADD/DEĞİŞTİR + ayrı ⚠ **YIKICI bölümde** DROP'lar; FK ekleme/kaldırma, PK ekleme/kaldırma, index CREATE/DROP ve **view/SP/fonksiyon gövde değişimi** dahildir. Script **çalıştırılmaz** — kopyala/kaydet penceresinde gelir, HEDEF bağlantıda Güvenli Yazma ile koşarsınız.
- 🎨 **Tema seçici (YENİ — 5 açık + 5 koyu):** sağ üstteki tema düğmesi artık menü açar: ☀ **Açık Tema** ve 🌙 **Koyu Tema** altında 5'er varyant (İndigo · Grafit · Slate Mavi · Petrol · Amber) — her öğenin yanındaki renk yuvarlağından seçersiniz, tercih kalıcıdır. Yerleşim/menüler/düğmeler değişmez, yalnız renk tonları. Görsel cila: menüler koyu temada da net okunur (koyu zemin + beyaz yazı), kapalı sol menüde ikonlar ortalı ve ferah, editör araç şeridi çözünürlüğe uyarlanır (geniş ekranda tek satır, dar ekranda düzenli kırılır).
- 🕯 **SOAP kimlik başlıkları:** Bearer token ve 🔐 **WS-Security UsernameToken** desteği (Basic zaten vardı) — ortam profiliyle birlikte şifreli (DPAPI) saklanır.
- **Küçük ama gündelik:** yeni sekme **aktif sekmenin veritabanıyla** açılır · nesne gezgininde bir veritabanına tıklayınca aktif sekme o veritabanına geçer · Log Analizi'nde Mongo koleksiyonu seçilince mesaj alanları **anında** dolar (tam envanter beklenmez).
- **Çoklu motor eşitliği:** Veri Arama · Kayıt Haritası (+DELETE script'i) · INSERT örneği artık **PostgreSQL, MySQL ve Oracle'da da** var — tırnaklama/limit/değer fonksiyonları (now() · NOW() · SYSDATE…) motora göre üretilir.
- **Sağlamlaştırma (10 düzeltme):** Mongo sorgularına gerçek zaman aşımı (sunucu tarafında kesilir) · profil değiştirirken geç gelen yanıt yeni ekranı artık ezemez · tr-TR'de "3.14" yapıştırınca sessizce 314 olmaz · Geri Al script'inde binary/tarih kolonları artık doğru literalle yazılır · satır sınırında kesilen çoklu sorgularda "sonraki ifadeler çalışmadı"

açıkça söylenir · CSV dışa aktarmada formül enjeksiyonu koruması (ve içe aktarmada geri soyulması) · IPv6 sunucu adresleri · kapatılan sekmelerin belleği hemen bırakılır.

0.24.7'de ne değişti — 🕸 Kayıt Haritası artık yalnız DOLU olanı getiriyor:

- **Boş ilişkiler gelmiyor.** Harita, sorguları açmadan önce her ilişkiyi tek tek yoklar ("bu kayda bağlı en az bir satır var mı?") ve **yalnız dolu çıkanları** SELECT olarak getirir. Eskiden bütün ilişkiler koşar, çoğu boş grid olarak açılırdı. Boş çıkanların dökümü yazılmaz; yalnız **kaç tanesinin** elendiği başlıkta belirtilir.
- **Bir adım öteye gidiyor.** Dolu çıkan bir tablonun kendi bağlantıları da (o da doluysa) haritaya eklenir — başlıkta **▲▲ / ▼▼** ve "… üzerinden" diye görünür. Örnek: Sipariş → Sipariş Satırları (dolu) → o satırların Ürünleri.
- **Tanım/lookup tabloları getirilmiyor.** Durum, tip, kod gibi tanım tablolarına giden **▲ üst** bağlar atlanır (hangileri atlandı, script başında tek satırda yazar). Bir tanımı *kullanan* gerçek kayıtlar (▼ alt) elenmez — asıl aradığınız onlardır.
- **Bilmeniz gereken:** harita artık açılmadan önce veritabanına küçük sorgular atar; durum çubuğunda ilerlemeyi görürsünüz. Çok dallanan şemalarda en fazla **200** ilişki yoklanır ve script'e bunu söyleyen bir uyarı düşülür.

0.19.12'de ne değişti — 🕸 Kayıt Haritası (ilk sürüm):

- **Sonuçtaki satıra sağ tık → "🕸 Kayıt haritası":** o kaydın yabancı anahtarlarla bağlı **▲ üst** (satırın işaret ettikleri) ve **▼ alt** (bu satırı işaret edenler) kayıtları tek tıkla getirilir — her ilişki ayrı grid, ilk kolonda hangi bağdan geldiği yazar; sorgular düzenlenebilir script olarak sekmededir (ilişki başına ilk 100 satır).
- **"🗑 Bağlılarıyla DELETE script'i":** önce bağlı alt kayıtlar, sonra satırın kendisi için güvenli sıralı DELETE script'i üretir — **çalıştırmaz**; Güvenli Yazma açıkken koşup 🔍 ile silinecekleri görüp COMMIT/ROLLBACK'e siz karar verirsiniz. Üst kayıtlar bilerek dahil edilmez; PK'sız tabloda üretilmez.
- Tek tablolu SELECT sonuçlarında çalışır; FK tanımsız ilişkiler görünmez (dürüstçe söylenir).

0.19.11'de ne değişti — 🪄 SP Sihirbazı:

- **Sol menüde yeni araç:** giriş parametrelerinizi ve çıkış kolonlarınızı yazın (örn. giriş: *TcKimlikNo* — çıkış: *Ad, Soyad, Unvan*) → **Analiz et**: adlar tüm tablolarda aranır (aynı ad birden çok tablodaysa aday listesinden seçersiniz) → **SP üret**: tablolar yabancı anahtarlardan JOIN'lenir (aradaki köprü tablolar otomatik bulunur) ve **CREATE PROCEDURE taslağı editörde açılır** — asla kendiliğinden çalıştırılmaz; canlı denetim sözdizimini anında yoklar, gözden geçirip siz çalıştırırsınız.
- FK ile bağlanamayan tablo olursa dürüstçe söylenir ve JOIN'i elle kurmak için Görsel Sorgu'ya yönlendirilirsiniz. İpucu: hazır taslağı 🤖 AI sohbetine yapıştırıp geliştirtebilirsiniz.

0.19.10'da ne değişti — büyük paket: yeni özellikler + kapsamlı sağlamlaştırma:

- 📤 **Şema Kopyalama (YENİ):** veritabanına sağ tık → "Şemayı başka bağlantıya kopyala" — tablo+PK+index+FK+view/SP/fonksiyon hedef bağlantıya kurulur (veri kopyalanmaz); hedef veritabanı yoksa oluşturulur; "📄 Script'i kaydet" ile tek .sql dosyası da alınabilir.
- **＋ Yeni veritabanı (YENİ):** nesne gezgini şeridindeki ＋ ile CREATE DATABASE (SQL Server/PostgreSQL/MySQL).
- 🔍 **COMMIT'ten önce değişiklikleri gör (YENİ):** Güvenli Yazma karar bandındaki yeni butonla etkilenen satırların **önceki** ve **şimdiki** hali alt alta iki grid'de (UPDATE/DELETE).
- **Editör:** belgedeki **@parametreler artık öneride** ('@' yazınca kendiliğinden açılır); Görsel Sorgu'ya **veritabanı seçici** geldi (tablo sürükleyince o tablonun DB'sine geçer).
- **Canlı denetim güçlendi:** sorguyu yapıştırınca **çalıştırmadan** TÜM sözdizimi hataları gelir — her hatalı satır dalgalanır, döküm Mesajlar'da.
- **Karşılaştırma dili netleşti:** "yalnız solda/sağda" yerine **yalnız kaynakta / yalnız hedefte**; tek taraftaki satırların değer detayı da gösterilir.
- **Cilalar:** sonuç bölgesi kapalıysa sorgu bitince kendini açar (0 satırda da grid başlıklarla görünür); satır sayısı+süre gridin üstünde; Ctrl+F arama kutusundaki metin kırpılması kökten çözüldü; içe aktarım önizlemesi "aktarımda TÜM dosya okunur" güvencesini açıkça yazar.
- **Kapsamlı sağlamlaştırma:** tam kod incelemesi + yoğun-veri stres turu — 7'si kritik 24 düzeltme (Güvenli Yazma işlem güvenliği, PostgreSQL aktarım veri kaybı koruması, aktarımda tablo adı tırnaklama, arka plan sekmesinin yanlış sorgu çalıştırması vb.).

0.19.9'da ne değişti — kararlılık & hız:

- **Büyük tabloda çökme giderildi:** sınırlamasız "SELECT * FROM devasaTablo" artık uygulamayı düşürmüyor. Sonuç güvenli bir bellek bütçesinde (gerçekçi tahminle) kesilir; "ilk N satır" bandı görünür — veriye zarar yok, "Tümünü dışa aktar" hâlâ sınırsız (diske akıtır).
- **Log Analizi donmuyor:** zaman kolonu indekssiz devasa log tablolarında sorgu artık en çok 30 sn çalışır; aşarsa "zaman aşımı" mesajı verir (sonsuz donma yerine). Son 24 saat sorgusu en yeni kayıtları önce getirir (zaman kolonu indeksliyse belirgin hızlanır).
- **MongoDB koleksiyonları anında görünür:** nesne gezgininde koleksiyon adları hemen listelenir, alan envanteri arka planda dolar — artık tüm koleksiyonların örneklenmesini beklemezsin.

0.19.8'de ne değişti — akıllı tamamlama · kolay INSERT · SOAP:

- **Editör otomatik tamamlama — büyük güncelleme:**
  - **JOIN … ON otomatik FK ⭐:** iki tabloyu JOIN'leyip "ON" yazınca yabancı anahtardan "s.MusteriId = m.Id" eşitliği en üstte ilk öneri gelir — `Enter` ile alırsın (otomatik yazılmaz).

    - **Kendiliğinden açılır:** FROM / JOIN / WHERE / ON sonrası boşlukta öneri penceresi `Ctrl+Space`'siz açılır.
    - **Karışan kolon netleşir:** JOIN'de aynı adlı kolon (Id) varsa "a.Id" / "b.Id" ayrı ayrı önerilir.
    - **"⋯ tüm kolonlar":** SELECT'te tek seçimle FROM tablosunun tüm kolonları eklenir.
    - **WHERE'de temiz liste:** yüklemde (WHERE/ON) yalnız mantık sözcükleri (AND / OR / LIKE …) önerilir.
    - **Parametre ipucu:** "EXEC sp " ya da "fonksiyon(" yazınca parametreler yüzen ipuçta görünür.
- **Kolay INSERT:** tabloya sağ tık → **INSERT örneği** → kaç satır sorulur → tip-farkında, çok satırlı örnek INSERT açılır (identity/computed/rowversion kolonları otomatik atlanır).
- **SOAP — WSDL dosyadan:** WSDL'i URL yerine 📁 **Dosya** ile yerel .wsdl/.xml'den yükleyebilirsin.
- **SOAP — XML renkleri düzenlendi:** zarf/yanıt XML'inde etiket, öznitelik adı/değeri ve **girdiğin değerler ayrı renklerde** (standart XML editörü gibi); zemin temaya göre (Bölüm 28).

0.19.7'de ne değişti — okunabilirlik cilası:

- **Seçili sorgu okunur:** editörde bir sorguyu fareyle seçince (çalıştırmak için) metin artık her iki temada net okunur — seçim rengi sözdizimi renkleriyle çakışmıyor.
- **Arama kutusu düzeldi:** `Ctrl+F` arama kutusunda yazılan karakter artık dikey ortalı (aşağı kaymıyor).
- **SOAP XML her zaman renkli:** SOAP istemcisindeki zarf ve yanıt XML'i SoapUI gibi **sabit açık zeminde** renklidir — koyu temada da renkler kaybolmaz.

0.19.6'da ne değişti:

- **SOAP ekranında XML renkli:** istek zarfı ve yanıt dökümü artık **sözdizimi vurgulu** (satır numaraları, renkli etiket/öznitelik/yorum) — büyük XML'ler çok daha kolay okunur (Bölüm 28).
- **Araç pencereleri minimize edilince kaybolmuyor:** SOAP, Log Analizi, Paket Aktarım, İçe Aktar, Geri Al gibi pencereleri küçültünce artık **görev çubuğundan** geri getirebilirsiniz.

0.19.5'te ne değişti:

- **Yazarken maliyet uyarısı kaldırıldı.** Girişken sarı bant yerine, editör araç çubuğundaki `Performans değerlendir` (eski "Neden yavaş?") ile **istediğinizde** performans/darboğaz raporunu alırsınız (Bölüm 30). Canlı sözdizimi denetimi (yazarken kırmızı dalga) sürüyor.
- **Log Analizi — Mongo koleksiyonları hızlandı:** koleksiyon alan envanteri artık paralel çıkarılır; çok koleksiyonlu veritabanında ekran çok daha hızlı açılır.

- **Log Analizi — "Son görülme" kolonu:** her hata grubunun **en son ne zaman görüldüğü** grid'de yeni bir sütunda gösterilir (zaman kolonu olan tablo/koleksiyonlarda).

0.18.0'da ne değişti — 5 acil iyileştirme:

- **Editörde kod katlama:** SQL Server'da `BEGIN…END`, `CASE…END` ve çok satırlı `/* */` yorumları sol kenardaki [−]/[+] işaretiyle katlanır — uzun prosedürlerde IF/WHILE/TRY gövdelerini gizleyip toparlanın.
- **Log Analizi okunabilirliği:** çok satırlı hata mesajları artık satırları bindirmiyor — listede tek satırlık önizleme + tam metin için altta okunur/kaydırılır panel.
- **SOAP İstemcisi — Basic auth:** WSDL/servis 401 istiyorsa kullanıcı adı + parola girin; ortam profiliyle kaydedilir (parola DPAPI ile şifreli).
- **Prosedürü çalıştır penceresi:** parametreli SP'ye sağ tık → "Çalıştır" artık SSMS gibi bir değer giriş penceresi açar; değerleri girip çalıştırın (metinler otomatik tırnaklanır, OUTPUT değerleri geri okunur).
- **AI asistanı dayanıklılığı:** yapılandırma bir an eksik okunursa artık ona kilitlenmiyor — sonraki denemede kendini tazeliyor.

0.17.0'da ne değişti — ⏪ ZAMAN MAKİNESİ (v15):

- 🔍 **Sorgu provası (Ctrl+Shift+P):** UPDATE/DELETE'inizi **çalıştırmadan** etkisini gösterir — "kaç satır etkilenecek + ilk 10'u" + hangi kolonların değişeceği. Editörün üstünde bant çıkar; hazırsanız ▶ **Şimdi çalıştır**. "Önce SELECT atıp bakayım" alışkanlığının tek tuşa inmiş hali (yalnız SQL Server).
- ⏪ **Geri Al paketleri (sol ray):** Güvenli Yazma modunda COMMIT ettiğiniz her UPDATE/DELETE'te, etkilenen satırların **eski hali** otomatik saklanır. Sonradan "Geri Al" penceresinden paketi seçip **tersine çeviren DML'i** üretirsiniz — yeni sekmede açılır, inceleyip Güvenli Yazma ile koşarsınız. Bir gün önceki bir UPDATE'i geri almanın hafif araçlarda örneği yok.

0.16.2'de ne değişti:

- **Editörde canlı sözdizimi denetimi (SQL Server):** yazmayı bıraktıktan ~1 sn sonra sorgunuz sunucuda yalnız *parse* edilir — sözdizimi hatası varsa kırmızı dalgalı çizgi hatalı satırda **daha çalıştırmadan** yanar, düzeltince söner. Hiçbir şey çalıştırılmaz.
- **Bakım sekmesine bekleme analizi:** ŞU AN bekleyen istekler (ne bekliyor, ne kadardır, kim blokluyor) + sunucu genelinde en yüklü 15 bekleme türü (yüzde paylı) — "sunucu neden yavaş?" sorusunun ilk bakacağınız iki tablosu.
- **AI asistanı yapılandırması kilitlendi:** ⚙ ayar düğmesi kaldırıldı — sağlayıcı/model/anahtar kurulumla gelir, uygulama içinden görüntülenemez ve değiştirilemez (bkz. Bölüm 25).

0.16.1'de ne değişti — canlı test düzeltmeleri:

- **SOAP İstemcisi:** ortam profilleri, istek geçmişi ve AI Yorumla artık çalışıyor (pencereye kalıcı depo bağlantısı geçmiyordu — kaydet/seç boş kalıyordu).
- **NOLOCK açılışta seçili:** Kilitsiz okuma anahtarı uygulama her açıldığında işaretli başlar (yalnız SQL Server; istemezseniz kapatın).

0.16.0'da ne değişti — 🔌 SOAP/WCF TEST İSTEMCİSİ (v14) (bkz. Bölüm 28):

- **Sol rayda yeni düğme: SOAP İstemcisi** — WCF/SOAP servisini SQLST içinden test edin: WSDL adresini yükleyin → operasyonu seçin → **örnek değerlerle hazır gelen SOAP zarfını** düzenleyin → Gönder. Yanıt girintili, süreyle birlikte; **Fault açıkça işaretlenir**. Tipik akış: SP'yi SQLST'de değiştir, aynı araçta servis katmanının hâlâ doğru döndüğünü doğrula.
- **İstek geçmişi kalıcı:** her gönderim listeye düşer (✓/⚠/✕ + süre); **çift tık** isteği geri yükler. **Ortam profilleri:** test/canlı WSDL+adresini adla kaydedin, tek seçimle geçin.
- **AI Yorumla:** istek + yanıt zarfını yapay zekaya özetletin — Fault'ta neden + muhtemel düzeltme önerilir.
- **SP'den servis iskeleti:** nesne gezgininde SP'ye sağ tık → 🔌 **Servis iskeleti üret (WCF)** — SP imzasından ServiceContract + ADO.NET çağrısı içeren hazır C# kodu sekmede açılır (OUTPUT parametreler dahil).

0.15.1'de ne değişti — canlı test cilası (kullanıcı bulguları):

- **İçe Aktar'da dosya TÜRÜ seçimi:** Excel'de ek ayar yok (hep **ilk sayfa** okunur — sayfa seçimi kaldırıldı) · **CSV'de kolon ayracı otomatik algılanır** (soru sorulmaz) · TXT'de **kolon VE satır ayracı** seçilir (tek satırlık `kayıt1|kayıt2|…` ihraç dosyaları da açılır).
- **Edit modunda sorgu:** ▶ Çalıştır düğmesi artık kutunun hemen yanında; `F5` / `Ctrl+Enter` de çalıştırır. Edit sekmeleri de **SQLST-N** adıyla açılır (╍ SQLST5 gibi).
- **Açık temada sol ray bir ton açık** (koyu temada eski koyu ton sürer).

0.15.0'da ne değişti — 📥 EXCEL/TXT İÇE AKTAR (v13) + editör iyileştirmeleri (bkz. Bölüm 27):

- **Sol rayda yeni düğme: İçe Aktar** — Excel (.xlsx/.xls) ya da TXT/CSV dosyasını okuyup bağlı sunucudaki bir tabloya aktarır. TXT'de **kolon ayracı** (listeden ya da tek karakter) ve **kodlama** (UTF-8 / Windows-1254) seçilir; Excel'de **sayfa** seçilir.
- **Önizleme:** ilk 200 satır tablo görünümünde; her kolonun başlığında **tahmin edilen tip** (tam sayı · ondalık · tarih · metin(N)). Sayı/tarih yorumu seçtiğiniz **Biçim** kültürüne göredir — Türkçe'de `1.250,75` ve `24.07.2026` doğru anlaşılır.
- **Eşleme + aktarım:** hedef tabloyu seçince kolonlar aynı ada göre otomatik eşlenir (🔑 işaretli, boş = aktarma); v12'deki raylar aynen: yazma kipi (yalnız ekle / 🔑 Ekle-Güncelle), önce temizle onayı, hata politikası, Durdur, kopyalanabilir rapor.

- **YENİ TABLO yolu:** hedef kutusuna listede olmayan bir ad yazın — kolonlar tahmin edilen tiplerle `CREATE TABLE` script'ine dönüşür, script **onayınıza sunulur**, onayla tablo oluşturulup aktarım başlar (onaysız hiçbir DDL çalışmaz).
- **Editör sekmeleri artık HEP SQLST-N:** tabloya SELECT, nesne scriptleme, görsel sorgu, AI… hangi yoldan açılırsa açılsın ad SQLST1, SQLST2… olur (geçmişten açılan kayıt, kayıtlı adıyla gelmeye devam eder).
- **Edit modunda sekme içi sorgu:** ayrı "Sorgu penceresi" kaldırıldı — sorguyu Edit sekmesinin üstündeki kutuya yazın, **sonucu aynı gridde düzenleyin** (PK kolonlarını SELECT etmeyi unutmayın; bkz. Bölüm 13).

0.14.0'da ne değişti — 📦 PAKET AKTARIM (v12): veriyi taşı/arşivle (bkz. Bölüm 26):

- **Sol rayda yeni düğme: Paket Aktarım** — SSIS mantığında veri taşıma. Kaynak, bağlı olduğunuz sunucu; hedef, **kayıtlı profillerinizden** seçilir (elle bağlantı bilgisi girilmez).
- **SQL ailesinde Tablo→Tablo:** kolonlar aynı ada göre **otomatik eşlenir** (son karar sizin); motorlar-arası da çalışır (ör. SQL Server → PostgreSQL). **Sorgu→Tablo sekmesi:** kaynakta serbest SELECT yazın, `Kolonları getir` sorguyu satır çekmeden inceler, sonucu hedef tabloya aktarırsınız.
- **Yazma kipleri:** "Yalnız ekle" ya da **"Ekle/Güncelle (🔑 anahtara göre)"** — hedefin birincil anahtarına göre var olan satır güncellenir, olmayan eklenir (UPSERT).
- **MongoDB'de koleksiyon→koleksiyon arşivleme:** ör. log koleksiyonunu arşiv veritabanına taşıma. İsteğe bağlı **find süzgeci**, alan eşleme gridi (yeniden adlandır/atla), **_id'ye göre** Ekle/Güncelle. Hedef veritabanı/koleksiyon adı **yeni de yazılabilir** — Mongo ilk yazışta oluşturur.
- **Güvenlik rayları:** başlamadan **özet onayı**; "önce temizle" ayrıca ve açıkça onaylatılır; 500'lük partiler + **Durdur**; hata politikası (ilk hatada dur / atla-raporla); bitişte **kopyalanabilir rapor** (okunan/eklenen/güncellenen/atlanan + hız + hata örnekleri).
- **Find yardımcısı taşındı ve akıllandı:** düğme artık editör araç çubuğunda (Biçimlendir'in yanında); Filter/Sort/Project kutuları yazımdan çıkınca kendiliğinden `{ }` 'ye sarılıp biçimlenir, bozuk JSON **Üret'i beklemeden** anında söylenir.

0.13.0'da ne değişti — 🤖 YAPAY ZEKA (v11): ÜCRETSİZ AI Asistan (bkz. Bölüm 25):

- **AI Asistan sekmesi (sohbet):** sol raydaki **AI Asistan** ile açılır — mesaj balonlu gerçek sohbet arayüzü, giriş kutusu altta. Doğal dille sorgu ürettirin ("siparişi olmayan müşterileri getir"), cevaptaki sorguyu ↗ **Sekmede aç** ile tek tıkla yeni sekmeye alın (asla otomatik çalıştırılmaz).
- **Sağlayıcılar:** varsayılan 🏠 **Yerel** — kendi AI ajanımız (Ollama + `qwen2.5-coder:7b`): anahtar gerekmez, internet gerekmez, **veri makineden çıkmaz**. Alternatif, kurumun kendi **OpenAI-uyumlu ucu** (Groq/OpenRouter/LM Studio); orada anahtar makinede şifreli (DPAPI) saklanır. `0.24.5` **Google Gemini kaldırıldı** — SQLST artık Google'a hiçbir istek

atmaz. Config'inde `"saglayici":"Gemini"` yazan makineler kendiliğinden 🏠 Yerel'e geçer (dosya kendini onarır; eski anahtarınız silinmez, yalnız kullanılmaz).

- **Sorduğunuz metne sadık kalır** 0.24.6 — serbest soruda asistan **yalnız sizin yazdığınızı** gönderir. Eskiden başka bir sorgu sekmesindeki metin (üstelik baktığınız sekme değil, *en son uğradığınız* sekme) kendiliğinden isteme giriyordu. Sorgu bağlamı isterseniz **"Editördeki sorguyu ekle"** kutusunu işaretleyin — artık bilinçli bir tercih.
  Şema yalnız gerektiğinde gider: "merhaba" gibi veriyle ilgisi olmayan bir mesajda tablo listesi gönderilmez. Bu bir hız meselesi: yerel model istemi okurken saniyede ~15 kelime işliyor, dolayısıyla gereksiz gönderilen tablo listesi **90 saniyeye** mal oluyordu. Soruda bir tablo adı geçerse ya da soru veriyle ilgiliyse şema yine gider.
- **Bağlam düğmeleri** — cevap ayrı pencerede, zengin metin olarak: sorgu sekmesinde **AI Değerlendir** (söz dizimi/mantık/risk + düzeltme) · **AI Açıkla** (sorgu ne yapıyor) · **AI Hatayı çözdür** (kırmızı hata → neden + düzeltilmiş sorgu); plan sekmesinde **AI Planı yorumlat**; Log Analizi'nde **AI Özetle** (kök neden + sonraki adım).
- **Gizlilik:** modele yalnız **ŞEMA** (tablo/kolon adları + tipleri) ve sizin yazdıklarınız/ ekrandaki gruplar gider — **satır verisi asla**; üretilen hiçbir sorgu kendiliğinden çalışmaz.
- **Görsel yenilikler:** editör araç çubuğu düğmeleri artık **canlı renkli ikon çipli** (yazılar nötr — daha okunaklı); yeni **AI kıvılcım** logosu; Plan ve Görsele çevir düğmeleri araç çubuğuna geri döndü; asistan cevapları **zengin metin** (kalın, kod çipleri, kutulu SQL).
- **Geçmişten sekme adıyla açılış:** geçmişteki kayıt hangi sekmeden çalıştıysa (SQLST3 gibi), çift tıkla açınca pencere **aynı adla** gelir.

0.12.1'de ne değişti — Mongo editör tamamlaması:

- **Sorgu sekmesinin editöründe Mongo otomatik doldurma:** find/aggregate JSON'unu elle yazarken de öneri gelir — kökte `find/filter/sort/projection/limit/skip/pipeline` anahtarları (hazır kalıbıyla girer), `"find"` değerinde **koleksiyon adları**, filter içinde **alanlar + $operatörler**, sort'ta **1/-1**, projection'da **1/0**, pipeline'da **$match/$sort/$project…** adımları. `"` `$` `{` `,` yazınca kendiliğinden açılır; `Ctrl+Space` her yerde.
- SQL anahtar sözcükleri Mongo sekmesine artık hiç sızmaz; kod parçaları (snippet) üstte kalır.

0.12.0'da ne değişti — "v11-öncesi" büyük paket (9 iş + yeni logo):

- **Yeni logo:** indigo-mor karo üzerinde veritabanı silindiri + amber şimşek — pencere/görev çubuğu ikonu, sol ray ve bağlantı ekranı aynı markayı taşır.
- **Sol menü artık hover'la açılır:** kapalıyken üstüne gelince yumuşak bir animasyonla açılır, çekilince kapanır; ☰ menüyü **sabitler** (hep açık), tekrar basınca hover moduna döner.
- **Log Analizi iki panel oldu: Günlük** (SON 24 SAAT bazlı — tüm tablo taranmaz) ve **Tüm Zamanlar** (başlangıç/bitiş tarihi seçilir, bitiş günü dahil; ikisi boşsa tüm tablo). Üstte ortak

veritabanı + tablo + mesaj kolonu seçicileri — seçimler artık görünür; **MongoDB dahil beş motorda da** çalışır (koleksiyon/alan).

- **Execution plan ve Metin ara sol raya taşındı** — editör araç çubuğu daha da sade; Plan aktif sorgu sekmesini hedefler.
- **Görsel Sorgu WHERE kolaylaştı:** kolonun yanındaki **huni** ikonuna tıkla → küçük pencerede operatör + değere gir (tarihse takvim, bit'se Evet/Hayır) → koşul üstte **çip** olarak dizilir; çipler arası **VE/VEYA** tıklamayla değişir, × kaldırır. (Kutuya çift tık da hâlâ çalışır.)
- **Veritabanı Haritası "Odak + Bağlam":** 30'dan çok tabloda açılış **mahalle kartlarıdır** (FK kümeleri: renkli kart + üye sayısı + öne çıkanlar; kümeler arası "N FK" bağı). Karta çift tık kümeyi açar; tabloya çift tık ya da aramada Enter **o tabloya odaklanır** (komşuluk halkası, Derinlik 1/2). Kalabalık görünümlerde kartlar kompakt çizilir.
- **Mongo Find yardımcısında Compass tarzı otomatik doldurma:** koleksiyon adları ve alanlar yazarken önerilir; Sort'ta alan sonrası **1/-1**, Filter'da `$` ile operatör listesi. Süslü parantezsiz girdi (`"_id": -1`) artık kendiliğinden sarılır.
- **Bağlantıyı değiştir düzeldi:** basınca bağlı ekran kapanır ve bağlantı ekranı açılır; yeni profile bağlanınca ana pencere yeni bağlantıyla gelir. X ile vazgeçilirse eski bağlantıya sessizce dönülmez — uygulama kapanır (açılıştaki davranışla tutarlı).

0.11.4'te ne değişti — editör renkleri + bağlantı ekranı:

- **Sorgu editöründe renkler daha net:** anahtar sözcükler (SELECT, FROM, WHERE…) **kalın canlı mavi**, sayılar **turuncu** — mavi/kırmızı/yeşilden ayrı, göze kolayca çarpar.
- **Bağlantı ekranı sığma düzeltmesi:** marka bandı eklendikten sonra alttaki "Zaman aşımı" satırı ve "Connection string'den doldur" düğmesi kırpılıyordu; pencere yüksekliği artırıldı, artık her şey sığıyor.

0.11.1'de ne değişti — Command Deck rötuşları:

- **Sol menü açılır-kapanır:** ray **açılışta kapalıdır** (yalnız renkli ikonlar; editör hemen geniş); üstteki ☰ düğmesiyle etiketli tam menü açılır, tekrar basınca kapanır.
- **Editör araç çubuğu renkli ikonlarla:** Çalıştır ▶ · Durdur (kırmızı) · Biçimlendir (mavi) · Plan (amber) · Metin ara (teal); Oto yenile/Güvenli yazma/NOLOCK anahtarları da renkli ikonlu.
- **Bağlantı ekranı yenilendi:** üstte indigo marka bandı, profil kartlarında motor-renkli nokta, ikonlu Bağlan/Test Et düğmeleri.

0.11.0'da ne değişti — "Command Deck" yeni tasarım: Uygulama baştan sona yeni bir görsel kimliğe kavuştu (5 taslak arasından seçilen):

- **Sol renkli komut rayı:** Görsel Sorgu · Veritabanı Haritası · Yönetim Paneli · Karşılaştır · Log Analizi artık solda **iri, etiketli, renk çipli** düğmeler — küçük sağ ikonların yerine. Büyük özellikler öne çıkar.

- **İndigo vurgu** + aydınlık çalışma alanı (koyu temada lacivert); her araç kendi renginde elde çizilmiş temiz ikon.
- **Üstte:** tıklanabilir bağlantı çipi + ikon-yazı bitişik arama/komut kutusu (Ctrl+P paleti) + tema.
- Nesne gezgini, sekmeler, sonuç grid'i ve tüm pencereler aynı indigo kimlikle güncellendi.

0.10.4'te ne değişti — yerleşim rötuşları:

- **Buton ipuçları anında açılıyor:** üst bardaki ikon araçların üzerine gelir gelmez ne yaptıkları görünür (pasif düğmede de — neden pasif olduğu okunur).
- **Editör araç çubuğu sadeleşti:** "Execution plan" → **Plan**, "Kilitsiz okuma (NOLOCK)" → **NOLOCK**; Çalıştır/Durdur düğmelerine de ipucu eklendi.
- **Sekme adları:** eski oturumlardan kalan "sorgu1" gibi adlar açılışta **SQLST1, SQLST2…** markasına taşınır (elle verdiğiniz adlara dokunulmaz).

0.10.3'te ne değişti — yerleşim yenilemesi (ince bar + saran sekmeler):

- **İnce komut barı:** üstteki iki satırlı düğme çubuğu yerini tek ince bara bıraktı — solda **tıklanabilir bağlantı rozeti** (tıkla → bağlantı ekranı) ve **arama/komut kutusu** (tıkla ya da `Ctrl+P` → "her yere atla" paleti: nesne, sekme, komut tek kutudan); sağda tüm araçlar **yalnız-ikon** (adları fareyle üzerine gelince görünür).
- **Saran sekmeler:** sekmeler kendi satırında; çok sekme açılınca SSMS/Visual Studio gibi **alt satıra sarar** — hiçbir sekme gizlenmez. **+** düğmesi son sekmenin yanında.
- Kazanç: araç çubuğu inceldi, editöre daha çok dikey alan; arama her an gözünün önünde.

0.10.2'de ne değişti — "Petrol" tasarım yenilemesi: Uygulama baştan aşağı yeni görsel kimliğe geçti (3 öneri arasından seçilen **Tasarım 1 · Petrol**):

- **Petrol mavisi vurgu + soğuk nötr griler** — açık ve koyu tema aynı özenle; turkuaz/antrasit kimlik emekli oldu.
- **Komut çubuğu:** üstteki koyu bant kalktı; ikonlu iki satırlı sessiz düğmeler, ince grup ayraçları, solda yeşil noktalı **bağlantı rozeti**. Harita düğmesinin adı artık **"Veritabanı haritası"**.
- **Sekmelerde durum noktası:** yeşil = tamam, kırmızı = hata, sarı = iptal; sorgu çalışırken nokta **nabız gibi yanıp söner**.
- **Kapsül anahtarlar:** NOLOCK · Güvenli yazma · Oto yenile artık modern aç/kapa anahtarı (renkli etiket uyarıları korundu).
- **Giriş kutuları:** yumuşak köşe + odaklanınca petrol çerçeve; listelerde seçim sol vurgu çizgisiyle.
- Sonuç grid'i: soluk başlıklar, satır hover'ı, biraz daha ferah satırlar; editörde yumuşak anahtar-sözcük mavisi.

0.10.1'de ne değişti — kararlılık + test bulguları düzeltmeleri:

- **Zor sorgularda donma/çökme giderildi:** grid artık yalnız satır sayısıyla (100.000) değil **bellek boyutuyla da** (~256 MB) sınırlı — geniş satırlarda (nvarchar(max)/ikili/XML/SELECT *) hangisi önce dolarsa okuma durur, "*bellek sınırına ulaşıldı*" bandı çıkar. Tümü için ⬇ **Tümünü dışa aktar** (sınırsız akış) kullanılır.
- 📄 **Rapor ve ⬇ Tümünü dışa aktar düğmeleri onarıldı** (buton taşıma sonrası çalışmıyordu/gri kalıyordu).
- **Hatalı sorgunun altı yanar:** çalıştırma hata verirse hatalı satırın altına editörde kırmızı dalga çizilir; metni düzeltmeye başlayınca söner (MongoDB hariç). **0.16.2'den itibaren beklemek de gerekmez:** SQL Server'da yazmayı bıraktıktan ~1 sn sonra sorgu sunucuda yalnız *parse* edilir — sözdizimi hatası varsa dalga **daha çalıştırmadan** yanar. Bu denetim hiçbir şey çalıştırmaz (DML dahil), yalnız söz dizimini sınar.
- **Biçimlendir** artık SELECT listesindeki kolonları **alt alta** dizer (CONVERT/OVER gibi fonksiyon virgülleri bölünmez).
- **Sorgu geçmişinde sekme adı:** her kayıtta hangi sekmeden (SQLST3 / kaydedilen ad) çalıştığı görünür.
- 📑 **Log analizi yeniden tasarlandı:** artık veritabanındaki **exception/log TABLOSUNU** analiz eder — tablo + mesaj kolonu seçilir (log/exception adlı tablo ve mesaj kolonu otomatik önerilir), en çok tekrarlayan kayıtlar sırayla gelir; parametre farkları elenir, aynı hata tek grupta toplanır. *Ekrandaki not:* bu ekranı log tabloları için kullanın; aksi takdirde verim alınamayabilir.
- 🎨 **Görsel Sorgu'da ▶ Çalıştır** script sekmesi açmaz — **sonuç aynı sekmenin altında** gösterilir.

0.10.0'da ne değişti — "denetim sürümü": Yönetim Paneli'ne yeni **Aktivite / Denetim** sekmesi: veritabanında **"kim ne iş yapmış"**. Bu araçtan çalıştırılan her sorgu — zaman · **kullanıcı** · işlem türü (Okuma/Ekleme/Güncelleme/Silme/Yapı/Yürütme) · durum · süre — süzgeç + CSV dışa aktarma ile listelenir; işlem türü SQL'den çıkarılır (CTE-gizli DML dahil). SQL Server'da ayrıca **canlı aktivite** (şu an kim ne çalıştırıyor). **Dört motorda** da vardır. Ayrıntı: Bölüm 24.

0.9.1'de ne değişti — editör ve tanılama iyileştirmeleri:

- **Editörde Bul & Değiştir** (`Ctrl+H`): bul/değiştir + BÜYÜK-küçük/tam-sözcük/regex + tümünü değiştir.
- **Toplu yorum kısayolları**: `Ctrl+K,C` yorumla · `Ctrl+K,U` kaldır · `Ctrl+/` değiştir.
- **Biçimlendirme artık yorumlu metinde de çalışır** — yorumlar korunur (Bölüm 15).
- 📑 **Log analizi** penceresi: son log kayıtlarında **en çok tekrarlayan hatayı** gösterir ("patlama" tespiti).
- **Edit modunda ▶ Sorgu penceresi**: aynı bağlantıda serbest SQL çalıştırma (Bölüm 15).

- Yeni sekmeler **SQLST1, SQLST2…** adıyla açılır, kaydedince dosya adını alır; sonuç bölgesi butonları tek sırada.

0.9.0'da ne değişti — "harita sürümü": Yeni 🗺 **Veritabanı Haritası**: şemayı modern bir ER-diyagramı olarak resmeder — tablolar kart, yabancı anahtarlar yön oklu yumuşak bağ. Kartın üstüne gelince ilişkiler aydınlanır; arama ile odaklama; başlıktan sürükleme + otomatik yerleştirme (**konum DB başına kalıcı**); Görsel Sorgu'ya köprü; **PNG/SVG dışa aktarma**. MongoDB'de koleksiyonlar bağsız kart. Ayrıntı: Bölüm 23.

0.8.2'de ne değişti — borç kapanışı: Bekleyen dört kod borcu kapatıldı:

- **Index farkı dört motorda** — şema karşılaştırmada index farkı artık yalnız SQL Server'da değil, PostgreSQL/MySQL/Oracle'da da çıkar.
- **Tablo sihirbazına UNIQUE / CHECK / FK kısıtları** ve **ALTER (var olan tabloya sütun ekleme) kipi** eklendi (bkz. Bölüm 22).
- **Mongo find yardımcısı artık iki yönlü** — mevcut bir `find` belgesinden panel ön-doldurulur.

Dürüst not: Bu borçların SQL'i (özellikle MySQL/Oracle index sorguları, ALTER ve FK/kısıt DDL'i) gerçek sunucuda değil, birim düzeyinde doğrulandı — MySQL/Oracle canlı doğrulama borcu sürüyor.

0.8.1'de ne değişti — editör ve ince ayarlar: Bu yama sorgu editörünü ve otomatik tamamlamayı iyileştirir:

- **Zengin sözdizimi renklendirmesi** — geniş T-SQL sözcük kümesi mavi (`PARTITION BY` , `OVER` , `WITH (NOLOCK)` , `AS` , pencere fonksiyonları, veri tipleri…); **sayılar ayrı renkte** (turuncu). Açık/koyu temaya uyarlanır.
- **Şema adları otomatik tamamlamada** — artık ▤ **şema** önerileri de gelir; `şema.` yazınca o şemanın nesneleri listelenir.
- **Kısa kolon adları** — otomatik tamamlama **2 karakterde** açılır (önce 3'tü); böylece `Id` / `No` gibi kolonlar da yakalanır.
- **Etiketli arama paneli** (`Ctrl+F`) — butonlar artık metinli: ↑ Önceki · ↓ Sonraki · ✕ Kapat.
- **Rapor ve Tümünü dışa aktar** düğmeleri sonuç bölgesine (grid export butonlarının yanına) alındı.

0.8.0'da ne değişti — "oluşturma sürümü": Bu sürüm iki oluşturma yardımcısı ekler:

- **Tablo oluşturma sihirbazı** — nesne gezgininde **"Tablolar" grubuna sağ tık** → ➕ **Tablo oluştur…**. Şema + tablo adı + kolonları (ad · tip · NULL · PK · IDENTITY · varsayılan) girin; motorun sözdizimiyle `CREATE TABLE` üretilir ve yeni sekmede açılır (dört SQL motoru; Bölüm 22).
- **Mongo find yardımcısı** 🍃 — MongoDB'de üst şeritteki 🍃 Find yardımcısı: Koleksiyon · Filter · Sort · Project · Limit · Skip alanlarından çalıştırılabilir `find` JSON'u üretir (elle JSON

yazmadan; Bölüm 22).

Ortak ilke: Her ikisi de **hiçbir şeyi kendiliğinden çalıştırmaz** — üretilen script/JSON yeni sekmede açılır, siz görüp `F5` ile çalıştırırsınız. Tablo sihirbazı MongoDB'de ve salt-okunur profillerde uygun değildir. MySQL/Oracle CREATE sözdizimi henüz gerçek sunucuda denenmedi.

0.7.0'da ne değişti — "karşılaştırma sürümü": Bu sürüm iki veritabanını karşılaştırmayı ekliyor — **Karşılaştırma sekmesi** 🔀 (üst şeritteki `⇄ Karşılaştır`):

- **Şema karşılaştırma** — soldaki aktif profil ile sağa girdiğiniz **başka bir sunucu** (aynı motor) arasındaki tablo/kolon/PK/FK/index farkları. Test↔prod şema kayması tek gridte.
- **Veri karşılaştırma** — bir tabloyu seçip (isteğe bağlı WHERE) iki veritabanı arasında **satır bazında** kıyaslar: yalnız solda / yalnız sağda / farklı. Sunucuda hesaplanan satır parmak izi sayesinde büyük tabloyu da kaldırır; **farklı satırlarda hangi kolon değişti** (kaynak → hedef) gösterilir; fark listesi **CSV**'ye aktarılır (Bölüm 21).

Dürüst sınırlar: Sağ taraf **aynı motorla** sınırlıdır (motorlar arası kıyas ayrı bir iştir). Veri karşılaştırma anahtar olarak **PK** kullanır (PK'sız tablo satır bazında kıyaslanamaz). Satır parmak izi MSSQL'de `BINARY_CHECKSUM` 'dır (çok nadir çakışma olabilir; XML/CLR tipli kolonlu tabloda sunucu hata verir). MySQL/Oracle veri kıyası henüz gerçek sunucuda denenmedi.

0.6.0'da ne değişti — "görsel sorgu sürümü": Bu sürüm SQL yazmadan sorgu kurmanın bir yolunu ekliyor — **Görsel Sorgu Tasarımcısı** 🎨:

- **Tabloları sürükle-bırak** — nesne gezgininden tabloları tuvale bırakın, kutu olur.
- **Kutuları bağla = JOIN** — başlıktaki ○'dan çekip bir kutuyu diğerine bağlayın; aralarında yabancı anahtar varsa **ON kolonları otomatik önerilir**. Join tipi (INNER/LEFT/RIGHT/FULL) çizgiye çift tıklayarak seçilir.
- **Çift tık → WHERE** — kutuya çift tıklayıp satır satır *Kolon · Operatör · Değer · VE/VEYA* koşulları girin; kolon checkbox'larıyla **SELECT listesini** daraltın.
- **İki yön:** `🎨 Görsel Sorgu` tuvalden SELECT üretir; `🎨 → Görsele çevir` ise **var olan bir SELECT'i tuvale döker** (kutu + bağ + şart olarak) — görsel ile metin arasında gidip gelirsiniz (Bölüm 18).

Dürüst sınırlar: Görsel Sorgu **yalnız SQL ailesindedir** (MongoDB'de gizli), Script→Görsel çevirme ise T-SQL ayrıştırıcısı kullandığından **yalnız SQL Server'da** görünür. Tuval hiçbir şey otomatik çalıştırmaz — yalnız script üretir; alt sorgu/fonksiyon gibi çevrilemeyen parçalar bir bilgi satırıyla atlanır.

0.5.0'da ne değişti — "teşhis ve verimlilik sürümü": 0.4.0 mevcut yetenekleri tüm motorlara taşımıştı; bu sürüm **yeni yetenekler ekliyor** ve bunları baştan beş motorda birden sunuyor:

- **Execution plan** — beş motorda da. Sorgunuzu sunucunun nasıl çalıştırdığını, hangi adımın pahalı olduğunu ve tahmin/gerçek sapmasını gösterir (Bölüm 7).
- **Sunucuda metin arama** 🔍 — "bu kolonu hangi prosedürler kullanıyor?" sorusunun cevabı. Seçili veritabanının view/SP/fonksiyon tanımlarında arar (Bölüm 7).

- **Yönetim Paneli'ne Bakım sekmesi** — tempdb'nin neyle dolduğu, bloklama *zinciri* (uykudaki kök bloklayan dahil) ve istatistik tazeliği. PostgreSQL, MySQL ve Oracle panellerine de ikişer yeni bölüm (Bölüm 14).
- **Kod parçaları** ⚡ — kısayolla genişleyen kalıplar; her kalıp yalnız kendi motorunda önerilir (Bölüm 7).
- **Otomatik yenileme** 🔁 — sorguyu aralıklı tekrarlar ve neyin değiştiğini yazar. Yazma sorgularında bilinçli olarak açılmaz (Bölüm 7).
- **Grid'de toplu düzenleme** — NULL atama (`Ctrl+0`), çoklu satır yapıştırma (`Ctrl+V`), kolonu toplu güncelleme. Üçü de yine Show Script önizlemesinden ve PK'lı WHERE'den geçer (Bölüm 13).

Dürüst sınırlar: MySQL/MariaDB ve Oracle'ın yeni panel bölümleri henüz gerçek sunucuda denenmedi; Oracle'da metin araması view'ları kapsamıyor. İkisi de yol haritasında kayıtlı.

Projenin çıkış noktası, eski bir iç aracın (MersisMSSQL) üç kronik hatasından ders almaktır:

- **Parola güvenliği:** Parolalar asla kod içinde ya da düz metin olarak durmaz — Windows DPAPI ile, yalnız sizin Windows hesabınızın çözebileceği biçimde şifrelenir.
- **Hata dürüstlüğü:** Hiçbir hata sessizce yutulmaz; veritabanı hataları numarası ve satırıyla Mesajlar bölümünde görünür.
- **Donmayan arayüz:** Tüm sorgular arka planda çalışır; uzun sorgu sırasında pencere kilitlenmez, `Esc` ile anında iptal edilir.

Tasarım ilkesi — koruma rayları: Araç, yanlışlıkla veri bozmayı zorlaştırır: filtresiz UPDATE/DELETE onay ister, seçili veritabanı dışına yazan sorgu onay ister, Güvenli Yazma Modu her yazmayı geri alınabilir kılar, salt-okunur profillerde yazma hiç gönderilmez. **Bu rayların tümü beş motorda birden çalışır** — MongoDB'de boş filtreli bir `delete` de aynı şekilde uyarı çıkarır.

Tasarım ilkesi — yarım özellik yok: Bir özellik bir motorda düzgün çalışamıyorsa o motorda **arayüzden gizlenir**; sessizce yanlış çalışan ya da anlamsız hata veren düğme bırakılmaz. Aşağıdaki bölümlerde "yalnız SQL Server'da" gibi notlar bunun içindir.

2. Kurulum ve İlk Açılış

- `SQLST-Kurulum-0.10.0.exe` dosyasını çalıştırın — **yönetici izni gerekmez** (kullanıcı-başına kurulum, hedef: `%LOCALAPPDATA%\Programs\SQLST`).
- Uygulama tek dosyadır; .NET kurulumu gerektirmez (kendi çalışma zamanını içerir). **Veritabanı sürücüleri de içindedir** — PostgreSQL, MySQL, Oracle ya da MongoDB için ayrıca istemci/sürücü kurmanız gerekmez.
- İlk açılış, tek dosyanın ayıklanması nedeniyle ~3 sn sürer; sonraki açılışlar ~1,5 sn'dir.

- Tüm veriler yereldir: profiller, sorgu geçmişi, nesne tarihçesi ve ayarlar `%APPDATA%\SQLST` klasöründe tutulur; hiçbir veri dışarı gönderilmez. **0.5.0 ve öncesinde** bu klasörün adı `%APPDATA%\MiniSSMS` idi; uygulama ilk açılışta eski klasörü yeni adına **otomatik taşır**, böylece kayıtlı profilleriniz, sorgu geçmişiniz ve kod parçalarınız korunur. Taşıma başarısız olursa uygulama bir uyarı gösterir ve eski klasör diskte olduğu gibi kalır.
- **Eski sürümden yükseltme:** Kurulum eskisinin üzerine gelir; mevcut profilleriniz, geçmişiniz ve ayarlarınız korunur. Motor alanı olmayan (0.2.0 ve öncesi) profiller **SQL Server** sayılır.

3. Desteklenen Motorlar

SQLST iki aileden oluşur: **SQL ailesi** (SQL Server, PostgreSQL, MySQL/MariaDB, Oracle — ortak ADO.NET gövdesi + motora özgü "lehçe") ve **MongoDB** (ayrı aile; SQL yok, sorgu dili JSON). Üst katman sorgu metnini asla yorumlamaz — metni yalnız motorun kendisi anlar.

Motor	Sürücü	Notlar
SQL Server MSSQL	Microsoft.Data.SqlClient	Referans motor. Tüm özellikler burada açıktır (Güvenli Yazma, Edit modu, T-SQL script'leri, zengin Yönetim Paneli). 2012 ve üzeri; LocalDB dahil.
PostgreSQL	Npgsql	Şema kavramı SQL Server'a benzer (`şema.tablo`). Tanımlayıcılar çift tırnaklıdır. `USE` yoktur — veritabanı değiştirmek yeniden bağlanmaktır (araç bunu sizin için yapar).
MySQL / MariaDB	MySqlConnector	**Şema = veritabanı**; ağaçta ara katman yoktur. Tanımlayıcılar ters tırnaklıdır (`` `tablo` ``). MariaDB resmî olarak bu sürücüyü önerir.
Oracle	Oracle.ManagedDataAccess.Core	Şema = kullanıcı; adlar BÜYÜK harfe katlanır. Sorgu sonundaki `;` sunucuya gönderilmez (ORA-00933) — PL/SQL blokları korunur. Tek komutta tek ifade çalışır.
MongoDB	MongoDB.Driver	Ayrı aile. Sorgu metni **tek JSON belgesidir** (Bölüm 8). Ağaçta tablo yerine **Koleksiyonlar** vardır; alanlar örneklemeyle çıkarılır.

> **Dürüst sınır — Oracle:** Oracle lehçesi birim testleriyle doğrulanmıştır, ancak geliştirme sırasında erişilebilir bir Oracle sunucusu bulunmadığından **canlı bir sunucuya karşı denenmemiştir**. İlk gerçek bağlantıda ince ayar gerekebilir.

Motora göre özellik haritası

Bir özellik bir motorda yoksa arayüzde de görünmez — aşağıdaki tablo neyin nerede beklendiğini tek bakışta verir.

Özellik	MSSQL	PostgreSQL	MySQL	Oracle	MongoDB
Sorgu çalıştırma, sonuç grid'i, dışa aktarma	✓	✓	✓	✓	✓
Nesne gezgini + arama	✓	✓	✓	✓	✓
İlk 200 satır *(motorun kendi söz dizimiyle)*	✓	✓	✓	✓	✓
Sorgu geçmişi + oturum kurtarma *(profil bazlı)*	✓	✓	✓	✓	✓
Salt-okunur profil koruması	✓	✓	✓	✓	✓
Filtresiz yazma uyarısı	✓	✓	✓	✓	✓
Yönetim Paneli *(her motorda kendi bölümleriyle)*	✓	✓	✓	✓	✓
Nesne tarihçesi (view/SP/fonksiyon)	✓	✓	✓	✓	—
Biçimlendir	✓	—	—	—	✓ *(JSON)*
Güvenli Yazma Modu 🛡	✓	✓	✓ *(DML)*	✓ *(DML)*	—
Kilitsiz okuma (NOLOCK)	✓	—	—	—	—
Edit modu (grid'de veri düzenleme)	✓	✓	✓	✓	—
Execution plan görüntüleme	✓	✓	✓ *(tahmini)*	✓ *(tahmini)*	✓
Kod parçaları ⚡ *(kalıplar motora göre)*	✓	✓	✓	✓	✓
Otomatik yenileme 🔁 *(yalnız okuma sorguları)*	✓	✓	✓	✓	✓
Sunucuda metin arama 🔍 *(nesne tanımları)*	✓	✓	✓	✓	✓ *(view + index tanımları)*
T-SQL script üretimi (ALTER, CREATE TABLE, EXEC…)	✓	—	—	—	—
`Ctrl+Enter` imleçteki ifade	✓	—	—	—	—
Connection string'den doldur	✓	✓	✓	✓	✓

"—" *işareti o motorda düğmenin/menünün* ***görünmediği*** *anlamına gelir; gri ve tıklanamaz bir öğe bırakılmaz.* `Ctrl+Enter` diğer motorlarda seçili metni (seçim yoksa tümünü) çalıştırmaya düşer.

4. Bağlantı Ekranı

Uygulama, bağlantı kurulana kadar bu ekranla açılır. Solda kayıtlı profil kartları, sağda profil formu bulunur. Kartlarda profilin motoru rozet olarak görünür.

Alan / Düğme	İşlevi
Motor	**(0.3.0 ile geldi)** SQL Server · PostgreSQL · MySQL/MariaDB · Oracle · MongoDB. Seçim, formun ve tüm uygulamanın davranışını belirler. Motor **profilin bir özelliğidir** — istediğiniz kadar farklı motorlu profil tanımlayabilirsiniz.
Profil adı	Kartta görünecek serbest ad (ör. "Üretim", "LocalDB", "Mongo-Test").
Sunucu	Sunucu/örnek adı — motora göre: `SRV01\SQL2019`, `localhost:5432`, `localhost:3306`, `srv:1521/ORCLPDB`, `localhost:27017`. **Veritabanı alanı yoktur:** bağlantı sunucuyadır, veritabanı sorgu sekmesinden seçilir.
Kimlik doğrulama	Windows (mevcut hesap — yalnız SQL Server) ya da kullanıcı adı + parola. Parola DPAPI ile şifrelenip saklanır; düzenlemede boş bırakılırsa eski parola korunur.
Salt-okunur	İşaretliyse bu profilde yazma/DDL sınıfı sorgular **hiç gönderilmez** — **beş motorda da**. Engellenenler ekranda parantez içinde yazar: SQL ailesinde `UPDATE · DELETE · INSERT · MERGE · CREATE · ALTER · DROP · TRUNCATE · EXEC`; MongoDB'de `insert · update · delete · findAndModify · drop · dropDatabase · dropIndexes · create · createIndexes · createUser · renameCollection · collMod · bulkWrite` komutları ve `$out` / `$merge` içeren `aggregate`. SQL Server'da ikinci katman olarak bağlantı `ApplicationIntent=ReadOnly` ile açılır. **Denetim temkinlidir:** yalnız ilk sözcüğe bakılmaz — yorum ve metin sabitleri ayıklandıktan sonra sorgunun *herhangi bir yerinde* yazma anahtarı aranır. Böylece CTE'ye gizlenmiş `DELETE` (`WITH … DELETE … RETURNING`) ve blok yorumla başlayan script'ler de yakalanır. Bedeli: nadiren zararsız bir sorgu da reddedilebilir (ör. `SELECT [delete] FROM t`) — o durumda kutuyu kaldırın. Tersi veri kaybı olurdu.
Zaman aşımı (sn)	**Bağlantı**: sunucuya erişim süresi. **Sorgu**: komut süresi; 0 = sınırsız.
📋 Connection string'den doldur	app.config / appsettings'ten kopyalanan bağlantı dizesini **ya da bir URI'yi** yapıştırın; form otomatik dolar. **Beş motorda da çalışır** ve **motoru dizeden tanır** — bkz. aşağıdaki kutu.
Test Et	Kaydetmeden bağlantıyı dener; sonuç yeşil/kırmızı olarak formda görünür.
Kaydet	Profili diske yazar (bağlanmaz).
Bağlan	Doğrular + sınar + kaydeder + ana pencereyi açar. Karta çift tık da aynıdır.
+ Yeni / Sil	Boş form açar / seçili profili (onayla) siler.

Connection string'den doldurma

Elinizde bir bağlantı dizesi varsa alanları tek tek doldurmanıza gerek yok. Tanınan biçimler:

Motor	Örnek
SQL Server	`Server=SRV01;Database=Db;User Id=u;Password=p` · `Integrated Security=true`
PostgreSQL	`Host=pg01;Port=5432;Database=db;Username=u;Password=p` · `postgresql://u:p@pg01:5432/db`
MySQL / MariaDB	`Server=my01;Port=3306;Uid=u;Pwd=p` · `mysql://u:p@my01:3306/db`
Oracle	`srv01:1521/XEPDB1` (Easy Connect) · `Data Source=srv01:1521/XEPDB1;User Id=u;Password=p` · `(DESCRIPTION=…)` (TNS)
MongoDB	`mongodb://u:p@localhost:27017/db` · `mongodb+srv://u:p@cluster0.abcd.mongodb.net/db`

> **Motor dizeden tanınır.** URI şeması (`postgresql://` , `mongodb://` …) ya da motora özgü anahtarlar (`Integrated Security` → SQL Server, `Host=` → PostgreSQL, `AllowPublicKeyRetrieval` → MySQL…) motoru belirler ve seçici otomatik değişir. **Bazı diziler gerçekten ayırt edilemez** — örneğin `Server=…;Uid=…;Pwd=…` hem SQL Server hem MySQL olabilir. Böyle bir durumda formda **seçili olan motor** varsayılır ve size "varsayıldı" notu gösterilir; yanlışsa motoru elle değiştirmeniz yeter. Araç sessizce bir motor seçmez.

> **Parola dizeden alınır ama güvenle saklanır:** URI içindeki `kullanıcı:parola@` kısmı ayrıştırılıp parola kutusuna konur — sunucu alanına **gömülmez**. Böylece parola, diğer tüm parolalar gibi kaydedilirken DPAPI ile şifrelenir; `profiles.json` 'a hiçbir zaman düz metin yazılmaz. Dizedeki veritabanı adı ise profile alınmaz (bilgi notuyla söylenir) — bağlantı sunucuyadır.

> **Her şey profil bazlıdır:** Sorgu geçmişi, açık sekmeler, Güvenli Yazma / geçmiş-kaydet gibi ayarlar ve nesne tarihçesi **profil başına** tutulur. Bağlantıyı değiştirdiğinizde önceki profilin oturumu kendi adına kaydedilir, sekmeler kapatılır ve yeni profilin kendi sekmeleri geri yüklenir — bir motorun sorguları başka bir motorda karşınıza çıkmaz. (Tema tercihi bilinçli olarak geneldir.)

5. Ana Pencere ve Üst Şerit

Ana pencere üç bölgedir: solda **nesne gezgini**, ortada **sorgu sekmeleri**, sağ kenarda **sorgu geçmişi şeridi**. En üstteki antrasit şeritte bağlantı bilgisi (motor rozetiyle) ve genel düğmeler durur:

> **Sol komut rayı — hover davranışı (0.12.0):** ray açılışta **kapalıdır** (yalnız renkli ikon çipleri). Farenizi rayın üstüne getirince yumuşak bir animasyonla **açılır**, çekince **kapanır**. Üstteki ☰ düğmesi rayı **sabitler** (hep açık kalır); tekrar basınca sabitleme kalkar ve hover davranışına dönülür.

Düğme	İşlevi
🛠 Yönetim Paneli	Bağlı olduğunuz motorun teşhis paneli — her motorda farklı bölümlerle (bkz. Bölüm 14).
⇄ Karşılaştır	Sonuç diff'i ve şema snapshot/fark araçları (bkz. Bölüm 16).
🌙 Tema	Tema menüsü (0.19.13): ☀ **Açık Tema** / 🌙 **Koyu Tema** altında 8'er renk varyantı (İndigo · Grafit · Slate Mavi · Petrol · Amber · Kor · **Visual Studio (SSMS)** · **Antrasit + Turkuaz (klasik ızgara)**) + "Açık ⇄ Koyu değiştir"; seçim anında uygulanır, tercih kalıcıdır. İlk 7 varyantta **tasarım aynıdır, yalnız renkler değişir** (VS/SSMS deseni): düz tek tonlu **monokrom sol ray** (çizgi ikonlar), ailenin vurgu renginde **durum çubuğu** ve bağlantı ekranında aynı renkte düz başlık bandı. **Visual Studio (SSMS)** varyantı SSMS 21'in oturduğu VS kabuğunun renkleridir — imza mavi çubuk dahil. **Antrasit** ise **klasik düzendir**: üstte antrasit bağlantı bandı (beyaz bağlantı metni + **Bağlantıyı değiştir** düğmesi), kutulu klasik sekmeler, düz renksiz düğmeler (Çalıştır dahil), kapsül yerine kare onay kutuları, tek renk ikonlar, açık sol menü, açık gri durum çubuğu ve sonuçlarda **ince çizgili tam ızgara** (Segoe UI yazı, dikey + yatay çizgi, alternatif satır yok); koyu hali aynı düzeni koyu renklerle uygular.
⌨ Kısayollar	Tüm klavye kısayollarının kılavuz penceresi (`F1`).
⏏ Bağlantıyı değiştir	Bağlı ekran kapanır, bağlantı ekranı açılır; yeni profile bağlanınca ana pencere yeni bağlantıyla gelir. **X ile vazgeçilirse uygulama kapanır** (0.12.0 — eski bağlantıya sessiz dönüş yok).

6. Nesne Gezgini

- Kök düzeyde sunucudaki veritabanları listelenir; sistem veritabanları (SQL Server'da **master/model/msdb/tempdb**, MongoDB'de **admin/config/local**) "Sistem Veritabanları" klasöründe katlanır.
- Bir veritabanını ilk açışınızda şeması (tablolar, view'lar, SP'ler, fonksiyonlar, kolon/parametre detaylarıyla) tek seferde yüklenip önbelleğe alınır.
- **MongoDB'de** tablo yerine **Koleksiyonlar** grubu vardır. Koleksiyonun alanları, belgelerden `$sample` ile 100 belge örneklenerek çıkarılır: alan adı → tipi; bir alan birden çok tipte görülmüşse `string|int` gibi yazılır, bazı belgelerde hiç yoksa "null olabilir" işaretlenir, `_id` anahtar sayılır. (Compass'ın yaptığı da budur — belge veritabanında sabit şema yoktur.)
- **MySQL/MariaDB'de** şema = veritabanı olduğundan ağaçta ayrı bir şema katmanı görmezsiniz.

- **Arama kutusu** yazdıkça tüm nesne türlerinde süzer (ağ erişimi olmadan, bellekten); kutudaki ✕ aramayı temizler. ⟳ şemayı yeniler. **Kolon araması (YENİ):** arama yalnız nesne adında değil **kolon adlarında da** eşleşir — bir kolon adını yazınca (ör. `musteriId`) o kolonu içeren tablolar listelenir; eşleşen kolon ağaçta **kalın** gösterilir. "Bu kolon hangi tablolarda var?" sorusunun tek kutuda, ağa çıkmadan yanıtı.
- Tablo düğümü açılınca kolonlar (🔑 PK işaretli, tip ve null bilgisiyle) görünür.
- 🔗 **Kolon bağları (YENİ):** bir kolona **sağ tık** → **"Tüm bağlantılarını göster"** — kolonun yabancı anahtar bağları küçük bir pencerede listelenir: **▲ giden** (bu kolon hangi tabloyu işaret ediyor) + **▼ gelen** (bu kolonu hangi tablolar işaret ediyor), kısıt adları ve bileşik anahtar eşleşmeleriyle. Bir satıra **çift tık**, ilişkiyi ortaya çıkaran **JOIN sorgusunu** (ilk 100 satır) yeni sekmede çalıştırır. FK grafı şema önbelleğinden gelir — ağa çıkılmaz; MongoDB'de FK olmadığından SQL motorlarına özgüdür.



dbo.Siparis.Id'nin bağları: ▲ 1 giden · ▼ 3 gelen — çift tık ilişkiyi JOIN sorgusuyla açar.

Sağ tık menüsü

Menü **hem nesne türüne hem de motora göre** süzülür — bir öğe o motorda anlamlı değilse görünmez.

Öğe	Tür	Motor	İşlevi
İlk 200 satır	Tablo, View, Koleksiyon	**Hepsi**	Sorguyu yeni sekmede açar ve **hemen çalıştırır**. Sorgu motorun kendi söz dizimindedir: `SELECT TOP 200` (MSSQL) · `LIMIT 200` (PostgreSQL, MySQL) · `FETCH FIRST 200 ROWS ONLY` (Oracle) · `{"find": …, "limit": 200}` (MongoDB).
🕘 Tarihçe	View, SP, Fonksiyon	SQL ailesi	Nesnenin yerel sürüm tarihçesi ve karşılaştırma penceresi (bkz. Bölüm 15).
Kolonları kopyala	Tablo, View	**Hepsi**	Kolon adlarını virgüllü liste olarak panoya alır.
▪▬ Düzenle (ilk 1.000)	Tablo	SQL ailesi	Düzenlenebilir veri grid'i açar (bkz. Bölüm 13). MongoDB'de yoktur.
SELECT script'i	Tablo, View	Yalnız MSSQL	SELECT iskeletini sekmede açar (çalıştırmaz).
🎨 Görsel sorguya ekle	Tablo	Yalnız MSSQL	Tabloyu **Görsel Sorgu tuvaline** koyup tasarımcıyı açar — ağaçtan sürükleyip bırakmanın menü karşılığı (bkz. Bölüm 18). Art arda eklerseniz kutular üst üste binmez.
Çalıştır (EXEC iskeleti)	SP	Yalnız MSSQL	Parametreleri yorumlu EXEC iskeleti üretir; parametresiz SP doğrudan çalışır.
Değiştir (ALTER)	View, SP, Fonksiyon	Yalnız MSSQL	Tanımı ALTER script'i olarak açar — düzenleyip `F5` ile güncellersiniz. Tanım o an yerel tarihçeye de kaydedilir.
Script: CREATE / DROP ve CREATE	View, SP, Fonksiyon	Yalnız MSSQL	Ham CREATE tanımı ya da `DROP … IF EXISTS` + `GO` + CREATE bloğu.
Script: CREATE TABLE	Tablo	Yalnız MSSQL	Kolonlar + PK + index'leriyle CREATE TABLE script'i üretir.
🔗 Tüm bağlantılarını göster	**Kolon**	SQL ailesi	Kolonun FK bağları (▲ giden + ▼ gelen) küçük pencerede; çift tık ilişkiyi JOIN sorgusuyla açar (yukarıdaki madde).
INSERT örneği	Tablo	SQL ailesi	Yazılabilir kolonlar için tip-farkında çok satırlı INSERT şablonu (identity/computed atlanır).
UPDATE örneği 0.22.0	Tablo	SQL ailesi	INSERT örneğinin eşi: `SET` 'te yazılabilir kolonlar örnek değerlerle, `WHERE` birincil anahtardan. **Tabloda PK yoksa** koşul `WHERE 1 = 0` olarak ve uyarı satırıyla gelir — şablondan yanlışlıkla filtresiz bir UPDATE çıkmaz. Çalıştırmaz, sekmede açar.

> 🔗 **FK kolonları ağaçta işaretli 0.22.0:** Kolon listesinde birincil anahtarlar 🔑, **yabancı anahtarlar** 🔗 ile görünür. Hangi tabloya bağlı olduğunu görmek için kolona sağ tık → **Tüm bağlantılarını göster**.

> **Neden T-SQL script'leri yalnız SQL Server'da?** Bunlar T-SQL üretir; başka bir motorda ya söz dizimi hatası verirlerdi ya da — **ALTER örneğinde olduğu gibi** — sessizce hiçbir şey yapmayan bir script üretirlerdi. Sessiz yanlış yerine görünmemeyi seçtik.

7. Sorgu Sekmeleri ve Editör

Her sekmenin **kendi kalıcı bağlantısı** vardır: SET seçenekleriniz, geçici tablolarınız ve açık transaction'larınız sekme yaşadıkça yaşar (SSMS davranışı). Sekme kapatılırsa açık kalan transaction otomatik geri alınır.

Sekmeleri yönetmek 0.22.0

İşlem	Nasıl
Sekmeyi kapat	Başlığın **sağ kenarındaki** ✕ (artık her sekmede aynı yerde) ya da `Ctrl+W`. Kapatılan **sorgu** sekmesi `Ctrl+Shift+T` ile metniyle birlikte geri gelir (son 10).
📌 Sabitle 0.23.0	Sekme başlığındaki 📌 **raptiyeye tıklayın** — sağ tık gerekmiyor (menüde de duruyor). Raptiye her sekmede soluk durur; sabitleyince **vurgu renginde** yanar ve ✕ gizlenir: sekme **kapatılamaz** — Ctrl+W, ✕ ve toplu kapatmalar onu atlar. Takip ettiğiniz bir sorgu/log sekmesini kazayla kapatmamak için. Tekrar tıklayınca sabitleme kalkar.
Toplu kapatma	Sağ tık → **Diğerlerini kapat** / **Tümünü kapat**. Sabit sekmeler her ikisinde de kalır.
Sekme değiştirme	Sonuç gridinin **kaydırma konumu, seçili satırı ve editör imleci korunur** — bir sekmede veri takip edip diğerinde çalışabilirsiniz.

> **Araçlar da sekme olarak açılır:** 🔁 LINQ ⇄ SQL ve 🪄 SP Sihirbazı artık ayrı pencere değil sekmedir (aynı aracı ikinci kez açınca yeni kopya değil mevcut sekme seçilir). Diğer araç pencereleri de ekranı kaplamak yerine makul boyutta açılır; isterseniz büyütürsünüz.

Sekme araç çubuğu

Öğe	İşlevi
DB: seçici	Sorgunun hedef veritabanı. Profil veritabanına bağlı değildir — her sekme farklı veritabanına bakabilir. (PostgreSQL'de `USE` olmadığından araç arka planda yeniden bağlanır; Oracle'da şema değişimi `CURRENT_SCHEMA` ile yapılır.)
▶ Çalıştır (F5)	Metin seçiliyse yalnız seçimi, değilse tümünü çalıştırır. **Çok ifadeli script'ler** motorun kendi kuralıyla parçalanıp sırayla aynı bağlantıda koşar: SQL Server'da `GO` ayırıcısı (`GO 5` tekrarı desteklenir), **Oracle'da `;` ve tek başına `/`** (bkz. aşağıdaki kutu). PostgreSQL/MySQL'de metin olduğu gibi gönderilir — sürücüleri çok ifadeli metni zaten kabul eder.
■ Durdur (Esc)	Çalışan sorguyu ~1 sn içinde iptal eder.
≡ Biçimlendir	**SQL Server'da** T-SQL'i biçimlendirir (`Ctrl+Shift+F`): anahtar sözcükler BÜYÜK, girinti düzenli ve **her `JOIN … ON …` tek satırda** tutulur (koşulları alt alta dağıtılmaz). **Yorum içeren metinde de çalışır** — yorumlar olduğu gibi korunur (bu durumda daha hafif bir biçimlendirici devreye girer, sözcüklerden satır böler). **MongoDB'de** JSON sorgusunu girintiler. Diğer motorlarda düğme görünmez.
🔍 Metin ara *(sol rayda)*	0.12.0'da editör araç çubuğundan **sol raya taşındı.** Seçili veritabanının nesne **tanımlarında** metin arar; sonuçlar ayrı sekmede açılır (bkz. aşağıdaki bölüm). SQL ailesinde view/SP/fonksiyon gövdeleri, **MongoDB'de view ve index tanımları**. Tablo verisi / belge verisi taranmaz.
🛡 Güvenli yazma	Güvenli Yazma Modu anahtarı (bkz. Bölüm 11). **SQL ailesinde görünür**, MongoDB'de yoktur. MySQL/Oracle'da DDL ifadelerinde devreye girmez (orada geri alınamaz). Tercih profil başına kalıcıdır.
🔓 Kilitsiz okuma (NOLOCK)	Açıkken sorgu READ UNCOMMITTED gibi davranır: kilit almaz ve beklemez, ama commit edilmemiş veri okuyabilir. **Yalnız SQL Server'da görünür; 0.16.1'den itibaren uygulama açılışında SEÇİLİ gelir** (kullanıcı tercihi 2026-07-26) — istemezseniz kapatın.

> 📂 **Dosya Aç (YENİ):** sol raydaki 📂 Dosya Aç (ya da `Ctrl+O`) bir **.sql dosyasını kendi adıyla** yeni sekmede açar — çoklu seçim desteklenir, Explorer'dan **pencereye sürükleyip bırakmak** da aynı kapıdan geçer. Kodlama otomatik çözülür: BOM varsa o kazanır; yoksa sıkı UTF-8 denenir, geçerli değilse **Windows-1254 (Türkçe ANSI)** ile açılır ve durum çubuğunda uyarılır — Türkçe karakterler hiçbir yolda sessizce bozulmaz. `Ctrl+S` ile geri kaydedersiniz (sekme zaten dosya adını taşır); 10 MB üstü dosyada onay sorulur.
>
> **Gezgin'den açma:** kurulumdan sonra .sql dosyasına sağ tık → **Birlikte aç** → **SQLST** (isterseniz "Her zaman bu uygulamayı kullan" deyin — SQLST mevcut SSMS ilişkilendirmenizi kendiliğinden değiştirmez). Dosya, bağlantı ekranından sunucu seçildikten sonra kendi adıyla sekmede açılır. SQLST zaten açıkken çift tık yeni bir SQLST penceresi başlatır (bilinçli sınır) — açık pencereye dosyayı sürükleyip bırakmak daha kestirmedir.

Oracle'da çok ifadeli script (SQL*Plus kuralı):

- `;` sıradan bir ifadeyi bitirir ve **sunucuya gönderilmez** (Oracle düz SQL'de sondaki noktalı virgülü kabul etmez — ORA-00933).
- **PL/SQL bloğunun içindeki `;` ayırıcı değildir.** `BEGIN…END;`, `DECLARE…` ya da `CREATE OR REPLACE PROCEDURE/FUNCTION/PACKAGE/TRIGGER/TYPE` ile başlayan blok tek parça gönderilir; blok yalnız **tek başına `/` olan satırla** biter (o satır da sunucuya gitmez).
- Dize (`'…'`, `q'[…]'`) ve yorum (`--`, `/* */`) içindeki `;` ve `/` ayırıcı sayılmaz.
- `CREATE TABLE` gibi düz DDL'ler PL/SQL sayılmaz — `;` ile biter.

Hata satır numaraları korunur: sunucunun bildirdiği satır, editördeki gerçek satıra eşlenir.

Execution plan beş motorda da

Sorgunuzu sunucunun nasıl çalıştırdığını gösterir. Düğme 0.12.0'da **sol raya taşındı** ve **aktif sorgu sekmesini** hedefler; **tek düğme** vardır, adı bağlandığınız motora göre değişir:

Düğme	Hangi motorlarda	Ne verir
Execution plan	SQL Server · PostgreSQL · MongoDB	Sorgu **çalıştırılır** ve plana **ölçülmüş** satır sayıları eklenir — tahminle karşılaştırabilirsiniz.
Tahmini execution plan	MySQL/MariaDB · Oracle	Bu motorlar ölçümlü plan vermediğinden sorgu **çalıştırılmaz**, yalnız derlenir. Sayılar optimizer **tahminidir** — düğme adı bunu saklamaz.

Plan **ayrı bir sekmede** açılır (Yönetim Paneli'nden bağımsızdır; orası sunucu genelini inceler, bu sekme yalnız sizin sorgunuzu). Metin seçiliyse yalnız seçimin planı alınır.

Ölçümlü plan yalnız OKUMA sorgularında alınır — yazma/DDL asla çalıştırılmaz. Ölçülmüş satır sayıları elde etmenin tek yolu sorguyu gerçekten çalıştırmaktır; bu da `DELETE`, `ALTER` ya da `CREATE PROCEDURE` gibi bir ifadede veriyi değiştirmek demektir. Bu yüzden araç, ifade yazma/DDL ise ölçümlü planı hiç denemez: ifadeyi yalnız **derleyip** tahminî planı getirir, veri değişmez ve plan yine tam olarak gelir. Plan sekmesi bunu açıkça yazar.

Bu davranış 2026-07-20'de düzeltildi. Öncesinde araç "gerçek plan bunu ÇALIŞTIRIR, yine de çalıştırılsın mı?" diye soruyor ve **Evet** denirse ifadeyi gerçekten çalıştırıyordu. İki bakımdan yanlıştı: plan düğmesine basan kimse veri değiştirmeyi kastetmez, üstelik uyarı kullanıcıyı "'*Tahmini plan*' *kullanın*" diye **artık var olmayan** bir düğmeye yönlendiriyordu (motor başına tek plan düğmesine geçilmişti). Doğru cevabı araç zaten bildiği için artık soru sorulmuyor.

Salt-okunur profillerde yazma sorgusunun planı yine **hiç** alınmaz. *MongoDB'de* ise plan alma zaten yazmayı uygulamaz (aşağıya bkz.).

- **Operatör ağacı:** her satırda operatörün **kendi maliyet payı** (%), üstünde çalıştığı tablo/index, tahmini ve (gerçek planda) gerçek satır sayısı. Yüzdeler **sıralama içindir** — "şu kadar sürer" demek değildir.
- ⚠ **Tahmin/gerçek sapması:** optimizer 10 satır beklerken 5.000 geldiyse ayrı bir bantta uyarılırsınız. **Planın en işe yarar sinyali budur** — genelde istatistiklerin eskidiği anlamına gelir. Küçük sayılarda (1 → 20) uyarı çıkmaz, çünkü orada oran yanıltıcıdır.
- **Uyarılar** tek listede toplanır: tempdb'ye taşma, index kullanımını engelleyen tip dönüşümü, join koşulu olmayan birleştirme…
- **Eksik index önerisi:** plan bir index öneriyorsa gösterilir ve Script'i sekmede aç ile `CREATE INDEX` script'i yeni sorgu sekmesinde açılır — **asla kendiliğinden çalıştırılmaz**. Öneri bir vaat değildir; her index yazma maliyeti ekler.
- **Çok ifadeli script:** SQL Server'da batch'teki her ifadenin planı ayrı ayrı gelir ve üstteki seçiciden aralarında geçebilirsiniz. **PostgreSQL, MySQL/MariaDB ve Oracle'da plan tek bir ifade için alınır** — birden çok ifade seçiliyse araç bunu söyler ve tek sorgu seçmenizi ister (bu motorların `EXPLAIN`'i tek ifade sarar).

PostgreSQL'de farklar

Aynı düğme PostgreSQL'de de çalışır (arkada `EXPLAIN` ya da okuma sorgularında `EXPLAIN ANALYZE`), ama motorun kendisinden gelen birkaç fark vardır:

- **Maliyet birimi farklıdır.** PostgreSQL maliyeti kabaca "disk sayfası okuma" cinsindendir; SQL Server'ın sayısıyla **karşılaştırılamaz**. İkisinde de yüzdeler yalnız *bu plan içinde* hangi adımın pahalı olduğunu söyler.
- **Eksik index önerisi yoktur.** PostgreSQL index önermez; o bölüm PG planlarında boş kalır. (Yönetim Paneli'ndeki PostgreSQL bölümleri kullanılmayan index'leri ayrıca gösterir.)
- **Uyarılar plandan türetilir:** sıralamanın diske taşması, hash'in belleğe sığmaması, index-only scan'in tabloya inmesi (VACUUM sinyali), planlanan paralelliğin sağlanamaması, filtrenin çok satır elemesi (index adayı).
- **Tek ifade:** PostgreSQL'de plan tek bir sorgu için alınır. Birden çok ifade seçiliyse araç bunu söyler.
- Bazı planlarda bir adım alt adımı sonuna kadar okumaz (ör. `LIMIT`). Böyle durumlarda o adımın maliyet payı hesaplanamaz ve ağaçta **açıkça belirtilir** — sessizce "%0" gösterilmez.

> **PostgreSQL'de `EXPLAIN ANALYZE` yazma sorgusunu gerçekten çalıştırır** — bir `DELETE`'i ANALYZE ile açıklarsanız satırlar silinir. Bu yüzden araç yazma/DDL ifadelerinde `ANALYZE`'ı **hiç kullanmaz**; sade `EXPLAIN` ile yalnız planı alır. Salt-okunur profillerde yazma sorgusunun planı yine hiç alınmaz. *Bu, aracın en erken canlı doğrulanan bulgularından biriydi (PG 16.4): `EXPLAIN (ANALYZE) DELETE …` tablodaki 1000 satırı 900'e düşürmüştü.*

MySQL / MariaDB'de farklar

- **Düğme burada "Tahmini execution plan" adıyla görünür** ve sorgu çalıştırılmaz. Sebebi: MySQL ile MariaDB ölçümlü planda ayrışır — MariaDB `ANALYZE FORMAT=JSON` ile JSON verir, MySQL 8 ise `EXPLAIN ANALYZE` ile JSON değil metin tablosu döndürür. İkisini birden güvenilir biçimde okuyamadığımız için ölçümlü plan açılmadı; gördüğünüz sayılar optimizer tahminidir.
- **Erişim türü** her adımda okunur biçimde yazılır: "Tam tablo taraması [ALL]", "Index araması [ref]", "Aralık taraması [range]"...
- **En işe yarar uyarı:** "Kullanılabilir index var ama seçilmemiş" — sorgunun index'i neden kullanmadığını araştırmanız gerektiğini söyler. Ayrıca filesort, geçici tablo ve "okunan satırların çok azı koşulu geçiyor" (index adayı) uyarıları vardır.
- **MariaDB'de maliyet yüzdeleri boş kalabilir:** bu sunucu planında maliyet bilgisi vermez. Böyle bir durumda araç yüzde **uydurmaz**; durumu açıkça yazar ve sıralama için satır sayıları ile erişim türüne bakmanızı söyler.
- Eksik index önerisi yoktur (bu motor öneri üretmez).

Oracle'da farklar

- **Düğme burada "Tahmini execution plan" adıyla görünür** ve sorgu çalıştırılmaz: ölçülen satır sayıları için Oracle'ın istatistik görünümlerine erişmek gerekir ve bu **ek yetki** ister. Araç yalnız kendi şemanızda çalışan, yetki istemeyen sorgular kullanır — bu yüzden gördüğünüz sayılar optimizer tahminidir.
- **Maliyet Oracle'ın kendi birimidir** ve diğer motorların sayılarıyla karşılaştırılamaz. Yüzdeler yalnız bu plan içinde hangi adımın pahalı olduğunu söyler.
- Adım adları Oracle'ın kendi terimleriyle gösterilir: `TABLE ACCESS (FULL)`, `INDEX (RANGE SCAN)`, `HASH JOIN`...
- **Uyarılar:** tam tablo taraması · tam index taraması · **kartezyen birleştirme** (genelde eksik join koşulu demektir) · sıralama adımı.
- Eksik index önerisi yoktur (Oracle'ın `EXPLAIN PLAN`'i öneri üretmez).

MongoDB'de farklar

- **Maliyet yüzdesi yoktur** — ve bu bir eksiklik değil, motorun gerçeği: MongoDB bir maliyet modeli sunmaz. Araç yüzde **uydurmaz**. Bunun yerine bakmanız gereken sayı **"incelenen belge"**dir: 5.000 belge incelenip 2 belge dönüyorsa index adayınız var demektir.
- **Satır TAHMİNİ de yoktur**, bu yüzden bu motorda "tahmin" sayısı ve tahmin/gerçek sapma uyarısı hiç gösterilmez. MongoDB'nin kardinalite tahmini ve istatistiği olmadığı için karşılaştıracak bir beklenti yoktur — *sıfır* yazıp sapma uyarısı üretmek uydurma bir sinyal olurdu.
- **En önemli uyarı** `COLLSCAN`: sorgu hiçbir index kullanmıyor. Index kullanılıyorsa aşama `IXSCAN` olur ve index adı gösterilir.
- **Bellekte sıralama** uyarısı: `SORT` aşaması, sıralamanın index'ten karşılanamadığını söyler; büyük sonuçlarda bellek sınırına takılıp sorgu hata verebilir.

- **Plan burada güvenlidir.** PostgreSQL'in aksine MongoDB'de `explain` bir yazma işlemini **uygulamaz** — bir `delete`'in planını alsanız bile belgeler silinmez. Bu yüzden bu motorda ek onay sorulmaz.
- Eksik index önerisi yoktur (MongoDB öneri üretmez).

Sunucuda metin arama SQL ailesinde

Araç çubuğundaki 🔍 Metin ara düğmesi, **seçili veritabanındaki** view, stored procedure ve fonksiyon **tanımlarında** metin arar — "bu kolonu hangi prosedürler kullanıyor?" sorusunun cevabıdır. Sonuçlar ayrı bir sekmede açılır.

- Solda eşleşen nesneler (**en çok eşleşen üstte**), sağda o nesnedeki eşleşen satırlar ve satırın içeriği listelenir.
- Bir satıra **çift tıklayınca** nesnenin tanımı yeni sorgu sekmesinde **o satırda** açılır. Tanım arama sırasında zaten alındığından sunucuya ikinci kez gidilmez.
- Arama **sunucuda** süzülür; ağdan yalnız eşleşen tanımlar geçer. Aradığınız metindeki `%`, `_` ve tırnak gibi karakterler joker sayılmaz, birebir aranır.
- **BÜYÜK/küçük duyarlı** kutusu kapalıyken sunucunun kendi karşılaştırma kuralı (collation) geçerlidir; açıkken sonuçlar ayrıca daraltılır.
- Tanımı **şifreli** nesneler (ör. `WITH ENCRYPTION` ile oluşturulmuş SP'ler) okunamadığından listede çıkmaz.
- Bir aramada **en çok 200 nesne** döner. Tavana takıldıysanız özet satırında **"yalnız ilk 200 nesne gösteriliyor"** uyarısı çıkar — liste sessizce kırpılmaz. Böyle bir durumda aramayı daraltın (daha uzun ya da daha özgül bir metin). Tavan, çok yaygın bir metin (ör. `SELECT`) arandığında veritabanındaki tüm tanımların ağdan geçmesini önler.

> **Tablo verisi taranmaz — bu bilinçli bir sınırdır.** Veri içinde arama `LIKE '%metin%'` gerektirir; baştaki joker yüzünden hiçbir index kullanılamaz ve her tablonun her metin kolonu baştan sona okunurdu. Milyonlarca kayıtta bu, aracı da sunucuyu da kilitlerdi. Bu yüzden arama yalnız nesne tanımlarını kapsar.

> **MongoDB'de ne aranır?** Belge veritabanında "stored procedure gövdesi" yoktur; bunun yerine **gerçekten saklanan iki tanım** taranır:
>
> - **View tanımları** — `viewOn` + `pipeline`. Bir view'ın tanımı tam olarak budur (SQL'deki `CREATE VIEW` gövdesinin karşılığı).
> - **Index tanımları** — anahtar belgesi ve seçenekleri. "*Bu alana hangi index'ler dokunuyor?*" sorusunun cevabıdır.
>
> **Koleksiyon adları bilinçli olarak taranmaz:** ad araması tanım araması değildir ve özelliğin anlamını motora göre kaydırırdı — SQL tarafında da tablo adları aranmıyor. Koleksiyon adı aramak için nesne gezgininin tepesindeki süzgeci kullanın. Sekmedeki kapsam satırı bu motorda neyin arandığını ayrıca yazar.

> **Oracle'da kalan sınır (henüz gerçek sunucuda denenmedi).** View tanımları artık aranıyor, ancak Oracle view metnini SQL'de doğrudan aranamayan bir kolonda (LONG) tuttuğu için aramanın kullandığı karşılık **tanımın ilk 4000 karakteriyle** sınırlıdır — daha uzun view'larda sonrası taranmaz. Bu yol **Oracle 12.2 ve üzeri** ister. Prosedür/fonksiyon/paket tanımlarında böyle bir sınır yoktur. **Bu motorun arama davranışı gerçek bir Oracle sunucusunda henüz denenmedi** — yol haritasındaki canlı doğrulama borcuyla birlikte kapatılacaktır.

Kod parçaları (snippet) yeni

Sık yazdığınız kalıpları kısayola bağlarsınız: editörde kısayolu yazıp tamamlama listesinden (`Ctrl+Space`) seçince gövde girer. Yönetim ekranı üstteki ⚡ Kod parçaları düğmesindedir.

- Gövdedeki `$0` **imlecin nereye gideceğini** belirler — genişletmeden sonra imleç oraya taşınır, işaret silinir. İşaret yoksa imleç gövdenin sonunda kalır.
- Tamamlama listesinde kod parçaları ⚡ **ile ve en üstte** görünür; açıklama alanında gövdenin tamamını önizlersiniz.
- Kutudan birkaç örnek çıkar (`sel100`, `cte`, `sayac`, Mongo'da `find` / `agg`). Hepsi düzenlenebilir ve silinebilir.
- **Yönetim ekranı bağlı olduğunuz motorun kalıplarını listeler** (+ "(her motorda)" işaretli olanları) — yani editörde önerilenlerle aynı küme. Başka motorların kalıpları *gizlidir, silinmemiştir*; o motora bağlandığınızda görünürler. Pencerenin üstünde neyin gösterildiği yazar.
- **Kayıt da bağlı olduğunuz motorla sınırlıdır:** Motor kutusunda yalnız o motor ve "(her motorda)" çıkar. Örneğin SQL Server'a bağlıyken MongoDB kalıbı kaydedemezsiniz — kaydedebilseydiniz kalıp kaydedildiği anda listeden kaybolur, siz de silindi sanırdınız. Başka bir motora kalıp eklemek için o motora bağlanın. *(Bağlantı yokken yönetim ekranı tüm motorları sunar.)*

> **Kalıplar nerede saklanıyor?** Bu bilgisayardaki yerel veritabanında (`%APPDATA%\SQLST\sqlst.db`). **Kalıplar bağlantı profilinden bağımsızdır** — hangi sunucuya bağlanırsanız bağlanın aynı kalıplar (motor süzmesiyle) gelir. Ancak **kullanıcılar/bilgisayarlar arasında paylaşılmaz:** kaydettiğiniz bir kalıbı ekip arkadaşınız göremez. Ortak bir kalıp havuzu bugün yoktur.

> **Motor alanı önemlidir.** Bir kalıp yalnız seçtiğiniz motorda önerilir — bu yüzden `sel100`, SQL Server'da `SELECT TOP 100`, PostgreSQL ve MySQL'de `LIMIT 100`, Oracle'da `FETCH FIRST 100 ROWS ONLY` olarak ayrı ayrı tanımlıdır. **"(her motorda)" seçeneğini yalnız gerçekten motordan bağımsız metinler için kullanın:** bir T-SQL kalıbını her motorda işaretlerseniz MongoDB sekmesinde de önerilir ve seçtiğinizde çalışmayan bir metin girer.

Aynı nedenle IntelliSense'in SQL anahtar sözcükleri (`SELECT`, `TOP`, `GETDATE` …) artık MongoDB sekmesinde önerilmiyor — orada karşılıkları yok.

Otomatik yenileme yeni

Sekme araç çubuğundaki 🔁 **Oto yenile** kutusu, sorguyu belirtilen saniye aralığıyla yeniden çalıştırır ve **her koşuda neyin değiştiğini** sonuç alanının üstünde yazar (*"+3 satır · 1 satır değişti"*). Canlı bir sayacı ya da kuyruğu izlemek için uygundur.

> **Yazma sorgularında açılmaz — bu bir kolaylık değil, güvenlik kuralıdır.** Bir `UPDATE` ya da `DELETE` 'in 30 saniyede bir kendiliğinden tekrar çalışması veri kaybı demektir. Kutuyu işaretlerseniz geri döner ve nedeni yazılır. Denetim temkinlidir: CTE içine gizlenmiş DML de yazma sayılır.

- En kısa aralık **2 saniyedir**. Sorgu aralıktan uzun sürerse yeni tetikleme **atlanır** — kuyruk birikmez.
- Açık bir işlem (Güvenli Yazma bandı) varken yenileme yapılmaz; kararınızı beklerken sorgu tekrarlanmaz.
- Sekmeyi kapatınca yenileme durur.
- **Dürüst sınır:** satırlar **sıraya göre** karşılaştırılır. Sorgunuzda `ORDER BY` yoksa sunucu satır sırasını garanti etmez ve sıra değişimi "değişmiş" görünebilir. Kararlı bir fark için sorguya `ORDER BY` ekleyin.

Editör yetenekleri

- **IntelliSense: 2 karakter yazınca kendiliğinden açılır** (ilk harfte açılmaz; `Id` / `No` gibi kısa kolon adları da böylece yakalanır); ayrıca `Ctrl+Space` her yerde ve nokta sonrası anında. **Şema adları da önerilir** (▤ şema): `şema.` yazınca o şemanın nesneleri gelir — şema-nitelikli yazımı kolaylaştırır. `takmaAd.` ya da `Tablo.` kolonları önerir. **Takma adların kendisi de öneriye çıkar** (v0.19.3): `… JOIN çalışan k ON` yazınca `k` nokta beklemeden listede gelir (şemadan bağımsız — sorgu metninden okunur). **Bağlama duyarlıdır:** `WHERE / ON / AND / OR / HAVING` içinde yalnız kapsamdaki kolonlar + takma adlar gelir (tüm-tablo ve şema listesi bastırılır, kolonlar boğulmaz); `FROM / JOIN` 'de tablo/şema, `SELECT` 'te ikisi de. **Kapsam = yalnız İMLEÇTEKİ ifadedir** (v0.19.4): kolonlar yalnız o ifadenin `FROM/JOIN` tablolarından gelir — joinli ifadede tüm tablolar, tek `SELECT` 'te yalnız o tablo. İfadeler `;` / tek başına `GO` / **boş satır** ile ayrılır; böylece editördeki diğer sorguların tabloları karışmaz. "spmk" gibi kısaltmalarla bulanık eşleşme yapar. Liste satır satır düzenlidir: her satırda **tür simgesi** (⚡ parça · ◆ kolon · ▤ şema · ▦ nesne · anahtar sözcük), adı ve sağda **tür etiketi** (kolonda tip · PK, nesnede tablo/view…) gösterilir; kutu, en uzun ada göre genişler. Öneri kaynağı her motorda o motorun kendi şemasıdır.
- **Tanım değeri önerisi** 0.22.0**:** Bir yabancı anahtar kolonuyla karşılaştırma yazarken (`WHERE t.DurumId =` , `… IN (`) tanım tablosunun değerleri **anlamıyla** listelenir: `1 Beklemede · 3 Onaylandı …` . Seçtiğinizde koda yalnız **sayı** yazılır. "3 müydü 4 müydü?" diye tanım tablosuna ayrı SELECT atmak gerekmez. Aynı kolon adı farklı tablolara işaret ediyorsa (belirsizse) öneri **hiç açılmaz** — yanlış kod önermektense susar. Yalnız SQL Server'da.

- **Tablo önizleme (Ctrl+hover) 0.22.0:** Editördeki bir tablo adının üstünde `Ctrl`'ye basılı tutup beklerseniz küçük bir kart açılır: **kolonlar** (🔑 / 🔗 işaretli, tipli) ve **ilk 5 satır**. Sekme açmadan "bu tabloda ne vardı?" sorusunun cevabı. Ctrl'siz normal hover'da açılmaz (yazarken rahatsız etmesin); veriler kısa süre önbellekte tutulur. **0.24.0'ten itibaren aynı önizleme nesne gezgininde de var:** tabloya/görünüme/koleksiyona **sağ tık** → `👁 Önizleme (kolonlar + ilk 5 satır)`. İçerik birebir aynıdır; fark, kalıcı bir pencere açılmasıdır — metni kopyalayabilir, oradan `▶ İlk 100 satırı sekmede aç` ile derine inebilirsiniz. İki yüzey aynı önbelleği paylaşır: önce üzerine gelip sonra sağ tıklarsanız ikinci sorgu gitmez.
- **İmleçteki statement:** `Ctrl+Enter` yalnız imlecin üstündeki üst-seviye ifadeyi çalıştırır; çalışan aralık editörde seçilerek gösterilir. **Bu ayrıştırma T-SQL'e özgüdür** — diğer motorlarda kısayol, seçili metni (yoksa tümünü) çalıştırmaya düşer.
- **Akıllı sekme adı:** "sorgu1.sql" sekmeleri, başarılı çalıştırmadan sonra "SELECT Musteri", "EXEC spRapor" gibi anlamlı adlar alır.
- **Hataya çift tık:** Mesajlar'da "Satır N" içeren hata satırına çift tıklayınca imleç editörde o satıra gider.
- **Oturum kurtarma:** Uygulama kapanınca açık sekmeler (içerikleriyle) saklanır, sonraki açılışta geri gelir — **profil başına**. `Ctrl+Shift+T` son kapatılan sekmeyi geri açar.
- **Uzun sorgu bildirimi:** 10 sn'den uzun süren sorgu, siz başka sekmedeyken ya da pencere arkadayken bitince sağ altta tıklanabilir bir bildirim çıkarır.
- **Sözdizimi renklendirmesi:** geniş bir T-SQL sözcük kümesi **mavi** boyanır — cümlecikler (`SELECT/ORDER BY/PARTITION BY/OVER…`), tablo ipuçları (`WITH (NOLOCK)`, `UPDLOCK…`), pencere/analitik fonksiyonlar, veri tipleri ve `AS` dâhil. **Sayılar ayrı renkte** (turuncu) gösterilir; yorumlar yeşil, `'metin'` kırmızıdır. Renkler açık/koyu temaya göre uyarlanır.

8. MongoDB ile Çalışmak

MongoDB'de SQL yoktur. SQLST'te **sorgu metni tek bir JSON belgesidir** — ne yazdığınızı yalnız sunucu yorumlar, araç araya girmez. Üç kalıp vardır:

1) Belge okuma — `find`

```
{
  "find": "musteri",
  "filter": { "sehir": "Ankara", "bakiye": { "$gt": 1000 } },
  "sort":   { "bakiye": -1 },
  "projection": { "ad": 1, "bakiye": 1, "_id": 0 },
  "limit": 200
}
```

2) Toplulaştırma — `aggregate`

```
{
  "aggregate": "siparis",
  "pipeline": [
    { "$match": { "durum": "tamam" } },
    { "$group": { "_id": "$musteriId", "toplam": { "$sum": "$tutar" } } },
    { "$sort":  { "toplam": -1 } }
  ]
}
```

3) Diğer her şey — komut

Yukarıdaki iki kalıba uymayan belgeler doğrudan sunucuya komut olarak gider: `insert`, `update`, `delete`, `drop`, `createIndexes`, `dbStats`, `count` …

```
{ "insert": "musteri", "documents": [ { "ad": "Ayşe", "sehir": "İzmir" } ] }
```

Sonuçlar nasıl gösterilir?

- Kolonlar, dönen belgelerdeki **üst düzey alanların birleşimidir** (görülme sırasına göre). Bir belgede olmayan alan o satırda *NULL* görünür — belge veritabanında bu normaldir.
- Skaler değerler doğal tiplerine çevrilir (`ObjectId` → metin, `Decimal128` → ondalık).
- **İç içe belge ve diziler** hücrede JSON olarak görünür; hücreye çift tıklayınca tam belge biçimli olarak açılır.

> **Olmayan koleksiyon sessiz kalmaz:** Seçili veritabanında bulunmayan bir koleksiyonu sorgularsanız MongoDB normalde boş sonuç döndürür — "yanlış DB seçtim" ile "veri yok" ayırt edilemez. SQLST bunun yerine açık bir hata verir ve o veritabanındaki koleksiyonları listeler.

> **Bilinçli kapsam dışı:** `mongosh`'un JavaScript söz dizimi (`db.musteri.find({...})`) desteklenmez — sorgular saf JSON belgesi olarak yazılır.

9. Sonuç Bölgesi

- Birden çok result set alt alta ayrı grid'lerde görünür (SP çıktıları dahil).
- NULL hücreler soluk italik *NULL* olarak gösterilir; kolon başlığının ipucunda tip yazar.
- **Tek geçişte yükleme** `0.24.0`**:** sonuç okunurken satırlar **doğrudan** grid tablosuna yazılır — ara bir liste tutulup sonra kopyalanmaz. Bu, satır başına bellekten **%36** tasarruf sağlar (ölçüm: 200.000 satır × 10 kolon — 708 → 455 bayt/satır) ve tepe bellek tam da çökmelerin yaşandığı *yükleme* anında düşer. 0.23.0'teki **"ilk 1.000 satır"** bandı **kaldırıldı**: sonucu ikiye bölüp ilk parçayı ayrıca kopyalamak hem bellek hem iş demekti; SSMS de böyle çalışmaz. Uzun sorguda ■ **Durdur** her an keser ve eldeki satırlar ekranda kalır.
- Grid iki tavanla korunur: **satır sayısı** (100.000) **ve bellek boyutu** (~256 MB) — hangisi önce dolarsa okuma orada durur ve "ilk N satır gösteriliyor" bandı çıkar (bellek nedeniyleyse "bellek sınırına ulaşıldı — geniş satırlar"). Sonuç sessizce kırpılmaz; geniş satırlı (nvarchar(max)/ikili/XML) "zor" sorgular artık donma/çökme yapmaz. Tümü için **⬇ Tümünü dışa aktar**.
- **Uygulama bellek tavanı** `0.23.0`**:** yukarıdaki bütçeler *tahmindir*; bellekteki nesne grafiği ham veriden kat kat büyük olabilir. Bu yüzden ayrıca sürecin **gerçek** yığını izlenir — tavana dayanınca okuma kesilir ("🧱 Uygulama bellek tavanına dayandı"), eldeki satırlar gösterilir. Amaç tek: uygulama **asla çökmesin**.
- **Seçim istatistiği:** Birden çok hücre seçince set başlığında `seçim: N hücre · SUM · AVG · MIN · MAX` özeti belirir (Excel durum çubuğu gibi).
- **Çok büyük hücre değerleri** `0.22.0`**:** MB'lık JSON/metin içeren sütunlarda gridde yalnız **ilk 2.000 karakter** gösterilir ve sonuna "… *(N karakter — tamamı için hücreye çift tık)*" notu düşer. **Tam değer** çift tıkta, kopyalamada ve dışa aktarmada aynen durur. Bu sınır olmadan tek bir dev hücre uygulamayı dondurup çökertebiliyordu (gerçek saha bulgusu).
- **ORDER BY yoksa sıra** `0.22.0`**:** Tek tablolu ve sıralamasız bir SELECT'te sonuç, görüntüde **birincil anahtara göre** dizilir ve mesajlara bilgi notu düşer. **Sorgunuz değiştirilmez** — SQL'de sıra garantisi yoktur, dizme yalnız ekrandadır.
- **Kolon seçme ve sıralama:** kolon **başlığına tıklamak kolonun tamamını seçer** (SSMS gibi) — **Ctrl+tık** kolon ekler, **Shift+tık** yan yana kolonları seçer; durum satırında seçimin toplamı/ortalaması görünür, **Ctrl+C** değerleri kopyalar. **Sıralama** başlıkta **sağ tık** → ↑ **Artan** / ↓ **Azalan** / ✕ **Sıralamayı kaldır**; sıralı kolonun başlığında ▲/▼ görünür. Çok büyük sonuçta kolon ekranda boyanmaz (donmasın diye) ama istatistik ve Ctrl+C yine kolonun tamamını kapsar.

Gridden hızlı işler 0.22.0

İşlem	Nasıl / ne yapar
Değer açıklaması (ipucu)	🔗 Yabancı anahtar kolonundaki bir hücrede fareyle beklersiniz: **3 → Onaylandı** ve altında kaynağı (`dbo.Durum.Ad`) görünür. Tanım tablosu **bir kez** okunup bellekte tutulur; sonraki hücrelerde sunucuya hiç gidilmez. Yalnız gerçek tanım tablolarında çıkar (çok büyük tablolarda çıkmaz).
Hızlı filtre	Hücreye sağ tık → **Bu değere filtrele** / **Bu değeri hariç tut** / sayı-tarih kolonlarında **≥** ve **≤**. Koşul sorgunuza eklenir (varsa `WHERE` 'e AND'lenir, `ORDER BY` 'ın önünde kalır), editördeki metin de güncellenir ve sorgu yeniden çalışır. Beğenmezseniz `Ctrl+Z`. Boş hücrede `IS NULL` kurulur. Yalnız SQL Server'da.
Kolon istatistiği	Kolon **başlığına** sağ tık → 📊 **İstatistik**: satır sayısı, farklı değer, en küçük/en büyük, sayısal kolonda ortalama ve toplam, NULL oranı — **tek sorguda**. Sorgunuzun kendi süzgeci (WHERE/JOIN) korunur; sayılar gridde görünen değil **eşleşen tüm satırlar** üzerinden hesaplanır. Sorguda `TOP` varsa pencere bunu ayrıca söyler.

Dışa aktarma ve kopyalama

Bu bölümün tamamı motor-nötrdür: MongoDB sonucunu da CSV/Excel/JSON olarak dışa aktarabilirsiniz.

Öğe	İşlevi
Değer biçimi	**Hiçbir alan biçimlenmez — veritabanında nasılsa öyle** (SSMS ile birebir): `datetime` → `2026-10-05 14:23:11.123` · `datetime2(3)` 3 kesir hanesi · `date` → `2026-10-05` · `decimal` → `1250.75` (nokta, binlik ayıraçsız) · `money` → 4 hane · `bit` → `1` / `0` · `uniqueidentifier` BÜYÜK harf. Grid, Ctrl+C, kopyala düğmeleri, CSV, HTML rapor ve hücre görüntüleyici hep aynı metni verir. (Bilgisayarın bölge ayarı yalnız SQLST'nin kendi yazılarını — süre, satır sayısı — etkiler.)
`Ctrl+C`	Seçili hücreyi/hücreleri **ham değeriyle** panoya alır: tek hücrede sona satır sonu **eklenmez** (WHERE koşuluna temiz yapıştırılır), uzun değer kırpılmadan tamamı gider, çok satırlı metin gerçek satır sonlarıyla korunur, NULL "NULL" yazılır; çoklu seçim sekmeli metin olur.
Başlıklı kopyala / Başlıksız	Seçili satırları (yoksa tümünü) sekmeli metin olarak panoya alır — Excel'e doğrudan yapıştırılır.
↓ CSV	Excel uyumlu CSV (UTF-8 BOM, ; ayraçlı).
↓ Excel	Tip koruyan .xlsx: sayılar sayı, tarihler tarih olarak gider; başlık satırı donuktur.
↓ JSON	Tip koruyan JSON dizisi (sayı/bool ham, tarih ISO 8601, binary base64).
Sağ tık → INSERT / IN listesi olarak kopyala	Seçili satırlardan T-SQL `INSERT` script'i ya da tekilleştirilmiş `IN (…)` listesi. **Yalnız SQL Server'da** (T-SQL üretirler).
Sağ tık → 🕸 Kayıt haritası	Satırın FK'larla bağlı ▲ **üst** ve ▼ **alt** kayıtlarını (dolu olanları, bir adım ötesiyle) ayrı grid'ler hâlinde getirir. **Zincirde derine inmek için haritanın sonucundaki herhangi bir satıra tekrar sağ tık → Kayıt haritası** deyin — kaynak tablo o satırın kendi tablosundan çözülür (talep → belge → belgenin altındakiler… istediğiniz kadar).
Hücreye çift tık / Sağ tık → Hücreyi görüntüle	Uzun içerik tam metin penceresinde açılır; içerik geçerli JSON ise girintili biçimlenir (Mongo'nun iç içe belgeleri için birebir budur).

Kolon araçları (göster/gizle)

Geniş sonuçlarda (çok kolonlu, çoğu NULL) ekranı sadeleştirmek için sonuç şeridindeki Kolonlar ▾ düğmesi:

- **Göster/gizle:** açılan listede her kolonun onay kutusu; işareti kaldırınca kolon gizlenir, tekrar işaretleyince geri gelir. Üstteki kutuya yazarak kolon **arayabilirsiniz** (50 kolonlu tabloda pratik).
- 🧹 **Boş kolonları gizle:** bu sonuçta **tümü NULL/boş** olan kolonları tek tıkla saklar (ör. çoğu NULL bir Kişi satırında 20 boş kolon kaybolur, geriye dolu alanlar kalır).
- **Tümünü göster:** gizlenenleri geri getirir.
- **Kolon başlığına sağ tık → 🚫 Bu kolonu gizle:** huniyi açmadan tek kolonu gizlemenin kısayolu.

> Bu yalnız **görünümü** etkiler — veriye, kopyalamaya ya da dışa aktarıma dokunmaz; her yeni sorguda tüm kolonlar yeniden gelir.

10. Sorgu Geçmişi ve Oturum

- Her çalıştırma (başarı/hata/iptal dahil) **yalnız bu bilgisayardaki** yerel depoya yazılır: sorgu metni, sunucu/DB, zaman, süre, satır sayısı, hata mesajı.
- **Geçmiş profil bazlıdır:** her bağlantı profilinin kendi geçmişi vardır. Üretim SQL Server'ınızın sorguları Mongo profilinizde karşınıza çıkmaz. Listeleme, arama, temizleme ve otomatik budama (profil başına son 1000 kayıt) hep bu kapsamda çalışır.
- Sağ kenardaki **"🕘 Sorgu geçmişi"** şeridine tıklayarak ya da `Ctrl+H` ile panel açılır.
- Panelde sorgu metninde arama yapılır; kayda **çift tık** sorguyu yeni sekmede açar (çalıştırmaz).
- Gizlilik: `Temizle` o profilin geçmişini siler; "Geçmişi kaydet" anahtarı kapatılırsa yeni kayıt tutulmaz.

11. Güvenli Yazma Modu 🛡 SQL ailesi

SQLST'in imza özelliği. Araç çubuğundaki 🛡 **Güvenli yazma** anahtarıyla açılır (varsayılan kapalıdır; kapalıyken davranış bildiğiniz gibidir). SQL Server, PostgreSQL, MySQL/MariaDB ve Oracle'da kullanılabilir; işlem ifadeleri her motorun kendi söz diziminde gönderilir.

- Mod açıkken yazma sorgusu otomatik olarak **açık bir işlem** içinde çalışır.
- Çalışma bitince editörün üstünde kaçırılamaz sarı bant belirir: **"🛡 İşlem açık — N satır etkilendi. Kalıcı olsun mu?"** + geri sayım.
- `✓ COMMIT` değişikliği kalıcılaştırır (öncesinde işlemin sağlığı `XACT_STATE()` ile denetlenir); `↩ ROLLBACK` geri alır — veri hiç değişmemiş olur.
- Süre dolarsa (varsayılan 5 dk) **otomatik ROLLBACK** yapılır — açık kilit unutulmaz.
- Hatalı ya da iptal edilen yazma **anında** geri alınır; yarım iş açık kalmaz.

> **Bilinçli sınırlar:** İşlem açıkken kilitler tutulur; tetikleyiciler çalışmış, IDENTITY / auto_increment değerleri tüketilmiş olabilir — ROLLBACK bunları geri almaz. Bant açıkken aynı sekmede yeni sorgu çalıştırılamaz (önce karar verin).

Motora göre kapsam — DDL geri alınabilir mi?

Modun tek sorusu şudur: **bant açıldıktan sonra motor gerçekten geri alabiliyor mu?** Cevap veri değiştiren ifadelerde (INSERT/UPDATE/DELETE) her motorda evettir; **şema değiştiren ifadelerde (CREATE/ALTER/DROP/TRUNCATE) motorlar ayrışır.**

Motor	DML	DDL	Not
SQL Server	✓	✓	DDL de işlemseldir.
PostgreSQL	✓	✓	**Transactional DDL** — `CREATE TABLE` bile geri alınabilir. PostgreSQL'in ayırt edici gücü.
MySQL / MariaDB	✓	—	DDL **örtük COMMIT** yapar: çalıştığı anda kalıcıdır.
Oracle	✓	—	DDL örtük COMMIT yapar — üstelik öncesinde bekleyen işlemi de kalıcılaştırır.

MySQL/Oracle'da DDL çalıştırırsanız ne olur? Bant **açılmaz** ve Mesajlar'da şu satırı görürsünüz: "🛡 *Güvenli Yazma bu ifadede DEVRE DIŞI: bu motorda DDL örtük COMMIT yapar — çalıştığı anda kalıcıdır ve geri alınamaz.*" Size geri alamayacağı bir şey için "geri al" düğmesi göstermektense, durumu açıkça söylemeyi tercih ettik.

Diğer dürüst sınırlar:

- **MongoDB'de Güvenli Yazma yoktur** ve anahtar görünmez: çok-belgeli işlem **replica set** gerektirir, tek düğümlü bir kurulumda motor bu sözü tutamaz.
- **MySQL'de işlemsiz tablo motorları (MyISAM):** bu tablolarda DML bile geri alınamaz. Tablo motoru sorgudan anlaşılamadığı için araç bunu tespit edemez — MyISAM kullanıyorsanız Güvenli Yazma'ya güvenmeyin.
- **İşlem sağlığı denetimi** (COMMIT öncesi "işlem hâlâ sağlam mı?") yalnız SQL Server'da yapılabilir (`XACT_STATE`). Diğer motorlarda kararınız doğrudan uygulanır; sunucu bir sorun bildirirse mesajı olduğu gibi görürsünüz.

12. Koruma Kapıları

Bu kapılar Güvenli Yazma'dan bağımsızdır ve her zaman açıktır.

Kapı	Motor	Ne zaman devreye girer
Filtresiz yazma uyarısı	**Hepsi**	WHERE'siz `UPDATE` / `DELETE` tespit edilirse (satır ve hedef adıyla) onay sorulur — varsayılan cevap Hayır'dır. SQL Server'da T-SQL ayrıştırıcısıyla, PostgreSQL/MySQL/Oracle'da metin çözümlemesiyle (yorumlar ve metin sabitleri ayıklanarak), **MongoDB'de boş filtreli** `delete` / `update` (`"q": {}`) tespitiyle. `TOP(n)` 'li olanlar bilinçli sınır sayılır, sorulmaz.
Salt-okunur profil	**Hepsi**	Yazma sınıfı sorgu sunucuya **hiç gönderilmez**. SQL Server'da ikinci katman `ApplicationIntent=ReadOnly` .
Seçili DB dışına yazma	Yalnız MSSQL	Sekmede X veritabanı seçiliyken sorgu üç parçalı adla (`db.şema.tablo`) ya da `USE` ile **başka** veritabanına yazıyorsa onay sorulur. Salt-okuma cross-DB sorguları serbesttir. (Üç parçalı ad ve `USE` T-SQL kavramlarıdır.)

13. Edit Modu (Grid'de Veri Düzenleme) SQL ailesi

SQL Server, PostgreSQL, MySQL/MariaDB ve Oracle'da kullanılabilir. Üretilen DML her motorun kendi söz diziminde yazılır — tırnaklama, metin/tarih/ikili literalleri ve işlem ifadeleri motora göre değişir; siz farkı görmezsiniz, ama önizlemede görebilirsiniz.

- Ağaçta tabloya sağ tık → ✏ **Düzenle** ile açılır — ilk **1.000** satır gelir 0.24.2 (eskiden 200; geniş satırlı tabloda bellek bütçesi daha azında keser ve artık **nedenini bilgi satırında söyler**). Daraltmak için sekme içi sorguya WHERE yazın.
- **Sekme içi sorgu kutusu gerçek editördür** 0.24.2: SQL renklendirme, ana editörle aynı **IntelliSense** (tablo/kolon önerileri, Ctrl+Space) ve Ctrl+F arama. F5 / Ctrl+Enter çalıştırır.
- Hücre düzenleme, satır ekleme (alttaki boş satır) ve satır silme (`Del`) yapılır; **sunucunun ürettiği kolonlar salt hücredir** — IDENTITY / auto_increment / identity kolonu, üretilmiş (computed / generated / virtual) kolonlar ve rowversion.
- **PK'sız tabloda grid salt-okunurdur** — "hangi satır?" belirsizliğiyle yanlış satır güncellenmez.
- 💾 Uygula üretilecek DML'i **önce önizleme penceresinde gösterir** (Show Script); onaydan sonra TEK transaction'da, komut komut çalıştırılır.
- Bir komut hata alır ya da 0 satır etkilerse (başkası aynı satırı değiştirmişse — çakışma) **tümü geri alınır** ve durum açıkça bildirilir.

Toplu işlemler yeni

Üç kısayol, tek tek hücre düzenlemeyi hızlandırır. **Hiçbiri sunucuya doğrudan gitmez** — üçü de bekleyen değişiklik kümesine yazar; 💾 Uygula'da yine DML önizlemesini görür ve onaylarsınız, komutlar yine satır satır ve **birincil anahtarlı WHERE** ile üretilir. Yani çakışma tespiti ve tek transaction güvencesi aynen geçerlidir.

İşlem	Nasıl	Ne yapar
∅ NULL ata	`Ctrl+0`	Seçili hücrelere NULL yazar. **NULL kabul etmeyen** ve sunucunun ürettiği kolonlar atlanır; kaç tanesinin neden atlandığı alt bantta yazar.
Çoklu satır yapıştırma	`Ctrl+V`	Panodaki sekmeyle ayrılmış bloğu (Excel ya da aracın kendi kopyalaması) seçili hücreden başlayarak sağa ve aşağı yerleştirir. Bloğun satırları tablodan fazlaysa yeni satır eklenir.
⇊ Kolonu toplu güncelle…	düğme	Bir kolonun tüm satırlarına aynı değeri (ya da NULL) yazar.
Sekme içi sorgu	üst kutu + ▶ Çalıştır (F5 / Ctrl+Enter)	**(0.15.0 — ayrı "Sorgu penceresi"nin yerini aldı)** Şeridin altındaki kutuya sorgu yazın; sonucu **aynı griddedir ve DÜZENLENİR**. Boş bırakılırsa tablonun ilk 200 satırı gelir. Düzenleyebilmek için sorgu **PK kolonlarını da SELECT etmelidir** — etmezse grid salt-okunur kalır ve alt bant nedenini PK adlarıyla söyler.

Kapsam: yalnız ekrandaki satırlar. Edit modu tabloyu ilk 200 satırla açar; toplu güncelleme de **yalnız o yüklü satırları** değiştirir, tablonun tamamını değil. Tüm tabloyu güncellemek istiyorsanız sorgu sekmesinde `UPDATE` yazın — orada Güvenli Yazma bandı ve WHERE'siz DML uyarısı devreye girer.

Değerler ham gösterilir ve ham okunur. Edit grid'i de sonuç grid'i gibi değerleri **veritabanındaki biçimle** gösterir (`2026-10-05 14:23:11.123` , `1250.75`). Hücreye yazarken ya da yapıştırırken ondalık için **nokta da virgül de** doğru okunur (`2.5` ve `2,5` → 2,5). **Binlik ayıracı kullanmayın:** nokta her zaman ondalık sayılır — `1.250` bin iki yüz elli değil **1,25** olarak yazılır (veritabanının kendi okuması da budur). Uygula önizlemesinde sayıları gözden geçirin.

Panodaki `NULL` metni NULL olarak yazılır (kopyalama da NULL hücreleri böyle yazar, gidiş-dönüş bozulmaz). Çevrilemeyen bir değer o hücreyi atlar, **diğerlerini bozmaz** ve nedeni alt bantta görünür.

Çakışma nasıl anlaşılır?

SQL Server'da tabloda bir `rowversion` kolonu varsa çakışma tespiti onunla yapılır — en güvenilir yol. **Diğer motorlarda (ve rowversion'sız SQL Server tablolarında)** kimlik PK ile, güvence ise **değiştirdiğiniz kolonların eski değerleriyle** sağlanır: siz düzenlerken başkası aynı hücreyi değiştirdiyse WHERE tutmaz, komut 0 satır etkiler ve tüm düzenleme geri alınır.

Neden diğer motorlarda rowversion kullanılmıyor? Çünkü birebir karşılığı yok ve benzeyenler yanıltıcı: PostgreSQL'in `xmin` 'i uygundur ama grid'in yükleme sorgusunu da değiştirmeyi gerektirir (ileride eklenebilir); MySQL'in `ON UPDATE CURRENT_TIMESTAMP` 'i bir sürüm belirteci *değildir* (aynı saniyedeki iki yazma aynı değeri alır); Oracle'ın `ORA_ROWSCN` 'i tablo

`ROWDEPENDENCIES` ile yaratılmadıysa blok düzeyindedir ve **yanlış çakışma** üretir. Yanlış güven vermektense eski-değer kıyası tercih edildi.

Motora özgü sınırlar:

- **Oracle:** tüm kolonları varsayılana bırakan satır eklenemez (Oracle'da karşılığı yoktur) — yeni satırda en az bir kolon doldurun. Boş RAW/BLOB değeri de yazılamaz. Edit modu **Oracle 12c ve üzeri** ister.
- **MongoDB:** Edit modu yoktur ve menü öğesi görünmez. Belge veritabanında sabit şema yoktur (her belgenin alanları farklı olabilir), alanlar iç içe olabilir ve `_id` dışında bir satır kimliği kavramı bulunmaz — grid düzenleyicisi burada sessizce yanlış belge yazardı. Belgeleri sorgu sekmesinden `update` komutuyla değiştirebilirsiniz.

14. Yönetim Paneli 🛠 motora göre

Üst şeritten açılır. **Panel bağlı olduğunuz motorun kendi teşhis araçlarını gösterir** — beş motorda da vardır, ama içerikleri farklıdır. Hepsi salt-okunurdur: panel hiçbir zaman kendiliğinden değişiklik yapmaz, yalnız gösterir ve gerektiğinde script'i sekmede açar.

SQL Server

En zengin paneldir; dört bölümdür. Üstte her zaman **uptime bandı** vardır: DMV sayaçları sunucu yeniden başlatılınca sıfırlandığından, veriler hangi tarihten beri birikiyorsa yazar; 7 günden kısaysa kırmızımsı "temkinli yaklaşın" uyarısı çıkar.

Eksik indexler

- Sunucunun önerdiği eksik index'ler **güç kovalarıyla** (YÜKSEK/orta/düşük) sıralanır — skor yalnız sıralama içindir, "şu kadar hızlanır" vaadi değildir. Düşükler varsayılan gizlidir.
- **Örtüşme analizi:** Öneri mevcut bir index tarafından zaten kapsanıyorsa "zaten var", yalnız INCLUDE eksikse "mevcuda INCLUDE ekle" notu düşülür — gereksiz index üretmezsiniz.
- Çift tık, uyarı yorumlu CREATE INDEX script'ini **sekmede açar; asla kendiliğinden çalıştırmaz**.

Kullanılmayan indexler

- Yeniden başlatmadan beri hiç okunmamış ama her yazmada güncellenen index'ler listelenir.
- Unique/PK ve yabancı anahtar destekleyen index'ler aday bile yapılmaz. Öneri **DISABLE**'dır, silme değil — geri almak `REBUILD` kadar kolaydır.

Yavaş sorgular

- İki liste iki ayrı soruya bakar: **En yavaş 25 sorgu (ortalama süre)** — tek çalışışı en uzun sürenler, "ekran kilitlendi" şikâyetinin yeri; **Toplam süreyi en çok tüketen 25 sorgu** — tek başına hızlı ama çok koşan, sunucu genelinde kazanç oradadır.

- Ortalama süre ile çalıştırma sayısını **birlikte** okuyun. Kaynak plan önbelleğidir (`sys.dm_exec_query_stats`): sunucu yeniden başlayınca ya da önbellek temizlenince sayaçlar sıfırlanır — üstteki uptime bandı bu yüzden önemlidir.
- Yavaşlığın sebebi çoğu kez eksik index'tir: **Eksik indexler** sekmesi öneriyi hazır `CREATE INDEX` script'iyle verir. (MongoDB bağlantısında aynı iş "Index önerileri / Yavaş sorgular" sekmesindedir.)

RCSI paneli

- Veritabanı başına RCSI / ALLOW_SNAPSHOT durumu.
- **Tek tuşla RCSI açma yoktur** — riskleri panelde yazılıdır; aşamalı geçiş script'i yalnız incelenmek üzere sekmeye açılır.
- Bu bir *izleme* paneli değil, **karar** panelidir: veritabanı başına bir kez verilen "RCSI'yi açayım mı?" kararı içindir. NOLOCK'a sarılma ihtiyacının asıl sebebi genelde RCSI'nin kapalı olmasıdır.

Havuz

En pahalı 20 sorgu (CPU), bağlantı sayıları, son tam yedek zamanları ve seçili veritabanının en büyük 50 tablosu — hepsi tek yenilemeyle.

Bakım yeni

Seçili veritabanına bakan teşhisler (üstteki DB seçicisiyle değişir); 0.16.2 ile **bekleme analizi** eklendi — toplam altı bölüm:

- **tempdb ne ile dolu?** Şişmenin *kaynağını* ayırt eder: kullanıcı nesnesi (geçici tablo), iç nesne (sıralama/hash'in belleğe sığmayıp taşması) ya da version store (RCSI/snapshot). Altında **tempdb'yi en çok tüketen oturumlar** listelenir — "kim şişiriyor" sorusunun cevabı.
- **Bloklama zinciri.** Kimin kimi beklettiğini gösterir. Asıl suçlu genelde zincirin **kökündedir** ve çoğu zaman **uykudadır** — açık işlem tutup bekleyen bir uygulama hiçbir sorgu çalıştırmadığı için "çalışan sorgular" listelerinde hiç görünmez. Bu bölüm kökü de zincirle birlikte gösterir; bloklama sorusunun tek adresi burasıdır.
- **İstatistik tazeliği.** Son güncellemeden bu yana geçen gün ve değişen satır oranı. Execution plan'daki **tahmin/gerçek sapmasının** bir numaralı kök nedeni budur. *Index bakımı düşünüyorsanız buraya bakın:* pratikte kazancın büyük kısmı istatistik güncellemesinden gelir.
- **Bekleyen istekler (0.16.2).** ŞU AN bir şey bekleyen tüm istekler — bloklamayla sınırlı değil: *ne bekliyor* (bekleme türü), *ne kadardır*, varsa *kimin bloğunda* ve hangi sorgu. "Sunucu şu an neden yavaş?" sorusuna anlık fotoğraf.
- **Sunucu geneli bekleme istatistikleri (0.16.2).** Başlangıçtan beri en çok zaman yakan 15 bekleme türü, yüzde payı ve CPU kuyruğu süresiyle; zararsız (arka plan) türler süzülür. Yüzdesi yüksek tür, sunucunun *kronik* darboğazını adlandırır — ör. `PAGEIOLATCH` disk, `CXPACKET/CXCONSUMER` paralellik, `LCK_M_*` kilit demektir.

> **Her bölümde "👁 Neyi takip edin" bandı vardır.** Panelin her alanı iki metin taşır: **açıklama** o bölümün *ne olduğunu*, **TAKİP notu** ise *hangi sütuna bakacağınızı, hangi değerin kötü sayıldığını ve ne yapmanız gerektiğini* söyler. Notlar bilerek "şu sütun şu eşiği geçerse şunu yap" biçiminde yazılmıştır — "bu bölüm önemlidir" türü genel geçer cümleler kullanıcıya hiçbir şey öğretmez. Birkaçı sizi *yanlış sonuca varmaktan* alıkoymak için de uyarır (ör. index kullanım sayaçları sunucu yeniden başlayınca sıfırlanır — "hiç kullanılmamış" görünen index ay sonu raporuna hizmet ediyor olabilir).

PostgreSQL

Altı bölüm: **kullanılmayan index'ler** (`pg_stat_user_indexes`), **tablo boyutu ve tarama biçimi** (seq scan / index scan oranı + ölü satır sayısı ve VACUUM tarihleri — şişmenin neden temizlenmediği), **canlı oturumlar** (bekleyenler işaretli), **bloklama zinciri** (`pg_blocking_pids` — kök bloklayan işaretli; genelde `idle in transaction` durumundadır), **istatistik tazeliği** (`n_mod_since_analyze` oranı) ve **veritabanı sağlığı** (önbellek isabet oranı, işlem sayaçları).

PostgreSQL'de ayrı bir "tempdb" bölümü yoktur: geçici dosya kullanımı zaten "veritabanı sağlığı" bölümündedir. Index parçalanması için de ayrı bölüm konmadı — PG'de bunun karşılığı ölü satır şişmesidir ve "tablo boyutları" bölümünde gösterilir.

*2026-07-19 sadeleştirmesinde **"Kilit çakışmaları" bölümü kaldırıldı:** bekleyenleri gösteriyor ama kimin kökte olduğunu söylemiyordu — yani "bloklama zinciri"nin eksik bir kopyasıydı. Bloklama sorusunun tek adresi artık zincir bölümüdür. Aynı gerekçeyle ANALYZE tazeliği tek yere toplandı: tarihler artık "tablo boyutları"nda tekrarlanmıyor.*

MySQL / MariaDB

Altı bölüm: **index kullanımı** (`performance_schema`), **tablo boyutları ve parçalanma** (`DATA_FREE`), **çalışan işlemler**, **açık işlemler ve kilit beklemeleri** (`INNODB_TRX` — "açık ama boşta" işlemler işaretli), **istatistik tazeliği** (`innodb_table_stats`) ve **sunucu durumu** (`SHOW GLOBAL STATUS`).

> **"Sunucu durumu" neden ham bir döküm?** Panelin tek süzülmemiş bölümüdür ve kaldırılması değerlendirildi; MySQL panelinin *tek* sunucu-geneli görünümü olduğu için kaldı. Süzülmüş bir sürüm taşınabilir değil: `information_schema.GLOBAL_STATUS` MySQL 8'de kaldırıldı, `performance_schema.global_status` ise MariaDB'de yok — ikisinde birden çalışan tek yol `SHOW` 'un kendisi. Süzemediğimiz için bölümün açıklamasında **hangi sayaçlara bakılacağı** tek tek yazılıdır (Threads_running, Slow_queries, Innodb_row_lock_waits, Created_tmp_disk_tables...).

Oracle

Altı bölüm: **index'ler ve durumları**, **segment boyutları**, **aktif ve bloklayan oturumlar**, **istatistik tazeliği** (optimizer'ın kör kaldığı tablolar), **bloklama zinciri** (kök bloklayan işaretli) ve **geçici alan**

kullanımı (`V$TEMPSEG_USAGE` — tempdb şişmesinin Oracle karşılığı).

Oracle'da "aktif oturumlar" bölümündeki `BLOCKING_SESSION` sütunu bilerek bırakıldı: bir bölümün tekrarı değil, "bu oturum birini bekliyor" işaretidir — zincirin kendisi ayrı bölümdedir.

> **MySQL/MariaDB ve Oracle bölümleri henüz gerçek sunucuda denenmedi.** SQL Server ve PostgreSQL bölümleri canlı doğrulandı; diğer ikisi yol haritasındaki canlı doğrulama borcuna bağlıdır. Bir bölüm o sunucuda çalışmazsa panel çökmez — yalnız o bölüm kendi hata bandıyla gelir.

MongoDB

Dört bölüm: **Index kullanımı** (`$indexStats` — her index'in kaç kez kullanıldığı; **hiç kullanılmayanlar en üstte ⚠ ile**, çünkü her yazmada maliyet öderler), **Koleksiyonlar** (`$collStats` — belge sayısı, veri/depolama/index boyutu, büyükten küçüğe), **Veritabanı özeti** (`dbStats`), **Çalışan işlemler** (`currentOp` , kilit bekleyenler işaretli).

"Veritabanı özeti" sayısal olarak "Koleksiyonlar"ın toplamıdır; sadeleştirmede kaldırılması değerlendirildi ama kaldı — tek bir `dbStats` komutudur, oysa koleksiyon dökümü her koleksiyon için ayrı komut çalıştırır. Yüzlerce koleksiyonlu bir veritabanında "bu DB ne kadar büyük?" sorusunu ucuza cevaplayan tek yer burasıdır.

Beşinci bölüm — 🔍 Index önerileri / Yavaş sorgular `0.22.0` **:** "Index kullanımı" var olan index'lerin **kullanılıp kullanılmadığını** gösterir (silinebilecekler); bu yeni bölüm ise tersini söyler: **hangi index EKSİK**. Üstteki liste, gerçekten çalışmış ve **hiç index kullanmamış** (COLLSCAN) sorgulardan üretilen önerilerdir — koleksiyon, alanlar, kaç kez çalıştığı, toplam kaç milisaniye yediği ve hazır `createIndex` komutuyla. Alttaki liste ham **yavaş sorgu geçmişi**dir (süre, incelenen/dönen belge sayısı, plan özeti, zaman).

> **Bu bölüm profiler'a bağlıdır:** MongoDB'de SQL Server'daki gibi "eksik index" veren bir sistem görünümü **yoktur**; tek dürüst kaynak `system.profile` koleksiyonudur. Profiler kapalıysa panel boş kalmaz, nedenini ve açma komutunu yazar: `db.setProfilingLevel(1, { slowms: 100 })` . **Öneriler çalıştırılmaz** — komutu kopyalayıp kendi kontrolünüzle koşarsınız (her index yazma maliyeti ve bellek demektir). Filtresiz tarama yapan sorgular için öneri üretilmez (onları index kurtarmaz).

> **Bölüm bazlı hata yalıtımı:** Bir bölüm sunucu ayarı ya da yetki nedeniyle okunamazsa (ör. MariaDB'de `performance_schema` kapalıysa) panelin tamamı bozulmaz — yalnız o bölüm nedenini yazar, diğerleri çalışmaya devam eder.

15. Nesne Tarihçesi 🕘

- Bir view/SP/fonksiyonun tanımı bu araçla her görüldüğünde (ya da SQL Server'da siz ALTER çalıştırmadan hemen önce) içerik yerel tarihçeye sürüm olarak kaydedilir. Aynı içerik ikinci kez sürüm açmaz.
- Sağ tık → 🕘 **Tarihçe**: sürüm listesi; tek sürüm seçince bir öncekiyle, iki sürüm seçince ikisi arasında **yan yana renkli fark** gösterilir.
- `Seçili sürümü ALTER olarak sekmeye aç` ile eski hâle dönüş script'i hazırdır (SQL Server).
- Tanımı açtığınızda son gördüğünüz hâlden farklıysa durum çubuğunda **"⚠ dışarıda değişti"** uyarısı çıkar.
- **Tarihçe profil bazlıdır:** aynı sunucuya bakan iki farklı profil birbirinin sürüm zincirini görmez ve bozmaz.

> **Dürüst sınır:** Tarihçe yalnız BU araçla görülen anları kaydeder — veritabanının gerçek denetim günlüğü değildir. MongoDB'de karşılığı yoktur (koleksiyonların "tanımı" yoktur).

16. Karşılaştırma Araçları ⇄

Sonuç diff'i

- Tipik kullanım: **aynı sorguyu** iki sekmede **farklı hedeflere** (sekme DB seçicisiyle — ör. test vs prod) karşı çalıştırın ya da aynı sorgunun önce/sonra iki çalıştırmasını alın.
- A ve B kaynaklarını seçin, ortak kolonlardan **anahtar kolon(lar)ı** işaretleyin, karşılaştırın: tek renkli grid'de ← yalnız A / → yalnız B / ≠ farklı (hücrede `eski → yeni`) satırları.
- Anahtar tekil değilse hangi değerin çakıştığı söylenerek dürüstçe reddedilir. Sonuç CSV'ye aktarılabilir.

Şema snapshot + fark

- `📸 Snapshot'ı dosyaya kaydet` seçili veritabanının şemasını (nesne+kolon+parametre) JSON dosyasına alır.
- Dosya ↔ canlı veritabanı ya da dosya ↔ dosya karşılaştırması: eklendi / silindi / değişti listesi (değişen imza `eski → yeni` olarak). "Migration gerçekten uygulandı mı?" doğrulamasının aracıdır.

HTML sonuç raporu

Sonuç bölgesinin sağ üstündeki (grid'in export butonlarının hemen üstündeki) `📄 Rapor`, son sorguyu + sunucu/DB + zaman + süre + tüm sonuç tablolarını tek .html dosyasına yazar — "bak şu sonuca" e-postasının kanıt eki. Yanındaki `⬇ Tümünü dışa aktar` ise son sorgunun **tüm** satırlarını

(100.000 grid sınırına takılmadan, akışla) CSV'ye yazar. Bu iki eylem sonuçlara ait olduğundan sorgu araç çubuğundan sonuç bölgesine alınmıştır.

17. Her Yere Atla Paleti (Ctrl+P)

`Ctrl+P` ile açılır; yazdıkça açık sekmeler, yüklü nesneler, veritabanları ve komutlar süzülür ("spmk" gibi kısaltmalar da eşleşir). `Enter` seçileni açar/çalıştırır: sekmeye atlar, nesnenin script'ini açar, veritabanı değiştirir ya da komutu (yeni sekme, Yönetim Paneli, Karşılaştır…) yürütür. Palet de motora göre süzülür — SQL Server dışında nesne girişleri "ilk N satır"a düşer, T-SQL script yolları görünmez.

18. Görsel Sorgu Tasarımcısı 🎨 SQL ailesi

SQL yazmadan JOIN'li sorgular kurmanın görsel yolu. Üst şeritteki 🎨 Görsel Sorgu düğmesi bir **tuval sekmesi** açar. Tuvalin kendisi kalıcı bir bağlantı tutmaz; ▶ Çalıştır ürettiği SELECT'i koşar ve **sonucu aynı sekmenin altındaki grid'de gösterir** (0.10.1 — script sekmesine geçilmez; satır ve bellek sınırları burada da geçerlidir). MongoDB ayrı bir dünya olduğundan bu düğme orada gizlidir.

1. **Tablo koy:** Nesne gezgininden bir tabloyu tuvale sürükleyip bırakın — ya da tabloya **sağ tık** → 🎨 **Görsel sorguya ekle** (0.20.2; sürüklemeye gerek yok, tasarımcı yoksa açılır, varsa tablo eklenip öne gelir). Kutu, tablonun kolonlarını checkbox'larla listeler.
2. **Bağla (JOIN):** Bir kutunun başlığındaki ∘ tutamağından tutup, bağlamak istediğiniz **diğer tablonun kutusunun üstüne sürükleyip bırakın** — hedef, karşı tablonun tüm kutusudur (küçük bir noktayı tutturmanız gerekmez). Aralarında bir çizgi (JOIN) oluşur ve **join düzenleyici hemen açılır**. İki tablo arasında **yabancı anahtar** varsa ON kolonları kendiliğinden önerilir. Düzenleyicide join tipini (INNER / LEFT / RIGHT / FULL — MySQL'de FULL gizli) ve ON kolon çiftlerini seçersiniz; sonradan değiştirmek için çizgiye **çift tıklayın**.
3. **Koşul (WHERE) — huni + çipler (0.12.0):** Kutu içindeki bir kolonun üstüne gelin, sağında beliren **huni** ikonuna tıklayın → küçük pencerede operatörü ve değeri girin (kolon **tarihse takvim**, **bit ise Evet/Hayır** açılır). Eklenen her koşul tuvalin üstünde **çip** olarak dizilir: çipe × ile kaldırın, çipler arasındaki **VE/VEYA** yazısına tıklayarak bağlacı değiştirin. Operatörler: `= ≠ > < ≥ ≤`, içerir/başlar/biter (LIKE) ve boş/dolu (IS NULL / IS NOT NULL). Değerler otomatik kaçışlanır (metin tırnaklanır, sayı tırnaksız kalır). Kutuda ⧩ rozeti dolu koşul sayısını gösterir. *Toplu düzenleme için kutuya çift tık da hâlâ çalışır.*
4. **Kolon seç:** Kutu içindeki kolon checkbox'larını işaretleyerek SELECT listesini daraltın. Hiçbiri işaretli değilse `SELECT *` üretilir.
5. **Çalıştır ya da script'e aç:** ▶ Çalıştır (F5) tuvaldeki her şeyden bir `SELECT … JOIN … WHERE` üretir, çalıştırır ve **sonucu doğrudan bu sekmenin altındaki grid'de gösterir** (0.10.1 — script sekmesi açılmaz; tuval sekmesindeyken `F5` de aynısını yapar). Sorguyu görmek/düzenlemek isterseniz ▶ Script'e aç onu yeni bir sekmede açar (çalıştırmadan).

> **Ters yön — Script → Görsel:** Var olan bir sorgunuz varsa üst şeritteki 🎨 → Görsele çevir düğmesi **aktif sekmedeki SELECT'i ayrıştırıp tuvale döker** — tablolar kutu, JOIN'ler çizgi, WHERE koşulları da kutu şartı olur. Böylece elle yazılmış bir sorguyu görsel olarak inceleyip değiştirebilirsiniz. Bu çevirme **yalnız SQL Server'da** görünür (T-SQL ayrıştırıcısı kullanır). Alt sorgu, türetilmiş tablo, fonksiyon çağrısı, `BETWEEN/IN`, **DISTINCT / GROUP BY / ORDER BY / TOP** ya da alt sorgulu bir `ON` gibi görsel modele sığmayan parçalar temsil edilemez. **Görsel sorgu bu durumda orijinalden farklı sonuç verebileceğinden**, çevirmeden önce **tam olarak neyin atlanacağını listeleyen bir uyarı penceresi** çıkar ve onayınızı ister — böylece sessizce yanlış bir sorguyla karşılaşmazsınız. Onaylarsanız sorgu mümkün olduğunca tuvale çıkar (kısmi). Şemada bulunamayan bir tablo varsa (ör. sekmedeki veritabanı farklıysa) o tablo ve ona bağlı bağ/koşullar düşer ve size bildirilir. **Not:** Görsel Tasarımcı yalnız düz `SELECT + JOIN + WHERE`

içindir; karmaşık raporlama sorgularını (alt sorgu/gruplama) görsele çevirmek için tasarlanmamıştır.

Neden hiçbir şey otomatik çalışmıyor? Tuval bilerek "kör" tutuldu: auto-refresh, NOLOCK, Güvenli Yazma gibi çalıştırma yardımcıları burada yoktur. Amaç sorguyu *kurmaktır*; çalıştırma her zaman üretilen normal sorgu sekmesinde, tüm koruma raylarıyla birlikte olur.

19. Klavye Kısayolları

Kısayol	İşlevi
`F5` / `Ctrl+E`	Sorguyu çalıştır (metin seçiliyse yalnız seçimi)
`Ctrl+Enter`	İmleçteki ifadeyi çalıştır (SQL Server; diğer motorlarda seçim/tümü)
`Esc`	Çalışan sorguyu durdur
`Ctrl+Shift+F`	Biçimlendir (SQL Server: T-SQL · MongoDB: JSON)
`Ctrl+Space`	Otomatik tamamlama (2 karakter yazınca ve '.' sonrası kendiliğinden de açılır)
`Ctrl+F`	Editör içi arama paneli — **metinli butonlarla** (↑ Önceki · ↓ Sonraki · ✕ Kapat) ve Aa/Söz/.* seçenekleriyle. `Enter` / `F3` sonraki, `Shift+F3` önceki, `Esc` kapatır.
`Ctrl+H` *(editörde)*	**Bul & Değiştir** penceresi — Bul/Değiştir alanları, BÜYÜK/küçük · tam sözcük · regex, Sonrakini bul · Değiştir · Tümünü değiştir. *(Editör odakta değilken Ctrl+H sorgu geçmişi panelini açar — aşağı bkz.)*
`Ctrl+K`, `C` / `Ctrl+K`, `U`	Seçili satırları toplu **yorumla** / yorumu **kaldır** (SSMS deseni). `Ctrl+/` tek tuşta değiştirir (toggle).
`Ctrl+N` / `Ctrl+T`	Yeni sorgu sekmesi (**SQLST1, SQLST2…** adıyla; kaydedince dosya adını alır)
`Ctrl+O`	**Dosya Aç (YENİ):** .sql dosyasını kendi adıyla yeni sekmede açar (çoklu seçim olur; pencereye **sürükleyip bırakmak** da açar). Kodlama otomatik: BOM → UTF-8 → Türkçe ANSI (1254, uyarıyla) — Türkçe karakter bozulmaz.
`Ctrl+S`	Aktif sorguyu .sql dosyasına kaydet (sekme dosya adını alır; Görsel Sorgu'da üretilen script kaydedilir)
`Ctrl+W` / `Ctrl+F4`	Sekmeyi kapat
`Ctrl+Shift+T`	Son kapatılan sekmeyi geri aç
`Ctrl+P`	Her yere atla paleti
`Ctrl+H` *(editör dışında)*	Sorgu geçmişi panelini aç/kapat
`Ctrl+B`	Nesne gezginini gizle/göster
`Ctrl+R`	Sonuç bölgesini gizle/göster
`F1`	Kısayol kılavuzu penceresi

20. Veri Konumları ve İpuçları

Ne	Nerede
Bağlantı profilleri (motor bilgisiyle)	`%APPDATA%\SQLST\profiles.json` (parolalar DPAPI şifreli)
Geçmiş / oturum / tarihçe / ayarlar *(profil bazlı)*	`%APPDATA%\SQLST\sqlst.db` (SQLite)
Uygulama logları	`%APPDATA%\SQLST\logs\`
Kurulum	`%LOCALAPPDATA%\Programs\SQLST`

İpuçları: Uzun süren bir işi başlatıp başka sekmeye geçin — bitince bildirim sizi bulur. · Prod profillerinizi **salt-okunur** işaretleyin; yazmanız gerektiğinde ayrı bir profil kullanın — bu koruma artık beş motorda da çalışır. · Aynı sunucu için farklı işlere farklı profiller açın: geçmiş, sekmeler ve ayarlar ayrı kalır. · SQL Server'da Güvenli Yazma Modu'nu alışkanlık edinin: UPDATE'in kaç satır etkilediğini **görmeden** commit etmezsiniz. · Deploy öncesi şema snapshot'ı alın; sonrası ile karşılaştırıp migration'ı kanıtlayın.

21. Karşılaştırma 🔀 SQL ailesi

Üst şeritteki ⇄ Karşılaştır düğmesi **iki veritabanını** karşılaştıran bir sekme açar. **Kaynak (sol)** o an bağlı olduğunuz profildir; **Hedef (sağ)** için bağlantı ekranındaki gibi **sunucu · kimlik türü · (SQL kimliğinde) kullanıcı/parola** girip DB'leri getir ile o sunucunun veritabanlarından birini seçersiniz. Hedef **kaynakla aynı motor** olmalıdır (üstte "aynı motor: …" yazar); motorlar arası kıyas kapsam dışıdır. İki alt sekme vardır:

Şema karşılaştırma

🔀 Karşılaştır iki veritabanının şemasını okuyup farkları tek gride döker:

- **Nesne varlığı** — yalnız kaynakta / yalnız hedefte olan tablo, view, SP, fonksiyon.
- **Ortak tablolarda** kolon farkı (yalnız solda / sağda / **tip·null·PK değişti**: `int NOT NULL → bigint NULL`), **yabancı anahtar** farkı ve **index** farkı. Index'ler **yapısal olarak** kıyaslanır (benzersizlik + kolonlar); index adları ortamlar arası değişse bile yapısal fark yakalanır.

Test↔prod arasındaki şema kaymasını (eksik kolon, kaçmış index…) tek bakışta görürsünüz.

Veri karşılaştırma

Veri alt sekmesinde kaynak veritabanının bir tablosunu seçer, isterseniz bir **WHERE** (iki tarafta aynen uygulanır) yazar ve 🔀 Veri karşılaştır dersiniz. Sonuç satır bazındadır:

- **◀ yalnız solda** · **▶ yalnız sağda** · **≠ farklı**.
- **≠ satırlarda hangi kolon değişti** gösterilir: `Tutar: 100 → 150 · Sehir: Ankara → İzmir` (kaynak → hedef). İlk 200 farklı satır için detay çekilir.
- Fark listesi ⬇ CSV ile dışa aktarılır.

> **Nasıl çalışır — büyük tabloyu nasıl kaldırır?** Tüm satırları çekip karşılaştırmak yerine, her satırın **parmak izi sunucuda** hesaplanır (MSSQL `BINARY_CHECKSUM`, PostgreSQL `md5(satır)`, MySQL/Oracle kolon birleşimi) ve yalnızca **(anahtar, parmak izi)** çiftleri taşınır. Böylece milyonlarca satırlı tablolar da kıyaslanır; ağır olan tam-satır çekimi yalnız **farklı** bulunan satırlar için yapılır.

> **Sınırlar:** Anahtar olarak **birincil anahtar (PK)** kullanılır — PK'sız tablo satır bazında kıyaslanamaz. Parmak izi çok nadir **çakışabilir** (aynı imza, farklı satır) ve MSSQL'de XML/CLR tipli kolon içeren tabloda sunucu hata verebilir. **MySQL ve Oracle veri kıyası henüz gerçek sunucuda denenmedi** (yol haritasında kayıtlı). Sağ taraf yalnız aynı motorla çalışır.

22. Oluşturma Yardımcıları

Tablo oluşturma sihirbazı SQL ailesi

Nesne gezgininde **"Tablolar" grubuna sağ tıklayıp** ➕ Tablo oluştur... deyin. Sihirbazın üstünde bir **kip** seçersiniz: 🆕 **Yeni tablo (CREATE)** ya da ➕ **Var olan tabloya sütun ekle (ALTER)**. **Şema** ve **tablo adı**nı yazar, kolonları satır satır eklersiniz: her kolon için **ad · tip · NULL · PK · IDENTITY · UNIQUE · varsayılan · CHECK ifadesi · FK → tablo · FK → kolon**. ▶ Script üret seçili motorun sözdizimiyle `CREATE TABLE` (ya da ALTER kipinde `ALTER TABLE … ADD`) üretir ve **yeni bir sorgu sekmesinde açar** — hiçbir şey kendiliğinden çalışmaz; script'i görür, gerekiyorsa düzenler, `F5` ile çalıştırırsınız.

- **Kısıtlar: PK** (bileşik olabilir), **UNIQUE**, **CHECK** (ifade, ör. `Yas >= 0`) ve **FK** (hedef tablo + kolon; hedef `şema.tablo` olabilir) dört motorda ANSI `CONSTRAINT` olarak üretilir.
- **Motora göre:** otomatik artan kolon MSSQL'de `IDENTITY(1,1)`, PostgreSQL/Oracle'da `GENERATED BY DEFAULT AS IDENTITY`, MySQL'de `AUTO_INCREMENT`. ALTER kipinde sütun ekleme de motora göredir: MSSQL `ADD …`, PostgreSQL/MySQL `ADD COLUMN …`, Oracle `ADD (…)`. İfade sonu (Oracle'da ';' yok) ve tırnaklama motora göredir.
- **Doğrulama:** boş ad/kolon, tekrar eden kolon adı, PK'nın NULL olması, birden çok IDENTITY ve eksik FK hedefi engellenir. **ALTER kipinde PK/IDENTITY eklenemez** (mevcut satır riski) — net hata verilir. Hata satırda gösterilir.

> **Sınırlar:** MongoDB'de (koleksiyon şemasızdır) ve **salt-okunur** profillerde menü görünmez. MySQL/Oracle CREATE/ALTER çıktısı ve FK/UNIQUE/CHECK DDL'i **henüz gerçek sunucuda denenmedi** (birim düzeyinde doğrulandı) — ilk canlı kullanımda ince ayar gerekebilir.

Mongo find yardımcısı 🌿 MongoDB

MongoDB'ye bağlıyken editör araç çubuğundaki 🌿 Find yardımcısı (0.14.0'dan itibaren **Biçimlendir'in yanında**) Compass'takine benzer alanlar açar: **Koleksiyon · Filter · Sort · Project · Limit · Skip**. Filter/Sort/Project alanlarına birer JSON nesnesi (`{ "aktif": true }`, `{ "ad": 1 }` …) yazarsınız; ▶ Üret bunları SQLST'nin çalıştırdığı `find` belgesine derler (`{ "find": "…", "filter": …, "sort": …, "limit": … }`) ve yeni sekmede açar. Böylece sort/filter eklemek için ham JSON'u elle düzenlemeniz gerekmez.

> **İki yönlü:** Yardımcı artık **geri de çalışır** — aktif sekmedeki metin bir `find` belgesiyse yardımcı ondan **ön-doldurulur**, yani var olan bir sorguyu görsel olarak düzenleyip yeniden üretebilirsiniz. Geçersiz bir JSON alanı girerseniz hangi alanın bozuk olduğu söylenir.

> **Canlı düzeltme (0.14.0):** Kutular süslü parantez istemez — `"aktif": true` yazıp çıkın, kutu kendiliğinden `{ "aktif": true }` biçimli hâline döner (Sort'taki görünüm her kutuda). Bozuk

> JSON yazdıysanız (ör. fazladan `:`) hata, **Üret'e basmayı beklemeden** hangi kutuda olduğu söylenerek altta belirir.

> **Alan önerisi düzeltildi** 0.24.0**:** Filter/Sort/Project kutularında **koleksiyonun alanları** önerilmiyordu; yalnız alandan *sonrası* geliyordu (Sort'ta `1/-1`, Project'te `1/0`, Filter'da `$` operatörleri). Sebebi: MongoDB'de koleksiyon **listesi** hız için **alansız** çekilir (her koleksiyonu örneklemek listeyi çok yavaşlatıyordu), yardımcı da alan sözlüğünü yalnız o listeden kuruyordu. Artık **koleksiyonu seçtiğinizde** o koleksiyonun alanları getirilir ve kutunun yanında **"12 alan"** yazar. Aynı koleksiyon bir kez sorgulanır, sonrası önbellekten gelir; ad yarım yazılırken sunucuya gidilmez.

🔗 $lookup sihirbazı MongoDB · 0.22.0

Mongo'nun JOIN karşılığı olan `$lookup` pipeline'ını elle yazmak zahmetlidir. Editör araç çubuğundaki $lookup düğmesi bunu seçimlerle üretir: **ana koleksiyon**, **bağlanacak koleksiyon**, **yerel alan** ve **hedef alan** (listeler alan envanterinden gelir; `…Id` ile biten alan ve `_id` otomatik seçilir), bir de **sonuç alanı** adı. Sağ tarafta pipeline **canlı** oluşur; ▶ Sekmede aç çalıştırılabilir hâlde açar.

- **Tek eşleşme olarak düzleştir** (`$unwind`): dizi yerine tek belge — SQL'deki JOIN'e en yakın hâl.
- **Eşleşmeyen belgeler de gelsin** (LEFT JOIN karşılığı, `preserveNullAndEmptyArrays`): yalnız düzleştirme açıkken anlamlıdır, kapalıyken seçenek de kapanır.
- **Önizleme satırı ($limit)** 0.23.0**:** pipeline'ın **sonuna** `$limit` eklenir (varsayılan **200**) — sihirbaz bir *keşif* aracıdır, sorgu ilk N eşleşmede durur. Tamamını istiyorsanız kutuya **0** yazın. Sınır bilerek sona konur: başa konsaydı yalnız ilk N *ana* belge birleşir, eşleşmeler sessizce kaybolurdu.
- **Index uyarısı** 0.23.0**:** hedef alan index'siz ise sihirbaz uyarır ve çalıştırılacak komutu verir (`db.X.createIndex({ alan: 1 })`). Sebebi ölçüldü: index'siz hedef alanda Mongo **her ana belge için** hedef koleksiyonu baştan tarar — 50 bin × 10 bin belgede limitsiz sorgu 60 sn'de iptal oldu; **limit + index ile 143 ms**.

🔎 "Bu id nerede geçiyor?" MongoDB · 0.22.0

MongoDB'de yabancı anahtar yoktur; bir `ObjectId`'nin hangi koleksiyonlarda kullanıldığını bulmak elle koleksiyon koleksiyon aramak demektir. Sonuç grid'inde bir hücreye sağ tık → 🔎 Bu id nerede geçiyor? deyin: SQLST, **değerin tipine uyan** tüm koleksiyon alanlarını tarar (ObjectId ararken ObjectId alanları, metin ararken metin alanları) ve **hangi koleksiyonda kaç belge** bulunduğunu listeler.

- Sonuçlar **tarama sürerken akar** — bitmesini beklemeden görürsünüz; ■ Durdur her an keser.
- Bir satıra **çift tık** → o koleksiyonun ilgili `find` sorgusu yeni sekmede açılır.

- Çok geniş şemalarda tarama **250 aday alanla** sınırlanır (sorgu yağmuru olmasın); sınıra değilse özet bunu açıkça söyler. Bu, SQL tarafındaki 🕸 **Kayıt Haritası**'nın Mongo karşılığıdır.
- **Hız** `0.23.0`**:** adaylar **6'lı gruplar hâlinde eşzamanlı** taranır (ölçüm: 8 aday sırayla 4.260 ms → paralel 654 ms) ve **index'li alanlar öne alınır** (aynı sayım index'liyken 2 ms, index'sizken 493 ms) — cevap çoğu kez anında görünür. Index'siz kısma geçildiğinde ilerleme satırı bunu yazar; ■ **Durdur artık ANINDA** sonlandırır (sunucudan yanıt beklenmez) ve o ana kadar bulunanlar ekranda kalır.
- **Pencere anında açılır** `0.24.0`**:** eskiden menüye bastıktan sonra pencere **açılmadan önce** veritabanının tüm koleksiyon envanteri bekleniyordu — büyük veritabanında saniyelerce ekranda hiçbir şey olmuyor, özellik bozuk sanılıyordu. Artık pencere ilk karede açılır, envanteri kendi içinde okur ("Koleksiyon envanteri okunuyor…") ve ■ Durdur **bu evrede de** keser. Envanter okunamazsa nedeni ana penceredeki durum çubuğuna değil, doğrudan bu ekrana yazılır.

📋 Shell komutu yapıştır `MongoDB · 0.22.0`

Compass'tan, bir meslektaşınızdan ya da internetten gelen `db.oturumlar.find({ kullaniciId: ObjectId("…") }).sort({ ts: -1 }).limit(10)` gibi bir **shell komutunu** panoya kopyalayıp editör araç çubuğundaki Shell yapıştır düğmesine basın. REST İstemcisi'ndeki "cURL yapıştır"ın Mongo karşılığıdır.

> **Artık bir ekran açılıyor** `0.24.0`**:** 0.23.0'e kadar düğme panoyu *sessizce* okurdu — çevirebilirse yeni sekme açar, çeviremezse yalnız alttaki durum çubuğuna yazardı; yani pano beklenen biçimde değilse ekranda hiçbir şey olmuyordu. Artık pencere açılır: **pano ön-dolu** ve tümü seçili gelir (`Ctrl+V` ile doğrudan üzerine yapıştırabilirsiniz), metni **düzenleyebilirsiniz**, çeviri **her tuşta yenilenir** ve varsa hata girdinin hemen altında görünür. Sonucu ▶ Sekmede aç ile sekmeye alır (çalıştırılmaz — `F5` sizde) ya da 📋 Kopyala ile alırsınız.

- **Compass'ın kendi çıktısı** `0.24.0`**:** Compass'ta "Copy" dediğinizde sorgu `db.getCollection('oturumlar').find(…)` biçiminde gelir — yani en sık yapıştıracağınız metin. Bu biçim eskiden *reddediliyordu*; artık çevriliyor (tek/çift tırnak, boşluklu yazım ve noktalı koleksiyon adı dahil).
- Çevrilenler: `find` · `findOne` · `aggregate` · `countDocuments` ve zincirdeki `.sort()` · `.limit()` · `.skip()` · `.projection()` (`.pretty()` / `.toArray()` sessizce atlanır).
- Shell'in "gevşek" yazımı da anlaşılır: **tırnaksız anahtarlar**, tek tırnaklı metinler, `ObjectId(…)`, `ISODate(…)`, `new Date(…)`, `NumberLong(…)` ve `/desen/i` düzenli ifadeleri doğru karşılıklarına dönüştürülür.
- Çevrilemeyen bir komutta (ör. `updateMany`) net bir mesaj verilir — sessizce yanlış sorgu üretilmez.

23. Veritabanı Haritası Tüm motorlar

Sol raydaki 🗺 Veritabanı Haritası düğmesi, seçili veritabanını bir **ER-diyagramı** olarak resmeder — modern, kart tabanlı bir görünüm (eski UML kutuları değil): her tablo bir **kart**, her **yabancı anahtar** yumuşak bir bağdır. Tamamen keşif/belgeleme içindir; **hiçbir şey çalıştırmaz**.

- **Kart:** şemaya göre renkli başlık şeridi + noktası; satırlarda 🔑 **PK** · 🔗 **FK** rozetleri, kolon adı ve tipi.
- **İlişkiler:** FK'ler kaynak karttan hedef karta **yön oklu** yumuşak eğriyle bağlanır; bileşik anahtar tek bağdır.
- **Keşif:** bir kartın **üstüne gelince** yalnız o kart ve doğrudan ilişkili tablolar aydınlanır, gerisi soluklaşır. Üstteki **arama kutusu** aynı odaklamayı adla yapar.
- **Yerleşim:** kartları başlıktan **sürükleyin**; ⚡ Otomatik yerleştir bağlı tabloları birbirine yaklaştırır; ⤢ Sığdır ızgaraya döner. **Ctrl+tekerlek** yakınlaştırır. **Konumlar veritabanı başına kalıcıdır** — dizdiğiniz gibi geri gelir (kayıt yalnız tam haritada yazılır; odak/küme görünümündeki taşımalar kalıcı düzeni bozmaz).
- **Görsel Sorgu'ya köprü:** kart **gövdesine tıklayıp** seçin (yeşil kenar + ✓), sonra 🎨 Görsele gönder ile seçili tabloları Görsel Sorgu tuvaline aktarın.
- **Dışa aktarma:** ↓ SVG (vektör — ölçeklenebilir/düzenlenebilir) ve ↓ PNG (görüntü). Belgeleme için idealdir.

Odak + Bağlam — büyük şemalarda gezinme 0.12.0

Büyük veritabanında "her şeyi tek tuvale çiz" yaklaşımı okunmaz olur. 0.12.0 ile harita **30'dan çok tablo** içeren şemalarda **Mahalle görünümüyle** açılır:

- **Mahalle kartları:** tablolar FK bağlantılarına göre **kümelere** ayrılır; her küme renkli bir kart olur — üstünde küme adı, **büyük üye sayısı** ve en bağlantılı 3 tablo. Kümeler arası ilişkiler tek kalın çizgi + **"N FK"** rozetiyle gösterilir.
- **Kümeye dalma:** mahalle kartına **çift tıklayın** — yalnız o kümenin tabloları tam kart olarak açılır. 🏘 Mahalle düğmesi geri döndürür.
- **Tabloya odaklanma:** herhangi bir tablo kartına **çift tıklayın** ya da arama kutusuna adı yazıp **Enter**'a basın — o tablo merkeze alınır, komşuları çevresine halka olarak dizilir. ◎ Derinlik: 1/2 komşuluğun kaç adım gösterileceğini seçer.
- **Kompakt kartlar:** görünür kart sayısı 30'u aşarsa kartlar yalnız başlıkla çizilir (kolon listesi gizlenir) — ekran hiç kalabalıklaşmaz. Küçük şemalarda davranış eskisi gibidir.

> **MongoDB:** Koleksiyonlar da kart olarak görünür — ama MongoDB'de yabancı anahtar yoktur, dolayısıyla **bağ çizilmez** ("ilişki yok"). Sürükleme, arama ve dışa aktarma yine çalışır; Görsel Sorgu köprüsü SQL ailesine özgüdür.

24. Aktivite / Denetim — "kim ne iş yapmış" Tüm motorlar

🛠 Yönetim Paneli içinde **Aktivite / Denetim** sekmesi, veritabanında "kim ne iş yapmış" sorusuna iki açıdan bakar:

- **Denetim izi** — bu araçtan (SQLST) çalıştırılan her sorgu: **zaman · kullanıcı · işlem türü** (Okuma/Ekleme/Güncelleme/Silme/Yapı/Yürütme) **· veritabanı · durum · süre · satır · SQL**. İşlem türü SQL'den çıkarılır — CTE'ye gizlenmiş DML bile doğru sınıflanır (`WITH (…) INSERT…` → Ekleme). Serbest metin **süzgeci** (kullanıcı/tür/db/SQL), **"Yalnız yazma"** ve ⤓ CSV dışa aktarma. **Dört motorda da** vardır (SQL Server · PostgreSQL · MySQL · Oracle · MongoDB). **0.24.0:** grid **sağa kaydırılabiliyor** (SQL kolonu artık sabit genişlikte — eskiden kalan yeri kapladığı için kaydırma çubuğu hiç çıkmıyordu, uzun SQL'in devamı görülemiyordu) ve bir satıra **çift tıklayınca** kaydın **tam SQL'i** ayrı pencerede açılıyor (sonuç grid'inin hücre penceresinin aynısı: kopyalanabilir, JSON'sa girintili; başlıkta hangi kayıt olduğu yazar).
- **Canlı aktivite** (SQL Server) — **şu an** kim bağlı, ne çalıştırıyor: oturum · giriş · makine · program · veritabanı · durum · çalışan SQL. (Diğer motorlarda karşılığı: MySQL PROCESSLIST, Oracle v$session, MongoDB "Çalışan işlemler" — kendi bölümlerinde.)

> **"Kim" nasıl bulunur:** Windows kimliğiyle bağlıysanız `ETKİALANI\kullanıcı`, SQL kimliğiyle bağlıysanız bağlantı kullanıcı adı yazılır. **Dürüst sınır:** denetim izi yalnız **bu araçtan** yapılanları kapsar. Başka uygulamalardan yapılan değişiklikleri ya da "geçmişte kim hangi satırı değiştirdi"yi görmek için veritabanının kendi denetimi (SQL Server Audit / CDC / trigger, PostgreSQL pgAudit, Oracle unified audit) kurulmuş olmalıdır — SQLST bunu sonradan uyduramaz.

25. AI Asistan 🤖 ÜCRETSİZ · Tüm motorlar

SQLST'nin yapay zeka yardımcısı (v11). **Tamamen ücretsiz** bir dil modeliyle çalışır ve bağlı olduğunuz veritabanının **şemasını bilerek** yardım eder: doğal dilden sorgu yazar, sorgunuzu denetler, hatanızı çözer, planınızı yorumlar, loglarınızı özetler.

Yapılandırma

AI asistanının sağlayıcı, model ve API anahtarı yapılandırması **kurulumla birlikte gelir**; uygulama içinden **görüntülenemez ve değiştirilemez** — arayüzde ayar düğmesi yoktur (0.16.2). Asistan düğmelerine bastığınızda "bu kurulumda yapılandırılmamış" uyarısı görüyorsanız kurulumu yapan kişiye başvurun. Desteklenen sağlayıcılar: 🏠 **Yerel (Ollama — internetsiz, varsayılan; aşağıya bakın)** ve OpenAI-uyumlu uçlar (kurumun kendi ucu, Groq/OpenRouter, LM Studio). 0.24.5 Google Gemini **kaldırıldı**.

> **Bulut ucu kullanıyorsanız — istek limiti:** art arda çok istek atarsanız sağlayıcı kısa bir süre "bekleyin" der — SQLST bunu gerekçesiyle birlikte gösterir; birkaç saniye sonra yeniden deneyin. (Yerel/Ollama'da limit yoktur.)

🏠 Yerel AI — internetsiz, veri makineden çıkmadan kapalı ağ

İnternete çıkışın olmadığı (ya da şema/sorgu bilgisinin dışarı gitmesinin istenmediği) ortamlar için asistan **tamamen yerelde**, **Ollama** ile çalışır: API anahtarı yok, internet yok, hiçbir veri makineden çıkmaz. SQLST, yereldeki Ollama'ya (`http://127.0.0.1:11434`) bağlanır; varsayılan model **Qwen2.5-Coder-7B**'dir (Apache-2.0 lisanslı, ücretsiz dağıtılabilir).

- **Kurulum — tek kurulumla her şey (kapalı ağ):** AI'lı SQLST kurulumunu çalıştırdığınızda **Ollama da model de aynı kurulumdan gelir** — ayrı bir şey indirip taşımanıza gerek yoktur. Kurulumda **"🏠 Yerel Yapay Zeka"** bileşeni seçili gelir; kurulum Ollama'yı sessizce (admin YOK) kurar ve gömülü modeli kaydeder (birkaç dakika sürer — model ~5 GB'tır). İnternet gerekmez. İstemeyen saha bu bileşenin işaretini kaldırıp yalnız uygulamayı kurabilir.
- **Model dosyası yanında yoksa (ör. GitHub'dan indirilen AI kurulumu):** AI'lı kurulum iki dosyadır — kurulum exe'si ve onunla **aynı klasörde** duran model dosyası (`.gguf`, 4,36 GB). Model dosyası yanında değilse kurulum bunu baştan söyler ve modeli kurulum sırasında **internetten indirir** (Ollama'nın resmi kütüphanesinden aynı model, ~4,7 GB; bağlantıya göre birkaç dakika ile yarım saat). Sonuç aynıdır: aynı model, SQLST'nin aynı ayarlarıyla. İnternet yoksa ya da şirket ağı indirmeyi engelliyorsa uygulama yine kurulur; yapay zeka sekmesi eksik adımı gösterir. Kapalı ağda internetsiz kurulum için model dosyasını kurulumun yanına koymak yeterlidir.
- **Hız beklentisi:** yerel model CPU'da çalışır — cevap **anlık değildir** (bir "AI Yorumla" birkaç on saniye sürebilir; ilk istek model belleğe yüklenirken en yavaşıdır). Makinede **boş RAM ne**

kadar çoksa o kadar hızlıdır (6 GB+ boş idealdir). 0.24.4 Model artık sorular arasında **30 dakika bellekte kalıyor** — eskiden 5 dakikada düşüyor ve sonraki soru yeniden yüklenmeyi bekliyordu (ölçüm: soğuk yükleme **108 sn** ↔ sıcak aynı soru **24 sn**). Yani arka arkaya sorulan sorularda yalnız ilki yavaştır. Ayrıca cevap uzunluğuna bir tavan kondu: model kendini tekrar edip dakikalarca yazmaya başlarsa üretim kesilir. Bulut sağlayıcı yoğunsa (503) SQLST kendiliğinden 3 kez dener — 0.24.4. 0.24.6 **Asistan sekmesini açtığınız anda hazırlık başlar.** SQLST, sorularınızın ortak başlangıcını (yönergeler + tablo listesi) modele önceden okutur; siz sorunuzu yazarken bu iş arka planda biter. Kazanç büyük: model bu başlangıcı ikinci kez okumuyor.

	Önce	Şimdi
"merhaba"	90–134 sn	**~1 sn**
Veri sorusu	86–128 sn	**~1 sn**

Bedeli: sekmeyi ilk açtığınızda hazırlık ~90 saniye CPU kullanır — arka planda, siz yazarken. Sekmeyi kapatıp yeniden açarsanız hazırlık zaten hazır olduğu için anında geçer. Sekmeyi açar açmaz soru sorarsanız hazırlığın kalanını beklersiniz.

- ⏱ **"Cevap vermiyor mu, çalışıyor mu?" — artık belli:** 0.24.5 bekleme balonunda **geçen süre sayacı** döner ("düşünüyor… 14 sn", uzun beklemede "1 dk 05 sn"). Eskiden ekranda sabit bir yazı durduğu için çalışan istekle takılmış istek ayırt edilemiyordu. Sayaç **tüm bekleme akışını** ölçer: şema okuma + istem hazırlama + model + üretilen SQL'in düzeltme turu. Sayaç ilerliyorsa iş sürüyor demektir — beklemek istemezseniz **İptal** her an çalışır.
- **Düşük bellekli makine:** RAM sıkışıksa yapılandırmadan daha küçük bir modele (ör. `qwen2.5-coder:1.5b`) geçilebilir — daha hızlı, biraz daha az isabetli.
- 🛡 **Güvenlik ağı (üretilen SQL denetimi):** asistanın ürettiği SQL, size verilmeden önce SQLST'nin **kendi çözümleyicisiyle** sınanır (SQL Server'da T-SQL parser, MongoDB'de JSON geçerliliği). Söz dizimi tutmuyorsa asistan **tek bir düzeltme turu** dener; yine tutmuyorsa "geçerli SQL üretilemedi — dikkatle inceleyin" diye **açıkça uyarılırsınız**. Küçük yerel modellerin ara sıra yaptığı hataları (ör. T-SQL'de `LIMIT` ya da yanlış konumda `TOP`) bu kapı yakalar — "yaklaşık/yanlış SQL" sessizce sunulmaz. *(Diğer motorlarda — PostgreSQL/MySQL/Oracle— o motorun çözümleyicisi elimizde olmadığından denetim yapılmaz; bu dürüstçe böyle bırakılır, sahte "geçerli" damgası vurulmaz.)*

> **Ollama'ya ulaşılamıyor mu?** Asistan size adım adım söyler: (1) Ollama kurulu mu, (2) çalışıyor mu (Başlat'tan açın ya da `ollama serve`), (3) model çekili mi (`ollama pull qwen2.5-coder:7b`).

Bu nasıl oluyor? — sade anlatım

Teknik terimlere boğulmadan, gündelik dille anlatalım. "Yapay zeka makinede çalışıyor" derken tam olarak ne oluyor:

- **Yapay zeka aslında dev bir dosyadır.** "Model" dediğimiz şey (bizde **Qwen2.5-Coder**), milyonlarca örnek koddan ve metinden damıtılmış, yaklaşık **5 GB'lık tek bir dosyadır**. İçinde "şunu sorarsan böyle cevap ver" kalıpları vardır. Kendi başına bir program değildir; onu **çalıştıracak bir motora** ihtiyaç duyar — tıpkı bir film dosyasının bir oynatıcıya ihtiyaç duyması gibi.
- **Ollama, bu modeli çalıştıran motordur.** Makinenize sessizce kurulan küçük bir programdır. İşi şudur: o dev model dosyasını belleğe alır, sizin sorunuzu ona verir, cevabı üretmesi için işlemciyi (CPU) çalıştırır ve sonucu geri getirir. Veritabanları için SQL Server ne ise, yapay zeka modelleri için Ollama odur — **modeli koşturan altyapı**.
- **SQLST ile Ollama nasıl konuşur?** Ollama, makinenizin **içinde**, kendi kendine dinleyen bir "kapı" açar (adresi `127.0.0.1:11434` — `127.0.0.1` "bu makine, sadece kendisi" demektir). SQLST sorunuzu bu iç kapıya bırakır, cevabı oradan alır. **Bu trafik makinenin dışına hiç çıkmaz** — ne internete, ne başka bir bilgisayara. Kablo çekilmiş olsa bile çalışır.
- **"Veri dışarı çıkmıyor" ne demek, neden önemli?** Bulut tabanlı yapay zekalarda (ChatGPT gibi) sorunuz ve verdiğiniz bilgiler internet üzerinden başka bir şirketin sunucusuna gider. Kapalı ağda çalışan kurumlar için bu **kabul edilemez**. Bizim çözümümüzde model sizin makinenizde olduğundan, sorunuz da şemanız da **o odadan hiç çıkmaz** — kağıda not almak gibi, size özel kalır.
- **Peki güvenilir mi? İşte kritik nokta.** Yerel modeller, buluttaki devasa modellere göre daha küçüktür; ara sıra **hatalı SQL** yazabilirler (yanlış bir anahtar sözcük, yanlış yere konmuş bir ifade gibi). Bizim ayrıcalığımız şu: SQLST zaten **gerçek bir SQL denetçisi** taşır (sorguları anlayan, ayrıştıran bir bileşen). Yapay zekanın yazdığı her sorguyu, size göstermeden önce bu denetçiden geçiririz. Sözdizimi bozuksa, ya modele "şunu düzelt" deyip **bir tur daha** denettirir, ya da düzelmezse sizi **açıkça uyarır**: "bu sorgu şüpheli, çalıştırmadan bakın". Yani yapay zeka yanılsa bile, **yanlış cevabı sessizce doğruymuş gibi elinize tutuşturmayız.**
- **Neden anlık değil?** Bulut yapay zekaları binlerce güçlü kartla (GPU) çalışır. Sizin makinenizde bu yoktur; model normal işlemcide (CPU) düşünür. Bu yüzden bir cevap **birkaç on saniye** sürebilir — özellikle günün ilk sorusunda, model belleğe ilk kez yüklenirken. Sabırlı olun; karşılığında verileriniz güvende ve internet faturanız yok. Makinede **boş bellek ne kadar çoksa cevaplar o kadar hızlanır.**

> **Tek cümlede:** Kendi yapay zekamız, evinizde çalışan bir yardımcı gibidir — dışarıdan kimse duymaz, sorularınız size özel kalır; biraz yavaş düşünür ama söylediği her SQL, size verilmeden önce bizim denetçimizden geçer.

Özellikler

Nerede	Düğme	Ne yapar
Sol ray	AI Asistan	**Sohbet sekmesi:** doğal dille soru sorun ("en çok siparişi olan 10 müşteriyi getir"). Cevaptaki sorgu ↗ Sekmede aç ile yeni sekmeye gelir — **asla kendiliğinden çalışmaz**, siz inceleyip F5 dersiniz. "Editördeki sorguyu ekle" işaretliyse üzerinde çalıştığınız sorgu da bağlama girer. Soruyu gönderince cevabın çıkacağı yerde AI logolu bir **"düşünüyor..."** balonu belirir (v0.19.4); cevap gelince onun yerini alır.
Sorgu sekmesi	AI Değerlendir	Editördeki sorguyu şemayla denetler: söz dizimi, mantık (yanlış kolon, eksik JOIN), risk (WHERE'siz yazma, kartesyen), iyileştirme + gerekirse düzeltilmiş sorgu.
Sorgu sekmesi	AI Açıkla	Devraldığınız sorgunun ne yaptığını adım adım düz Türkçe anlatır.
Sorgu sekmesi	AI Hatayı çözdür	Son çalıştırmanın kırmızı hatasını götürür: neden + düzeltilmiş sorgunun tamamı.
Plan sekmesi	AI Planı yorumlat	Alttaki execution plan'ı otomatik alır: en pahalı adımlar, tahmin/gerçek sapması, hangi index yardım eder.
Log Analizi	AI Özetle	Ekrandaki en çok tekrarlayan hata gruplarına muhtemel kök neden + pratik sonraki adım.
Profiler	AI Yorumla	Seçili olayı/grubu/kilitlenmeyi yorumlar: muhtemel neden + somut öneri (index/sorgu); kilitlenmede modele ham XML değil çözülmüş özet gider.

Gizlilik kuralları

- Modele giden şema özeti yalnız **tablo/kolon adları ve tipleridir** — **satır verisi, bağlantı bilgisi, sunucu adı asla gitmez**. Bunun dışında yalnız sizin yazdıklarınız (soru/sorgu/hata metni) ve Log Analizi'nde zaten ekranda olan gruplar gider.
- API anahtarı bu makinede **Windows DPAPI ile şifreli** saklanır; düz metin görünmez ve uygulama arayüzünde hiçbir yerde gösterilmez.
- Üretilen hiçbir sorgu kendiliğinden çalıştırılmaz — her zaman önce sekmede görürsünüz.

26. Paket Aktarım 📦 Tüm motorlar

Sol raydaki 📦 Paket Aktarım, SSIS mantığında veri taşıma penceresi açar: **kaynak** = bağlı olduğunuz sunucu, **hedef** = kayıtlı profillerinizden seçtiğiniz sunucu (elle bağlantı bilgisi girilmez; hedef listede yoksa önce Bağlantı ekranından profil kaydedin). Tipik kullanımlar: tabloyu başka ortama kopyalama, rapor sorgusunun sonucunu kalıcı tabloya alma, Mongo log koleksiyonunu arşiv veritabanına taşıma.

SQL ailesi: Tablo→Tablo

1. Kaynak veritabanını ve tabloyu seçin; hedef profil → veritabanı → tabloyu seçin (sistem veritabanları ve view'lar hedef listesine girmez).
2. **Eşleme gridi** hedef kolon başına kurulur: aynı adlı kolonlar **otomatik eşlenir**, karşılıksız kolon boş kalır (**boş = aktarılmaz**). Birincil anahtar kolonları 🔑 ile işaretlidir. Son karar her zaman sizindir — istediğiniz satırın kaynağını değiştirin ya da boşaltın.
3. 📦 Aktar önce **özet onayı** gösterir; onayla birlikte 500'lük partiler hâlinde aktarım başlar (ilerleme sayaçları canlı, ■ Durdur her an geçerli — o ana dek tamamlanan partiler kalır, sayaçlar dürüsttür).

Motorlar-arası serbesttir (SQL ailesi içinde): SQL Server'dan PostgreSQL'e, MySQL'den SQLite'a… Değerler parametreli INSERT ile taşınır; tip dönüşümünü sürücü yapar, dönüşemeyen satır hata politikanıza düşer.

SQL ailesi: Sorgu→Tablo

Kaynak grubundaki **Sorgu** sekmesine geçin, serbest SELECT'inizi yazın ve Kolonları getir deyin — sorgu **satır çekilmeden** incelenir (yalnız şema) ve sonuç kolonları eşleme gridine gelir. JOIN'li, hesaplamalı, takma adlı her sorgu olur; aktarımda sorgunuz aynen çalıştırılır ve sonucu hedef tabloya yazılır.

Yazma kipleri ve hata politikası

- **Yalnız ekle** (varsayılan): her kaynak satırı INSERT edilir.
- **Ekle/Güncelle (🔑 anahtara göre)**: hedef tablonun birincil anahtarına göre var olan satır **güncellenir**, olmayan **eklenir** (UPSERT). Tüm anahtar kolonların eşlenmiş olması ve anahtar dışında en az bir kolon kalması gerekir — eksikse pencere başlatmadan uyarır.
- **Önce hedefi TEMİZLE**: aktarımdan önce hedef tablodaki tüm satırlar silinir (DELETE). İşaretleseniz bile başlatırken **ayrıca ve açıkça** onaylatılır.
- **Hata olursa**: **İlk hatada dur** (aktif parti geri alınır, o ana dek işlenen kalır) ya da **Hatalı satırı atla ve raporla** — hatalı satırların ayrıntısı toplanır (10.000'e kadar; ekrandaki raporda ilk 20'si görünür).

Bitişte durum bandının altına **kopyalanabilir rapor** gelir: okunan · eklenen · güncellenen · atlanan · süre · yaklaşık hız + hata örnekleri. Hata atlandıysa raporun yanında `⬇ Hata raporunu kaydet` düğmesi çıkar: atlanan satırların **tam listesini** (satır no + neden) metin dosyasına yazar. Ayrıca her aktarımın özeti ve hata ayrıntıları (ilk 200) **uygulama günlüğüne** de yazılır — raporun son satırı günlük dosyasının tam yolunu söyler (`%APPDATA%\SQLST\logs\sqlst-TARİH.log`). Bunların hepsi Mongo penceresi için de geçerlidir.

MongoDB: koleksiyon→koleksiyon arşivleme 🍃

MongoDB'ye bağlıyken aynı düğme Mongo penceresini açar — tipik senaryo **log koleksiyonunu arşiv veritabanına taşımaktır**:

- **Süzgeç (find JSON):** yalnız eşleşen belgeler taşınır — ör. `{ "Seviye": "Hata" }` ya da tarih aralığı. Boş bırakılırsa tüm belgeler.
- **Alan eşleme gridi:** koleksiyonu seçince alanlar (örneklemden çıkarılan envanter) listelenir; hedef adı **yazılabilir** — yeniden adlandırın ya da boşaltıp atlayın. **Grid'e hiç dokunmazsanız belgeler OLDUĞU GİBİ kopyalanır** (örneklemde görünmeyen nadir alanlar dahil — güvenli varsayılan). Bir şey değiştirirseniz yalnız listedeki alanlar taşınır; özet onayı bunu açıkça hatırlatır.
- **Hedef veritabanı/koleksiyon adı yeni olabilir:** listeden seçmek yerine adı yazın — Mongo ilk yazışta kendisi oluşturur (arşiv DB'si önceden var olmak zorunda değil).
- **Ekle/Güncelle** burada **_id'ye göredir**: aynı _id'li belge hedefte varsa değiştirilir, yoksa eklenir — arşivlemeyi tekrar tekrar koşmak güvenlidir.

> **Bilinen sınırlar:** SQL↔Mongo **çapraz** aktarım yoktur (aile içinde kalır). MongoDB'de parti geri alma yoktur (bağımsız sunucu transaction desteklemez) — Durdur'a bastığınızda o ana dek yazılan belgeler hedefte kalır; rapor bunu açıkça söyler. SQL tarafında hedef tabloda tetikleyici varsa satır satır INSERT/UPDATE bunları çalıştırır — büyük aktarım öncesi göz önünde bulundurun.

27. Excel/TXT İçe Aktarma 📥 SQL ailesi

Sol raydaki 📥 İçe Aktar, bir Excel (.xlsx/.xls) ya da TXT/CSV dosyasını okuyup **bağlı olduğunuz sunucudaki** bir tabloya aktarır. Tipik kullanımlar: müşteriden gelen Excel listesini tabloya yükleme, başka sistemden dışa aktarılmış CSV'yi içeri alma, elle tutulmuş bir listeyi kalıcı tabloya çevirme.

Adım adım

1. **Gözat** ile dosyayı seçin — **Tür** uzantıdan kendiliğinden seçilir, gerekirse değiştirin (0.15.1 kuralları): **Excel**'de ek ayar yoktur, her zaman **ilk sayfa** okunur (hücre tipleri dosyadan gelir — tahmin daha isabetlidir); **CSV**'de kolon ayracı **OTOMATİK algılanır** (TR Excel'in `;` CSV'si de standart `,` de sorusuz açılır); **TXT**'de **kolon ayracını** (`;` `,` Tab `|` ya da tek karakter yazın) ve **satır ayracını** ("Otomatik" = satır sonları; kayıtlar tek satırda özel karakterle ayrılıyorsa onu seçin/yazın) siz seçersiniz. CSV/TXT'de kodlama (UTF-8 / Windows-1254; Türkçe karakter bozuksa 1254'ü deneyin) da buradadır. Tırnaklı alanlar desteklenir: `"İstanbul; Kadıköy"` içindeki ayraç kolonu bölmez.
2. **Biçim** kültürünü doğrulayın: sayı ve tarih yorumu buna göredir — Türkçe'de `1.250,75` ve `24.07.2026`, Uluslararası'da `1,250.75` ve `2026-07-24`. **İlk satır başlık** işaretliyse kolon adları ilk satırdan alınır.
3. 🔍 Önizle: ilk 200 satır tablo görünümünde gelir; her kolonun başlığında **tahmin edilen tip** yazar (tam sayı · ondalık · tarih · metin(N) — örneklemden). Dosyada devamı varsa alt bant söyler; **aktarımda dosyanın tamamı** gider.
4. **Hedef** veritabanını ve tabloyu seçin — eşleme gridi hedef kolon başına kurulur, dosya kolonlarıyla **aynı adlar otomatik eşlenir** (🔑 = birincil anahtar; boş = aktarılmaz).
5. 📥 Aktar: özet onayından sonra 500'lük partilerle yazılır — ilerleme sayaçları, ■ Durdur ve bitişte **kopyalanabilir rapor** (okunan/eklenen/güncellenen/atlanan + hız + ilk 20 hata örneği) v12 Paket Aktarım'la birebirdir.

Yeni tablo (CREATE) yolu

Hedef tablo henüz yoksa combodan seçmek yerine **adını yazın** (`Satislar` ya da `raporlar.Satislar` ; SQL Server'da şemasız ad `dbo` 'ya gider). Eşleme gridi dosya kolonlarına döner ve her kolona **önerilen SQL tipi** atanır; boşalttığınız kolon tabloya girmez. Aktar'da üretilen `CREATE TABLE` script'i **onayınıza sunulur** — "Oluştur ve aktar" demeden hiçbir DDL çalışmaz. Kolonlar bilerek hep NULL izinli üretilir (200 satırlık örneklem dosyanın devamını bilemez) ve birincil anahtar üretilmez — bu yüzden yeni tabloda yazma kipi "Yalnız ekle"dir.

Yazma kipleri ve dönüşüm kuralları

- **Yalnız ekle** / **Ekle-Güncelle (🔑)** / **önce temizle** / **hata politikası**: Bölüm 26'daki Paket Aktarım kurallarıyla aynıdır.

- **TXT değerleri hedefe gitmeden tipe çevrilir:** `"1.250,75"` seçili kültüre göre 1250,75 olur; boş alan **NULL** yazılır. Çevrilemeyen değer (ör. sayı kolonunda `abc`) satır hatasıdır — politikanıza göre ya durur ya atlanıp raporlanır; mesajda **satır numarası + kolon + ham değer** vardır, dosyada kolayca bulursunuz.

> **Bilinen sınırlar:** hedef yalnız SQL ailesidir (Mongo'ya dosya aktarımı yok); Excel'de yalnız değerler okunur (formüller sonuç değerleriyle gelir); önizleme tip tahmini ilk 200 satırdan yapılır — dosyanın ilerisinde farklı biçimli veri varsa satır hatası olarak politikaya düşer.

28. SOAP/WCF Test İstemcisi 🔌 Tüm motorlar

Sol raydaki 🔌 SOAP İstemcisi, bir WCF/SOAP servisini SQLST içinden test etmenizi sağlar — tipik senaryo: DB'deki SP'yi SQLST'de değiştirdiniz, o SP'yi saran servis hâlâ doğru dönüyor mu? Ayrı bir araca (SoapUI vb.) gerek kalmadan aynı pencerede doğrularsınız. Hedef, SP saran WCF servislerinin standardı olan `basicHttpBinding` + document/literal'dir.

Adım adım

1. **WSDL** adresini yazın (örn. `http://sunucu/Servis.svc?wsdl`) → ⇩ Yükle. WCF'nin çok parçalı WSDL yayını otomatik izlenir (önce `?singleWsdl` denenir); operasyonlar solda listelenir. **Servis 401 (kimlik) istiyorsa** (ör. bazı kamu servisleri) üstteki **Kullanıcı adı** + **Parola** alanlarını doldurun — HTTP Basic auth hem WSDL indirmede hem çağrılarda kullanılır. Ortam profiliyle kaydederseniz parola bu makinede DPAPI ile şifreli saklanır.
2. Operasyonu seçin — **Action** ve **örnek SOAP zarfı** hazır dolar: parametreler tip bazlı örnek değerlerle gelir (int→0, tarih→2026-01-01…), liste alanlarında "tekrarlayın" yorumu bulunur, iç DataContract tipleri doğru namespace'le nitelenir. Durum bandında parametre özeti (ad · tip · seçimlik/dizi) görünür.
3. Zarfı düzenleyin ve ▶ Gönder (zarf kutusunda `F5` / `Ctrl+Enter`). Zarf ve yanıt **XML sözdizimi vurgulu** (satır numaralı; renkli etiket/öznitelik/yorum) gösterilir; yanıt girintilenir, durum bandında HTTP kodu ve süre yazar. **Fault** dönerse açıkça işaretlenir — hata ayrıntısı yanıt gövdesindedir. **Adres** kutusu WSDL'den gelir ama elle değiştirilebilir: aynı zarfı test/canlı uca yöneltebilirsiniz.

Geçmiş, ortamlar ve AI

- **İstek geçmişi (kalıcı):** her gönderim sol alttaki listeye düşer — saat, ✓/⚠/✕ durumu, operasyon, süre. **Çift tık** isteği (adres + action + zarf) geri yükler; düzenleyip yeniden gönderebilirsiniz. Son 200 istek saklanır.
- **Ortam profilleri:** 💾 ile mevcut WSDL + adresi bir adla kaydedin ("Test", "Canlı"…); üstteki combodan seçince ikisi birden dolar — ortamlar arası tek seçimle geçilir. Aynı adla kaydetmek profili günceller.
- **AI Yorumla** (yanıt başlığında): istek + yanıt zarfı yapay zekaya gider — başarılıysa dönen verinin özeti, Fault'taysa neden + istekteki sorumlu alan + pratik düzeltme önerisi. Gizlilik: yalnız ekranda zaten gördüğünüz zarflar gönderilir.

SP'den servis iskeleti üretme

Nesne gezgininde bir SP'ye sağ tık → 🔌 **Servis iskeleti üret (WCF)**: SP imzasından `ServiceContract` arayüzü + implementasyon + ADO.NET SP çağrısı içeren hazır C# dosyası yeni sekmede açılır — projenize kopyalayıp ad alanını ve bağlantı dizesini düzenlersiniz. OUTPUT parametreler gerçek `out` parametre olarak üretilir; dönüş, SP'nin ilk sonuç kümesini taşıyan

`DataTable`'dır (kendi modelinize eşlemek isterseniz Fill sonrası dönüştürün — iskeletin başındaki yorumda da yazar). Yalnız SQL Server'da görünür.

> **Bilinen sınırlar:** WS-Security / `wsHttpBinding` ve sertifikalı kimlik doğrulama desteklenmez (kapsam dışı — zarfı elle düzenleyebildiğiniz için birçok özel durum yine de çözülür); rpc-stili eski WSDL'ler çözümlemede uyarıyla atlanır. Çok girift XSD uçlarında örnek zarf "içeriği elle doldurun" yorumu bırakır — operasyon yine çağrılabilir.

29. Zaman Makinesi — Prova & Geri Al ⏪ SQL Server

SQLST'nin kaza önleme felsefesinin en ileri iki aracı (v15). İkisi de yalnız SQL Server'da çalışır ve birbirini tamamlar: **Prova** yazma işleminden *önce* etkisini gösterir; **Geri Al** ise COMMIT ettikten *sonra* geri dönüş kapısı bırakır.

🔍 Sorgu provası

Bir UPDATE/DELETE yazdınız ama çalıştırmadan önce "kaç kaydı, hangi kayıtları etkileyecek?" diye merak ediyorsunuz — çoğumuz bunun için önce elle bir SELECT atarız. `🔍 Prova` (araç çubuğunda, ya da `Ctrl+Shift+P`) bunu tek tuşa indirir:

- Sorgunuzun WHERE'i **salt SELECT'e** çevrilip çalıştırılır — **tablonuza tek harf dokunulmaz** (DELETE bile olsa yalnız sayılır ve gösterilir).
- Editörün üstünde bant çıkar: **"N satır etkilenecek"** + UPDATE'te **hangi kolonların değişeceği**. WHERE yoksa bant kırmızı uyarır: "TÜM TABLOYU etkileyecek".
- Etkilenecek ilk 10 satır sonuç grid'inde görünür. Kararınızı verince banttaki `▶ Şimdi çalıştır` ile sorgu olağan yoldan (WHERE'siz uyarısı, Güvenli Yazma dahil tüm korumalarla) koşar. Metni değiştirirseniz prova kendiliğinden söner.

> MVP kapsamı: tek ifadeli, doğrudan tabloyu hedefleyen UPDATE/DELETE. Çok ifadeli script, CTE'li (WITH…) ya da FROM/JOIN'li karmaşık DML'de prova "desteklenmiyor" der ve sizi engellemez — sorgu yine normal çalıştırılabilir.

⏪ Geri Al paketleri (uçuş kaydedici)

Sol raydaki `⏪ Geri Al`, "keşke o UPDATE'i çalıştırmasaydım" anının panzehiridir. **Güvenli Yazma modu açıkken** COMMIT ettiğiniz her UPDATE/DELETE'te, SQLST değişiklik uygulanmadan önce **etkilenen satırların eski halini** sessizce bir "geri al paketi"ne kaydeder.

- **Liste:** pencere solunda paketler en yeni üstte — fiil, tablo, satır sayısı, tarih. Seçince sağda paketi *doğuran sorgu* ile etkilenen satırların *eski değerleri* görünür.
- **Geri alma script'i:** `⏪ Geri alma script'i üret` paketten tersine çeviren DML üretir — UPDATE paketi → eski değerleri PK ile geri yazan UPDATE'ler; DELETE paketi → silinen satırları geri ekleyen INSERT'ler. Script **yeni bir sekmede açılır, çalıştırılmaz**: inceleyip Güvenli Yazma ile siz koşarsınız.

> **Sınırlar (bilinçli):** paket yalnız **Güvenli Yazma + COMMIT** yolunda alınır (ROLLBACK'te zaten veri değişmedi). Etkilenen satır **50.000'i** aşarsa paket alınmaz (sonuç mesajında belirtilir). Çok büyük kolonlar (varbinary/text/XML, rowversion) pakete girmez. Ters UPDATE için tablonun **birincil anahtarı** gerekir — yoksa eski değerler yalnız görüntülenir (elle kullanırsınız). Paketler ~30 gün ve toplam ~500 MB sınırında tutulur; eskiler kendiliğinden budanır. Uyarı: geri alma

> script'i de bir yazma işlemidir — o sırada satırlar başkasınca yeniden değişmiş olabilir; çalıştırmadan önce gözden geçirin.

30. Performans Değerlendirmesi SQL ailesi

Editör araç çubuğundaki `Performans değerlendir` (eski "Neden yavaş?"), sorgunun execution plan'ını tek bir **Türkçe darboğaz raporuna** çevirir — istediğinizde, tek tıkla. Öncelik sırasına göre renkli kartlar (Yüksek kırmızı / Orta amber / Bilgi gri):

- 🔑 **Eksik index** — sunucunun önerdiği CREATE INDEX script'iyle.
- 🔍 **Tam tarama** — index'siz okunan operatör, planın maliyet yüzdesiyle.
- 📊 **İstatistik tazeliği** — tahmin/gerçek satır sapması ≥10× ise (UPDATE STATISTICS önerisiyle).
- ⚠ **Sunucu uyarısı** — tempdb'ye taşma, örtük tip dönüşümü (Türkçe ipucuyla).

Rapordaki `Script'i aç`, düzeltmeyi (CREATE INDEX / UPDATE STATISTICS) **yeni bir sekmede açar — asla kendiliğinden çalıştırmaz**. Destekleyen motorda gerçek (ölçülü) plan alınır; yazma/DDL sorgusunda çalıştırmadan tahmini plana düşülür.

31. Excel ↔ Tablo Tam Eşitleme 🔄 SQL ailesi

📥 İçe Aktar penceresinde yazma kipini **"Tam eşitle (🔑 fark önizle → Güvenli Yazma)"** seçin. SQLST dosyayı hedef tabloyla **anahtara göre** kıyaslar ve farkı üç kovada gösterir:

- ➕ **Yeni** — Excel'de var, tabloda yok → INSERT.
- ✏️ **Değişen** — anahtar eşleşti, kolon farklı → UPDATE (eski→yeni yan yana).
- 🗑 **Silinecek** — tabloda var, Excel'de yok → DELETE (tam eşitleme; silme opt-in kutusuyla).

Script Üret INSERT/UPDATE/DELETE'i **yeni bir sorgu sekmesine** getirir ve **Güvenli Yazma** bandından geçirir — satır sayısını görüp COMMIT/ROLLBACK verirsiniz. Anahtar = hedef PK; identity/computed kolonlar doğru işlenir (identity INSERT'i IDENTITY_INSERT ile sarılır).

> **İpucu:** Excel'de yinelenen anahtar varsa önceden uyarılırsınız (çift INSERT/UPDATE kazasını önler). char(n) dolgusu ve kültür farkı (1.250,75) sahte "değişti" üretmez.

32. REST İstemcisi 🌐 Tüm motorlar

Sol raydaki 🌐 REST İstemcisi, bir HTTP/REST API'sini SQLST içinden test etmenizi sağlar — düzeni **Postman'dan tanıdığınız** gibidir: solda Koleksiyonlar ↔ Geçmiş, üstte renkli metod + URL + Gönder, altında Params · Kimlik · Başlıklar · Body · Değişkenler sekmeleri, en altta yanıt. Kurumsal proxy, **localhost muafiyeti** ve tarayıcı benzeri başlıklar hazırdır (kendi makinenizdeki servisler de sorunsuz). İstek ile yanıt arasındaki **ayırıcı çubuk** sürüklenir.

Adım adım

1. **Method** (GET yeşil · POST turuncu · PUT mavi · DELETE kırmızı) seçin ve **URL**'i yazın.
2. **Params — URL ile ÇİFT YÖNLÜ senkron:** URL'ye `?anahtar=değer&…` yazdıkça tablo kendiliğinden dolar; tabloda ekleyip değiştirdikçe URL güncellenir. ✓ **işaretini kaldırdığınız satır URL'den düşer ama listede kalır** (tekrar işaretleyince geri gelir). Her satıra not için **Açıklama** kolonu vardır; gönderimde değerler otomatik kaçışlanır.
3. **Kimlik sekmesi:** Yok / **Basic** / **Bearer** / **API Key** (Başlık ya da Query) — `Authorization` otomatik kurulur. **Body** sekmesinde anahtar/değer tablosu ile JSON editörü iki yönlü eşitlenir; 🔤 **Değişkenler**'deki {{Ad}} değerleri URL/başlık/gövdeye uygulanır (ortam kümeleri sol alttaki **Ortam**'dan kaydedilir/seçilir).
4. ▶ Gönder → yanıtın sağ üstünde **renkli rozetler**: durum kodu (2xx yeşil · 3xx sarı · 4xx/5xx kırmızı) + süre + boyut. Gövde **renklendirilmiş JSON**; **Başlıklar** ve **Çerezler** (Set-Cookie çözülmüş) ayrı sekmelerdedir. 4xx/5xx bir hata değil normal yanıttır — asıl bilgi gövdededir. Uzun isteği ■ Durdur keser.

Yardımcı üreticiler (URL'nin sağındaki ikonlar)

- 🤖 **AI üret:** doğal dille tarif edin ("JSONPlaceholder /users'a GET at") — method/URL/başlık/gövde dolar.
- 📋 **cURL yapıştır:** panodaki curl komutunu (DevTools: Copy as cURL) isteğe çevirir.
- 💻 **Kod üret:** bu isteği cURL / C# (HttpClient) / Python (requests) koduna döker.

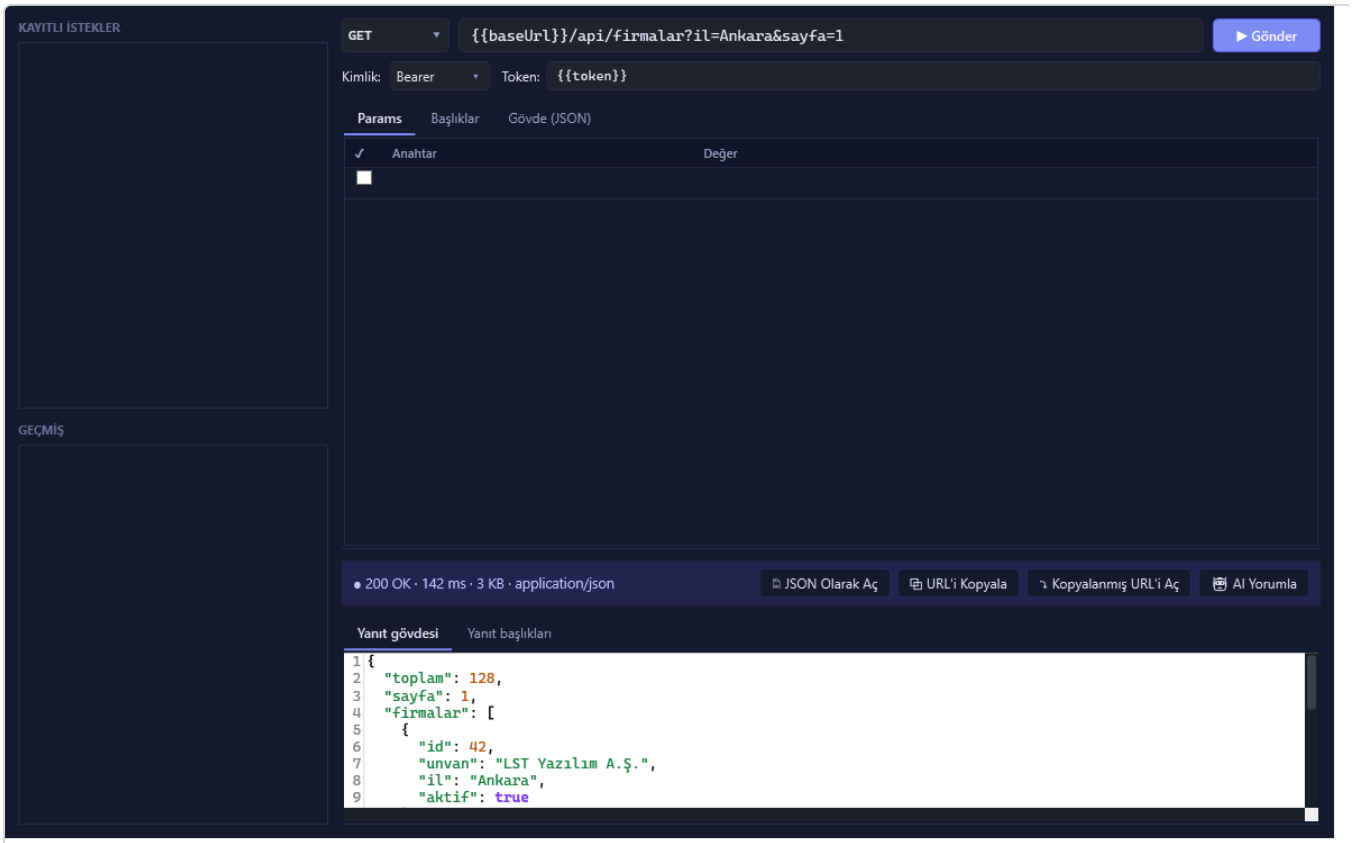
Yanıt araçları

Yanıtın altındaki sıra: 🗄 **Tabloya kaydet** (JSON'dan kolon çıkarıp tablo oluşturur + satırları aktarır) · 🔍 **DB ile karşılaştır** · 🔗 **Değişkene çıkar** (ör. `data.token` → {{token}}) · 🔎 **JSON path** · 🤖 **AI Yorumla**. Sağdaki ⋯ menüsünde: 📌 **Sabitle** + 🤖 **Diff yorumla** (iki yanıtın farkı), ⧉ **URL'i kopyala** / ↴ **Kopyalanmış URL'i aç**, 🗎 **Yanıtı ayrı pencerede aç**.

Koleksiyonlar + geçmiş

Sol panel sekmelidir: 📁 **Koleksiyonlar**'da kayıtlı istekler klasör başlıklarıyla listelenir (kaydederken ada `Grup/Ad` yazın — ör. `Kimlik/Token al`); çift tıkla yüklenir, sağ tık → 🗑 silinir. 🕘 **Geçmiş**'e her

gönderim düşer (method · URL · durum · süre; çift tıkla geri yükle).



Method + URL + Bearer token → Gönder → 200 OK · süre · boyut + renkli JSON yanıt; sağda JSON Olarak Aç · URL'i Kopyala · Kopyalanmış URL'i Aç · AI Yorumla.

> **Bilinen sınırlar:** Postman'daki test script'leri (JS assertion) kapsam dışıdır — SQLST'nin amacı "DB'nin yanında hızlı API deneme"dir (istek zincirleme için 🔗 Değişkene çıkar vardır). İkili (binary) yanıt gövdesi metin olarak gösterilir; çok büyük yanıtlarda süreyi ■ Durdur ile kesebilirsiniz.

33. LINQ ⇄ SQL 🔁 SQL ailesi

Sol raydaki 🔁 LINQ ⇄ SQL **iki yönlü** çevirmendir: C# LINQ'inizi bağlı olduğunuz motorun SQL'ine, T-SQL SELECT'inizi C# LINQ'ine çevirir — **kendi çeviri motorlarımızla, yapay zekasız ve çevrimdışı**. "Bu LINQ hangi SQL'i üretiyor?" ve "bu SQL'in LINQ karşılığı ne?" soruları tek ekranda kapanır. Ayrıştırma hazır kütüphaneyle (C# için Roslyn, SQL için ScriptDom — ikisi de MIT), **çeviri kuralları tamamen SQLST'nin**dir ve deterministiktir.

Adım adım

1. **SQL soldadır** — DB tarafı birincil çalışma alanı. SQL yazarken **ana editörün önerileri** buradadır: yazdıkça + `Ctrl+Space` ile şemadan tablo/kolon/anahtar sözcük tamamlama.
2. **SQL → LINQ:** soldaki T-SQL SELECT için ➜ LINQ'a Çevir (Shift+F5) → sağda **query-syntax LINQ** (çok JOIN'de en okunur biçim; Take/Skip/Distinct metod eki olarak). Şema gerekmez — tablolar SQL'de zaten yazılıdır.
3. **LINQ → SQL:** sağa LINQ'inizi yazın/yapıştırın — **metod zinciri** (`db.Musteriler.Where(m => …).Select(…)`) ya da **query syntax**; `var q = …;` kabuğu otomatik soyulur → ⬅ SQL'e Çevir (F5) → solda motorunuzun söz dizimiyle SQL (TOP / LIMIT / FETCH farkları dahil). Tablolar **aktif veritabanının şemasından** eşlenir — çoğul adlar (`db.Musteriler`) `Musteri` tablosuna çözülür.
4. ▶ Sekmede Aç ve Çalıştır → SQL yeni sorgu sekmesinde hemen koşar; SQL'i Kopyala panoya alır.

SQL → LINQ kapsamı

INNER + **LEFT JOIN** (→ `into … DefaultIfEmpty()` ; bileşik ON anahtarları anonim nesneyle), **virgüllü FROM** (→ çoklu `from`), **çok tablolu GROUP BY** (grup elemanı JOIN'li satırdır: toplamlar `g.Sum(x => x.t.Kol)` ; çakışan anahtar adları otomatik ayrıştırılır) + HAVING, **COUNT(DISTINCT …)**, **EXISTS** (→ `db.T.Any(x => …)`), **IN alt sorgusu** (→ `Select().Contains()`), **UNION / UNION ALL / INTERSECT / EXCEPT** (→ Union/Concat/Intersect/Except), **bağıntılı skaler alt sorgular** (toplamlı → `Count/Sum…` ; `TOP (1) kolon` → `FirstOrDefault`), **ORDER BY'da SELECT takma adı**, TOP/OFFSET-FETCH (→ Take/Skip), LIKE desenleri (basitler → StartsWith/EndsWith/Contains; karışıklar → `EF.Functions.Like` uyarıyla), IN, IS NULL, CASE (→ `?:`) ve geniş fonksiyon kümesi (UPPER/LEN/YEAR… + **ISNULL/COALESCE/NULLIF** → `??` , **CAST/CONVERT**, **SUBSTRING**, **DATEADD/DATEDIFF**).

Şema + FK farkındalığı (SQLST farkı)

- **Navigation üyeleri:** `s.Musteri.Ad` gibi bir erişim şemada kolon değilse **FK grafından** çözülür ve otomatik `JOIN` üretilir.
- **Koleksiyon navigation:** `m.Siparisler.Any(s => s.Tutar > 1000)` → `EXISTS (SELECT 1 …)` ; `m.Siparisler.Count()` → alt sorgu ile sayım.

- **Kolon doğrulaması:** şemada görünmeyen kolon çeviriyi durdurmaz ama **uyarı** olarak bildirilir (önbellek bayat olabilir).

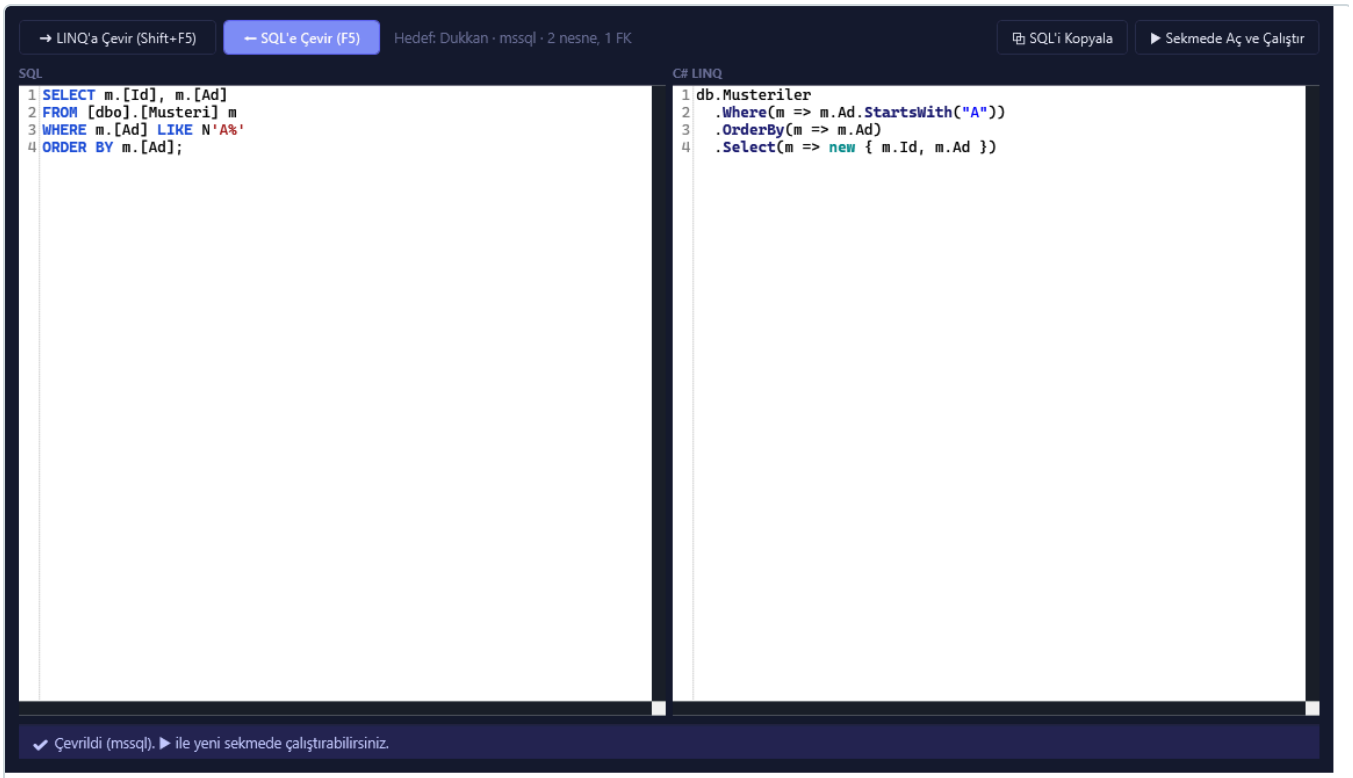
Desteklenen yapılar

`Where` · `Select` (anonim nesne, takma ad) · `OrderBy/ThenBy(Descending)` · `GroupBy` + `g.Key` / `Count` / `Sum` / `Min` / `Max` / `Average` (HAVING dahil) · `Join` (metod + query) · `Distinct` · `Take/Skip` (sayfalama, motoruna göre) · `First/Single` · `Count` · `Any` (EXISTS) · string metotları (`StartsWith/EndsWith/Contains` →LIKE, `ToUpper/ToLower/Trim/Substring/Replace/Length`) · tarih parçaları (`.Year/.Month/.Day…` , motoruna göre `YEAR()/EXTRACT()`) · `liste.Contains(x)` →IN · `?:` →CASE WHEN · `string.IsNullOrEmpty` · null karşılaştırmaları (IS NULL / IS NOT NULL).

> **İki yön SİMETRİKTİR (0.20.8):** SQL→LINQ'in ürettiği her kalıp LINQ→SQL'den geri döner — LEFT JOIN kalıbı (`join…into + DefaultIfEmpty`), çoklu `from` (CROSS JOIN), `db.T.Any(…)` → EXISTS, `Select().Contains()` → IN alt sorgusu, `??` → COALESCE, `EF.Functions.Like` → LIKE, `Union/Concat/Intersect/Except` → küme işleçleri. Gidiş-dönüş (SQL→LINQ→SQL) testle kanıtlıdır.

> **CTE'ler ve türetilmiş tablolar (0.20.8):** `WITH ad AS (…)` zincirleri (çoklu, birbirine başvuran) ve `FROM (SELECT …) x` alt sorguları artık çevrilir — her biri LINQ'ta **"ara sorguyu değişkene al"** kalıbına açılır: `var ad = (…);` + ana sorgu. Ters yön de simetriktir: `var` 'lı çok ifadeli LINQ metni SQL'e `WITH` zinciri olarak döner (gidiş-dönüş testle kanıtlı). Özyinelemeli CTE ve CTE kolon listesi net hatadır.

> **Sınırlar (bilerek):** her iki motor da tam karşılığı üretemeyeceği yapıyı **net bir hata mesajıyla** söyler, asla "yaklaşık" çeviri yazmaz (sessiz yanlış yasak). LINQ→SQL'de: `let` , blok gövdeli lambda, **dış değişkenler** (değeri yerine yazın). SQL→LINQ'ta: **RIGHT JOIN** (mesaj LEFT'e çevirmeyi önerir), **özyinelemeli** CTE, **pencere fonksiyonları** (ROW_NUMBER OVER…), TOP PERCENT/WITH TIES, DATEDIFF(month/year). MongoDB'de SQL üretilmez; `GETDATE()` → `DateTime.Now` ve DATEDIFF uyarıyla çevrilir (saat/sınır-sayımı farkları).



Solda SQL (önerili editör), sağda C# LINQ: StartsWith ↔ LIKE N'A%', projeksiyon ve sıralama birebir; ▶ ile sekmede çalıştırılır.

34. SQL Profiler 🔍 SQL Server

Sunucudaki **TÜM uygulamaların** sorgu trafiğini canlı izler — SSMS Profiler'ın karşılığı. Önemli fark: SSMS Profiler Microsoft tarafından kullanımdan kaldırılıyor; SQLST izlemeyi modern **Extended Events** altyapısıyla yapar (sunucuya çok daha az yük bindirir) ama size Profiler'ın bildiğiniz akışını sunar. Sol menüden `Profiler` ile açılır; yalnız SQL Server bağlantısında görünür.

Adım adım

1. **Şablon seçin: Standart** (tamamlanan Batch + RPC — SSMS'in Standard'ı) · **Yavaş sorgular** (yalnız süresi eşiği aşanlar; eşik ms cinsinden yanında) · **Hatalar** (kullanıcıya dönen gerçek hatalar, önem ≥ 11).
2. İsterseniz **Veritabanı** süzgeci seçin — sunucu tarafında süzülür; *(tümü)* = süzgeç yok.
3. `▶ İzlemeyi başlat`: SQLST sunucuda `SQLST_Profiler` adlı Extended Events oturumunu kurar ve olaylar 2-3 saniye içinde akmaya başlar. Bu işlem sunucuda **ALTER ANY EVENT SESSION** yetkisi ister — yoksa ekran ne isteyeceğinizi söyler.
4. Akışta her olay için: zaman · tür (Batch/RPC/Hata) · süre · CPU · okuma · satır sayısı · veritabanı · **uygulama adı** · kullanıcı · SPID · sorgu metni. Satırı seçince tam metin altta; sağ üstteki kutu akışı metne göre süzer (sorgu/uygulama/veritabanı/kullanıcı). Sorgu satırlarında `Sekmede aç` yakalanan sorguyu **yeni sorgu sekmesinde** açar (olayın veritabanı önseçili) — planına bakar, düzeltir, çalıştırırsınız.
5. `⏸ Duraklat` yalnız EKRAN akışını durdurur (oturum sunucuda sürer; sürdürünce birikenler gelir). `■ Durdur` izlemeyi bitirir ve oturumu **sunucudan siler** — SQLST arkasında oturum bırakmaz (sekmeyi kapatmak da temizler).
6. `Kopyala` / `⬇ CSV` ekrandaki akışı dışa verir.

🧠 Grupla — SQLST farkı

`🧠 Grupla` düğmesi akışı **imza görünümüne** çevirir: aynı sorgunun parametre **varyantları tek satırda** toplanır (Log Analizi'yle aynı imza motoru — `WHERE Id = 5` ile `WHERE Id = 9412` aynı gruptur). Her grupta canlı sayaçlar: **kez · ortalama süre · en yüksek süre · toplam CPU** + ilk/son görülme + örnek metin; en çok tekrarlayan üstte durur, olay geldikçe satırlar yerinde güncellenir. SSMS Profiler'ın basit gruplamasından farkı budur: parametre farkı sizi yanıltmaz — "hangi sorgu sunucuyu yiyor" tek bakışta. Ham akış arkada birikmeye devam eder; düğmeyi kapatınca kaldığınız yerdesiniz. Kopyala/CSV aktif görünümü verir (grup modunda grup satırları). Gruplar ekran tavanından **bağımsız** sayar — ham satır düşse de sayaçlar akışın başından beri doğrudur.

AI Yorumla

Asistan yapılandırılmışsa özet bandında `AI Yorumla` görünür: seçili olayı (metrikleriyle), grubu (kez/ortalama/en yüksek) ya da kilitlenmeyi (çözülmüş özet — ham XML gitmez) modele yorumlatır;

muhtemel neden + somut öneri (index/sorgu düzeltme) alırsınız. Yanıt alttaki 🤖 **AI yorumu** bölümünde açılır. Gizlilik kuralları AI Asistan bölümündekiyle aynıdır (Bölüm 25) — satır verisi gitmez, ekranda zaten görünen olay metni gider.

🕸 Kilitlenme (deadlock) şeması

Deadlock (kilitlenme) şablonuyla izlemeyi başlatın — sunucudaki her kilitlenme akışa **Deadlock** satırı olarak düşer. Satırı seçip `🕸 Şemayı aç` (ya da çift tık): süreçler üstte oval (⛔ **KURBAN kırmızı** — sunucunun geri aldığı süreç), çekişilen kaynaklar altta; **yeşil ok = sahip**, **turuncu kesikli ok = bekleyen** (kilit kipi etikette). Sürece tıklayınca çalıştırdığı sorgu altta görünür. `⬇ .xdl kaydet` raporu SSMS'in çift tıkla açtığı biçimde dosyalar — ekip arkadaşına gönderilebilir kanıt; `⧉ XML'i kopyala` ham raporu panoya alır. SSMS'te bu iş ayrı kurulum ister; burada tek şablon seçimidir.

> **Bilinçli sınırlar:** ekranda en çok **5.000 olay** tutulur (en eskiler düşer — durum satırı söyler; kalıcı kayıt için CSV; gruplar düşüşten etkilenmez). Akış ~2 sn aralıkla tazelenir. SQLST **kendi izleme trafiğini akışa yazmaz** (2 sn'lik okuma sorgusu ve bağlantı havuzu sıfırlamaları elenir) — gördüğünüz her satır gerçek uygulama trafiğidir; sorgu sekmelerinizden çalıştırdıklarınız görünmeye devam eder. Bu sürümde yalnız SQL Server; diğer motorların karşılıkları sonraki adımlarda.

35. Full-Text Search 🔎 SQL Server

Sol menüdeki Full-Text (yalnız SQL Server bağlantısında görünür), sunucunun tam metin arama gücünü tek ekrana toplar: **LIKE '%...%' her satırı tarar**; Full-Text index'li tabloda **CONTAINS kelime index'inden gider** — milyonluk tabloda saniyeler yerine milisaniyeler. Üç bölümdür: üstte **envanter**, solda **arama yardımcısı**, sağda **kurulum sihirbazı**.

Adım adım

1. **Yenile:** sunucuda FTS bileşeni var mı bakılır. **Yoksa** (LocalDB hiç desteklemez; Express'te "Advanced Services" gerekir) ekran ne isteyeceğinizi söyler — kurulum sunucu/DBA işidir.
2. **Envanter:** FULLTEXT INDEX'li tablolar — katalog, kolonlar (dilleriyle), değişiklik izleme ve **doldurma durumu** (ilk kurulumdan sonra "Tam doldurma sürüyor" görmek normaldir; bitince "Boşta"). Satıra sağ tık → 🗑 **silme script'i** (çalıştırmaz, sekmeye açar).
3. 🔎 **Ara:** tabloyu (envanterden tıklamak da seçer) ve kipi seçin: **Kelime/deyim** · **Önek** (`fatu*` — fatura/faturalama...) · **Yakınlık** (iki terim en çok N kelime arayla) · **Çekimler** (koştu/koşacak/koşuyor) · **Serbest metin** (FREETEXT — gevşek anlam eşleşmesi). **Skorlu** işaretlenirse sonuç RANK kolonuyla gelir — en alakalı üstte. ▶ Ara sorguyu yeni sekmede çalıştırır; sorgu düzenlenebilir SQL'dir, CONTAINS sözdizimini buradan öğrenirsiniz.
4. 🧰 **Kurulum sihirbazı:** FTS'i olmayan tablo → metin kolonlarını işaretle → dil (varsayılan **Turkish** — Türkçe kök bulma/çekim için) + katalog → 📜 Script üret. Script **çalıştırılmaz**; gözden geçirip **Güvenli Yazma açıkken** koşarsınız. Ön şart: tabloda TEK kolonlu + UNIQUE + NOT NULL bir anahtar index (tipik PK) — yoksa sihirbaz açıkça söyler.

🔎 Veri Arama ile birlikte

FTS index'i kurduğunuz tabloların meyvesini **Veri Arama** da kendiliğinden yer: SQL Server'da arama başlarken FTS'li tablolar tek sorguyla öğrenilir ve o tabloların FTS kolonları `=` yerine **CONTAINS** ile aranır — hem kelime index'inden gittiği için hızlıdır, hem de `=`'in asla bulamayacağı eşleşmeleri yakalar: aradığınız değer uzun bir açıklama/not kolonunun **içinde geçiyorsa** da bulunur. Böyle aranan tabloların sonuç etiketi ⚡ **FTS** taşır; FTS'siz tablolar eski yoldan aranmaya devam eder.

🛠 Yönetim — Index'ler · Kataloglar · Stoplist'ler

Ekranın üst bölümü üç alt sekmedir:

- **Index'ler:** envanterde her index'in **öğe sayısı**, **bekleyen değişiklik** sayısı, değişiklik izleme modu, doldurma durumu, **son doldurma** (tür + zaman; sürüyorsa "(sürüyor)"), stoplist'i ve etkin olup olmadığı görünür. Bir satır seçince alttaki şerit açılır: ▶ Tam doldur (tüm tablo baştan) · ↻ Artımlı doldur (yalnız değişen satırlar — tabloda **timestamp/rowversion** kolonu gerekir; yoksa düğme kapalıdır) · ⇪ Bekleyenleri işle (izleme MANUAL iken biriken değişiklikler) · ■ Durdur · **Değişiklik izleme** (AUTO: kendiliğinden · MANUAL: "Bekleyenleri

işle" ile · OFF: yalnız elle tam doldurma) · ⏸ Etkin / devre dışı · 🗑 Silme script'i. Doldurma sürerken ekran durumu **5 sn'de bir kendiliğinden tazeler** ("● canlı izleniyor"), bitince durur.

- **Kataloglar:** her kataloğun index sayısı, öğe sayısı, boyutu (MB) ve durumu. ⧉ Birleştir (REORGANIZE) parçalı index'leri toplar (hafif, çevrimiçi) · ★ Varsayılan yap · 📜 Yeniden kurma script'i (REBUILD — tüm index'ler baştan dolar; ağır iş olduğu için script sekmede açılır, çalıştırılmaz) · 🗑 Silme script'i (katalogdaki index'lerin silme satırları yorum olarak eklenir — bilerek açarsınız).
- **Stoplist'ler:** sistem listesi ve sizin tanımladıklarınız; seçilen listenin kelimeleri dile göre (varsayılan Türkçe). Bu kelimeler index'e girmez, aramada yok sayılır ("ve", "ile", "bir"…).

Doldurma, izleme, etkinlik ve katalog birleştirme/varsayılan işlemleri **onay sorar** ve hemen çalışır; **salt-okunur bağlantıda** kapalıdır (script düğmeleri yine çalışır).

> **Bilinçli sınırlar:** bu sürümde yalnız SQL Server (PostgreSQL/MySQL/Oracle'ın tam metin mekanizmaları bambaşkadır — sonraki adımlar). Stoplist'e kelime ekleme/çıkarma ve eşanlamlı (thesaurus) yönetimi kapsam dışı — stoplist'ler görüntülenir.

36. SQL Agent ⏱ SQL Server

Sol menüdeki SQL Agent (yalnız SQL Server bağlantısında görünür), sunucudaki zamanlanmış işleri (**job**) SSMS'teki "Job Activity Monitor" gibi tek ekranda gösterir: hangi job ne zaman koştu, başarılı mı oldu, hata verdiyse **hangi adımda ve hangi mesajla**, bir sonraki koşu ne zaman. Buradan job'u başlatabilir, durdurabilir, açıp kapatabilir ve başka sunucuya taşımak için script'ini alabilirsiniz.

Ekran

1. **Üst bant:** Agent durumu rozeti — **çalışıyor** (yeşil) · **durmuş** (sarı: job'lar görünür ama zamanı gelen hiçbiri çalışmaz) · **yok** (kırmızı: Express ve LocalDB'de Agent servisi bulunmaz; Azure SQL Database'de de yoktur). Yanında sunucu, sürüm ve **yetkiniz**. Job adında arama, **Otomatik yenile (10 sn)** ve ⟳ Yenile.
2. **Süzgeç çipleri:** Tümü · ❌ **Son koşusu hata** · ▶ **Şu an çalışan** · ⏸ **Kapalı** (sayılarıyla) ve kategori seçimi.
3. **Liste:** açık/kapalı · job · kategori · son koşu · **sonuç rozeti** ("Hata · adım 2", "▶ Çalışıyor · adım 1/3", "İptal edildi"…) · süre (çalışan job'da canlı) · sonraki koşu · sahip. Kapalı job'lar soluk görünür. Tarih ve süreler veritabanının verdiği biçimdedir (`2026-10-06 09:45:00`).
4. **Seçili job — Geçmiş:** her koşunun genel sonucu "(job)" satırı ve adım satırları; bir satırı seçince sağda **tam hata mesajı** çıkar.
5. **Adımlar:** tür (T-SQL, PowerShell, CmdExec…), veritabanı, başarıda/hatada ne yapılacağı ve yeniden deneme. Seçili adımın komutu sağda görünür; T-SQL adımını Sorgu sekmesinde aç ile kendi veritabanında açıp deneyebilirsiniz.
6. **Zamanlamalar:** Türkçe cümleyle — "Her gün 02:00", "Hafta içi her gün, 08:00–18:00 arası her 15 dakikada bir", "Her ayın ilk pazartesisi 07:00".

İşlemler

- ▶ Başlat ▾ — **Baştan başlat** ya da Adımlar'da seçtiğiniz **adımdan başlat**. Her işlem önce onay sorar.
- ■ Durdur — yalnız job çalışırken açıktır; sürmekte olan adım iptal edilir.
- ⏸ Kapat / ▶ Aç — kapalı job zamanı gelince çalışmaz (elle başlatılabilir). Agent durmuşken de yapılabilir.
- 📜 Script olarak al — job'u adımları ve zamanlamalarıyla yeniden oluşturan script'i yeni sorgu sekmesinde açar, **çalıştırmaz**. Başka sunucuya taşımak ya da yedeklemek için kullanılır; script tek transaction'dır, hata olursa yarım job bırakmaz.

+ Yeni job · ✎ Düzenle · ⧉ Kopyasını oluştur — sihirbaz

Üst banttaki + Yeni job ve seçili job'un yanındaki ✎ Düzenle / ⧉ Kopyasını oluştur dört adımlı bir sihirbaz açar. Sihirbaz job'u **kendisi oluşturmaz**: script üretir ve yeni sorgu sekmesinde açar;

gözden geçirip siz çalıştırırsınız.

1. **Genel:** job adı, açık mı, kategori (listeden seçin ya da yeni ad yazın — yoksa oluşturulur), sahip (varsayılan: bağlı kullanıcı), açıklama.
2. **Adımlar:** ekle · çoğalt · ↑↓ ile sırala · sil. Her adımın adı, veritabanı, komutu, başarıda ve hatada ne olacağı ("Sonraki adıma geç", "Başarıyla çık", "Hatayla çık", "Belirli adıma git") ve yeniden deneme sayısı/aralığı. Sırayı değiştirince "N. adıma git" hedefleri kendiliğinden doğru numarayı alır. `📋 Açık sorgu sekmesinden al` komutu açık bir sekmeden kopyalar; `✓ Söz dizimini denetle` komutu **çalıştırmadan** derler (SET NOEXEC) ve hatanın satırını gösterir. Adım türü T-SQL'dir; mevcut job'daki PowerShell/CmdExec gibi adımlar salt görüntülenir ve aynen korunur.
3. **Zamanlama:** bir kez · günlük · haftalık (günleri seçerek) · aylık (ayın N. günü ya da "ayın ilk pazartesisi") · Agent başlarken · CPU boştayken; gün içinde bir kez ya da "her N dakika/saat, 08:00–18:00 arası". Tarihler `yyyy-MM-dd`, saatler `SS:dd`. Altta Türkçe cümle canlı görünür. Zamanlama eklemezseniz job yalnız elle başlatılır.
4. **Özet ve script:** üretilecek script'in önizlemesi.

Alt çubuk eksikleri anında yazar (boş ad, adımsız job, tekrar eden adım adı, olmayan adıma yönlendirme, haftalıkta gün seçilmemesi, bitişi başlangıçtan önce zamanlama…); hata varken `📜 Script üret` kapalıdır. Uyarılar (ör. adımın hatada kendine dönmesi) script'i engellemez. **Düzenle** script'i job'u **silmeden** günceller — job'un kimliği ve geçmişi korunur; adımlar ve zamanlamalar yeniden yazılır.

> **Yetki ve güvenlik:** job'ları görmek için msdb'de **SQLAgentReaderRole** (tümü), **SQLAgentUserRole** (yalnız kendi job'larınız) ya da sysadmin gerekir; yetkiniz yoksa ekran "job yok" demez, "görme yetkiniz yok" der. **Salt-okunur bağlantıda** başlat/durdur/aç-kapat kapalıdır. Bu sürümde yalnız SQL Server; operatör/e-posta bildirimi ve T-SQL dışı adım oluşturma kapsam dışı.